

---

Vorlesungsskript

# Datenanalyse in der Physik

---

Alexander Eichler, Martin Kroner, Carlos Cotrini, Marcel Lüthi  
Laura Völker, Uwe von Lüpke, Mario Sauppe, Güney Tekin,  
Konstantin Herb

ETH Zürich, FS 2024

# Inhaltsverzeichnis

# Kapitel 1

## Einführung in Python

### 1.1 Warum Python?

**Python ist relativ einfach:** Die schlanke, leicht lesbare Syntax von Python macht die Sprache ideal zum Skripten für Datenauswertung.

**Python ist eine Interpretersprache:** Grundsätzlich muss jeder Programmier-Code zunächst in Maschinensprache übersetzt werden, damit der Prozessor die im Code enthaltenen Anweisungen ausführen kann. Dies kann entweder mittels Compiler oder Interpreter geschehen. Viele Sprachen wie zum Beispiel C++ arbeiten mit Compilern. Hierbei wird der gesamte Quellcode zunächst in Maschinensprache übersetzt. Die dadurch generierte Datei (Executable) kann prinzipiell mehrmals ausgeführt werden. Python hingegen ist eine Interpretersprache. Ein Interpreter übersetzt den Quellcode zur Laufzeit, also Zeile für Zeile, ohne eine ausführbare Datei anzufertigen. Wird dabei in einer Zeile ein Fehler entdeckt, stoppt der Interpreter augenblicklich. Das ist vor allem zum Debuggen von Code hilfreich.

**Python ist Open-Source:** Im Gegensatz zu z.B. MATLAB ist Python kostenlos (d.h. open-source) verfügbar. Dadurch hat sich eine grosse Nutzergemeinde entwickelt und es sind viele hilfreiche Zusatzpakete entstanden. Zudem existiert online eine ausführliche Dokumentation. Die Verwendung von Python ist auch nicht an eine zeitlich begrenzte Lizenz gebunden.

**Python ist weit verbreitet:** In der Industrie und in wissenschaftlichen Laboren wird Python für zahlreiche Anwendungen genutzt. Sie können Ihr erworbenes Wissen also später direkt anwenden.

### 1.2 Anaconda und Jupyter Notebooks

Wir empfehlen, Anaconda als Python-Distribution zu verwenden. Anaconda kann unter <https://www.anaconda.com/products/distribution> installiert werden. Mit Anaconda wird auch Jupyter Notebooks mitinstalliert; das ist eine web-basierte interaktive Umgebung, mit der Jupyter-Notebook-Dokumente erstellt werden können. In Jupyter Notebooks ist der Programmier-Code in sogenannte Zellen unterteilt, die entweder Kommentare (Markdown Typesetting) oder Code (Python) enthalten. Die Zellen können separat ausgeführt werden, so dass beispielsweise eine zeitintensive Berechnung nicht jedes Mal wiederholt werden muss, um ein Detail an einem Plot zu ändern.

## 1.3 Erste Schritte

1

Für dieses Unterkapitel ist ein Jupyter-Notebook verfügbar. Siehe `Python.ipynb`

### 1.3.1 Bibliotheken

Eine Bibliothek ist eine Sammlung von Funktionen oder Klassen, die eine Programmiersprache um weitere Befehle erweitert. Zusätzlich zur generell verfügbaren Standardbibliothek können in Python-Skripten weitere Bibliotheken importiert werden. Im Rahmen dieser Vorlesung ist vor allem die numpy-Bibliothek relevant, die Datenstrukturen zur Handhabung von Vektoren und Matrizen sowie Funktionen für numerische Berechnungen bietet. Sie kann mit dem Befehl

```
import numpy as np
```

importiert werden. Hierbei ist `np` das Alias, unter dem im folgenden Code numpy-Funktionen aufgerufen werden können – so spart man sich das ständige tippen von “numpy” vor jedem Befehl. Zum Beispiel kann der in der numpy-Bibliothek gespeicherte Wert von  $\pi$  aufgerufen werden als

```
np.pi
```

Des Weiteren werden wir zur Darstellung von Daten die matplotlib-Bibliothek verwenden, die das Plotten von Daten in einfachen Scatterplots oder Histogrammen mit wenigen Befehlen ermöglicht. Häufig wird nur das Pyplot-Submodul der Bibliothek importiert.

```
import matplotlib.pyplot as plt
```

### 1.3.2 Datentypen und Variablen

#### Wichtige Eingebaute Datentypen

Die untenstehende Tabelle liefert einen Überblick über die wichtigsten Datentypen, die mit der Standardbibliothek von Python verfügbar sind. Einige dieser Datentypen (z.B. Ganzzahlen, Gleitkommazahlen und Strings) kennen Sie sicherlich schon von anderen Programmiersprachen. Vor allem die etwas komplizierteren Datentypen unterscheiden sich allerdings je nach Programmiersprache (z.B. ist in MATLAB der sogenannte cell-array ein Datentyp, der in Python nicht verfügbar ist).

| Datentyp                                     | Abkürzung | Beispiel | Informationen |
|----------------------------------------------|-----------|----------|---------------|
| Ganzzahl (engl: Integer)                     | int       | 4        |               |
| Gleitkommazahl (engl: Floating-Point Number) | float     | 4.0      |               |
| Komplexe Zahl (engl: Complex Number)         | complex   | 4 + 4j   |               |
| Boolean                                      | bool      | False    |               |

<sup>1</sup>Zum Erlernen von Python sind zahlreiche Online Tutorials verfügbar. Siehe [w3schools.com](http://w3schools.com) oder [realpython.com](http://realpython.com). Auch die Dokumentationen der einzelnen Bibliotheken, wie etwa [numpy.org](http://numpy.org) oder [matplotlib.org](http://matplotlib.org) können hilfreich sein.

|                         |          |                                                                 |                                                                                                                                                                                                                                                                                |
|-------------------------|----------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Null                    | NoneType | <code>None</code>                                               |                                                                                                                                                                                                                                                                                |
| String                  | str      | <code>"hello"</code>                                            |                                                                                                                                                                                                                                                                                |
| Liste (List)            | list     | <code>['John', 30, [1.85, 70]]</code>                           |                                                                                                                                                                                                                                                                                |
| Tupel (Tuple)           | tuple    | <code>('John', 30, [1.85, 70])</code>                           | Sehr ähnlich zu Listen (z.B. erfolgt das Auslesen einzelner Elemente mit analogem Syntax), allerdings mit dem entscheidenden Unterschied, dass Tupel unveränderliche (sogenannte immutable) Objekte sind; d.h. es können nicht einzelne Elemente eines Tupel verändert werden! |
| Wörterbuch (Dictionary) | dict     | <code>{'name': 'John', 'Alter': 30, 'Masse': [1.85, 70]}</code> | Assoziative Listen; die Indizes sind hier keine Zahlen, sondern Stichwörter (Keywords)                                                                                                                                                                                         |

Wie wir im Folgenden sehen werden, können wir den Datentyp eines Objekts jederzeit sehr einfach mit der sogenannten `type`-Funktion überprüfen.

### Zuweisung von Variablen

In vielen Programmiersprachen, wie zum Beispiel C++, müssen Variablen zunächst mit einem entsprechenden Datentyp deklariert werden, bevor eine Zuweisung erfolgen kann. In Python verhält sich das anders. Hier ist der Datentyp nicht an die Variable gebunden, sondern an deren Wert. Variablen können daher direkt zugewiesen werden und der Datentyp des zugewiesenen Wertes kann dynamisch aktualisiert werden. Das folgende, einfache Beispiel weist der Variable `A` den Wert 5 zu.

```
A = 5
```

Wir können nun mit der `type`-Funktion überprüfen, von welchem Datentyp die Variable `A` ist

```
type(A)
```

und erhalten den Output `< class int >`, `A` wurde also automatisch als Integer gesetzt. Wenn wir `A` hingegen wie folgt definieren:

```
A = 5.0
```

wird `A` automatisch als Gleitkommazahl deklariert, der Output unserer Typ-Prüfung wäre `< class float >`. In Jupyter Notebooks sind Variablen-Definitionen global, d.h. sobald die entsprechende Zelle ausgeführt wird ist die Variable im gesamten Notebook verfügbar. Es ist somit auch möglich, Variablen in einer weiteren Zelle einfach zu überschreiben.

### Arrays

Wir werden im Rahmen dieser Vorlesung Daten hauptsächlich als numpy-Arrays speichern und

nicht als die in der Standardbibliothek von Python verfügbaren Listen. Numpy Arrays sind Vektoren beliebiger Dimensionalität. Sie können mit einem einfachen Befehl generiert werden. Beispielsweise können wir einen eindimensionalen Vektor definieren, der die Zahlen “1,2,3,4” enthält, und ihn der Variable *a* zuweisen mittels

```
a = np.array([1, 2, 3, 4]) # ein-dimensionaler Array
```

Analog können wir auch mehrdimensionale Arrays definieren, zum Beispiel einen zweidimensionalen Array mit zwei Zeilen und vier Spalten

```
b = np.array([[1, 2, 3, 4], [5, 6, 7, 8]]) # zwei-dimensionaler Array
```

Die Dimensionalität (engl: Shape) eines Arrays kann sehr einfach als Tupel ausgelesen werden.

```
print(b.shape)
```

Im Falle von unserem Array *b* wäre der Output dieses Befehls (2,4). Häufig ist es praktisch, Numpy Arrays einer vorgegebenen Dimensionalität (*m,n*) zu generieren, die nur Nullen oder Einsen enthalten.

```
a = np.zeros(4) # eindimensionaler Array mit Nullen
b = np.ones((4,2)) # zwei-dimensionaler Array mit Einsen
```

Ebenfalls können gleichmässig verteilte Zahlenwerte über ein gewisses Intervall generiert werden (im unten stehenden Beispiel 5 Werte zwischen 0 und 10).

```
c = np.linspace(0,10, 5)
```

Spezifische Elemente von Arrays können mit den entsprechenden Indizes in eckigen Klammern aufgerufen werden. Die Indizierung von Arrays beginnt (genau wie in C++) bei 0, das erste Element vom oben definierten Array *a* können wir deshalb auslesen mittels

```
print(a[0])
```

Das letzte Element eines Arrays kann mit dem Index -1 aufgerufen werden. Ebenfalls können ganze Bereiche von Arrays aufgerufen werden, indem wir den ersten und letzten Index des Bereiches mit Doppelpunkt getrennt angeben.

```
print(a[0:2])
```

### Zusatz für interessierte Studenten: Pass By Assignment

Vom Programmieren in anderen Sprachen kennen Sie möglicherweise bereits die Konzepte *pass by reference* und *pass by value*. Python verwendet stattdessen *pass by assignment*. Abhängig vom Objekt-Typ ergeben sich damit die gleichen Resultate, die auch mit *pass by reference* und *pass by value* erzielt worden wären. Für sogenannte *immutable* Objekt-Typen (also z.B. int, float, bool, str, tuple und complex) ist das Resultat von *pass by assignment* identisch zu *pass by value*. Im Falle von sogenannten *mutable* Objekt-Typen (also z.B. list, set und dict) können wir dasselbe Resultat wie bei *pass by reference* erzielen, solange wir nur die Inhalte (content) der Objekte modifizieren.

## 1.3.3 Schleifen und Funktionen

### For-Loops

Während viele Programmiersprachen sogenannte Schleifen (engl: Loops) über Sonderzeichen (z.B. geschweifte Klammern) definieren, müssen diese in Python durch Einrückung erzeugt werden. Während das zu Beginn vielleicht ein wenig umständlich erscheinen mag, verbessert es vor allem in längeren und komplexeren Code-Abschnitten die Übersichtlichkeit. Der folgende Beispielcode beschreibt einen einfachen For-Loop, der über über alle Elemente unseres numpy-Arrays *c* läuft und diese ausgibt (“ausdruckt”).

```
for ci in c:
    print(ci)
```

Das print-Statement ist im For-Loop enthalten, da die entsprechende Zeile eingerückt ist. Eine mögliche Alternative, die genau dieselbe Funktion erfüllt wie der obige For-Loop, wäre:

```
for i, ci in enumerate(c):
    print(c[i])
```

## Funktionen

Funktionen sind ein fundamentales Werkzeug beim Schreiben von Code. Im Folgenden wollen wir eine kleine Einführung zu Funktionen in Python geben. Sie ist gezielt einfach und kurz gehalten, beinhaltet aber alles, was Sie für den Start benötigen. Die Definition einer Python-Funktion beginnt mit 'def', gefolgt vom Funktionsnamen und, falls benötigt, den zu übergebenden Argumenten in der Klammer. Analog zu for-Schleifen ist der zur Funktion gehörende Code dadurch definiert, dass er eingerückt ist.

```
def function_name(parameter1, parameter2, keywordargument=initialvalue):
    """Docstring"""
    #some code
    return return_value1, return_value2
```

Dabei können parameter1, parameter2 und keywordargument beliebige Datentypen sein, z.B. int, np.ndarray oder str. Der Unterschied zwischen keywordargument und parameter1 oder parameter2 ist, dass die Parameter die Reihe nach zugewiesen werden (das erste Argument wird dem ersten Parameter zugewiesen usw.), während Keyword Arguments auch über das Keyword zugewiesen werden können. Zusätzlich können Keyword Arguments einen Standardwert haben (in unserem pseudo-Code wäre dieser Wert initialvalue). Das return Statement ist nur notwendig, wenn man einen Wert zurückgeben will, und kann ansonsten weggelassen werden. Es können eine oder mehrere mit Kommas separierte Variablen ausgegeben werden und sie können beliebige Datentypen annehmen. Eine Besonderheit von Python-Funktionen ist, dass sie auch auf Variablen ausserhalb der Funktionsdefinition zugreifen können. Dies kann ein verlockend einfacher Weg sein, um Variablen an eine Funktion zu übergeben. Es ist aber ratsam, die benötigten Variablen über die Funktionsdefinition zu übergeben. Besonders bei verschachtelten Codes kann es schnell vorkommen, dass eine Funktion nicht mehr richtig funktioniert, weil eine Variable ausserhalb der Funktionsdefinition geändert wurde.

### 1.3.4 Laden und Speichern von Dateien

Häufig wollen wir Daten analysieren, die mit einem Computerprogramm oder mit einem Messgerät aufgenommen worden sind und die in einer Datei vorliegen. In diesen Dateien können die Daten in verschiedenen Formen gespeichert sein. Das Dateiformat ist üblicherweise in der Endung des Namens der Datei charakterisiert. Wir beschränken uns hier ausschliesslich auf Textdateien (d.h. Endung .txt), in denen die Daten im ASCII-Format vorliegen. Den Inhalt dieser Dateien wollen wir in Python laden und einer Variable (z.B. einem Array) zuweisen. Nach der Analyse wollen wir diese Daten wiederum in einer neuen Datei abspeichern.

Der Inhalt einer Messdaten-Datei besteht üblicherweise aus einem Header mit Metadaten und den Messdaten selbst. Der Header ist dazu da, die Messung zu beschreiben, so dass man später nachvollziehen kann, zu welchem Experiment die Daten gehören. Ausserdem enthält er normalerweise eine Erklärung zur Struktur der Daten, z.B. was die einzelnen Zeilen und Spalten bedeuten. Beim Laden der Daten müssen wir den Header überspringen. Der folgende Beispielcode generiert zunächst einen kleinen Datensatz, speichert diesen einmal ohne und einmal mit Header ab und liest die entsprechenden Files dann wieder ein.

```
x1 = np.linspace(0, 2, 101)
y1 = x1**2
```

```

np.savetxt('DataOut.txt', (x1, y1), delimiter = ',') # ohne Header
np.savetxt('DataOutHeaded.txt', (x1, y1), delimiter = ', ', header='Beispieldatei
    ') # mit Header

(x2, y2) = np.loadtxt('DataOut.txt', delimiter = ',')
(x3, y3) = np.loadtxt('DataOutHeaded.txt', delimiter = ', ', skiprows =1)

```

*Achtung:* Um auf eine Datei zugreifen zu können, müssen wir ihren Speicherort in der Verzeichnisstruktur des Computers kennen. Im einfachsten Fall liegt die Datei in unserem Arbeitsverzeichnis, dem sogenannten *current working directory*, in dem auch die Datei mit unserem Programmcode abgespeichert ist. Den Pfad zu unserem Arbeitsverzeichnis können wir sehr einfach mit dem unten stehenden Code herausfinden.

```

import os
#Ermitteln des current working directory und sofortige Ausgabe
print(os.getcwd())

```

Wenn unsere Text-Datei, auf die wir zugreifen können, in unserem Arbeitsverzeichnis liegt, können wir sie direkt laden. Falls die Datei an einem Speicherort ausserhalb unseres Arbeitsverzeichnisses gespeichert ist, müssen wir den Pfad zu diesem Speicherort berücksichtigen und den folgenden modifizierten Code verwenden.

```

# in der Datei DataOut.txt sind unsere Daten abgespeichert
# der Pfad zu dieser Datei ist in der Variable path gespeichert
np.loadtxt(os.path.join(path, 'DataOut.txt'))

```



# Kapitel 2

## Die Messung als Zufallsprozess

### 2.1 Modell und Experiment

In der Physik ist es oft weder möglich noch notwendig, die gesamte Komplexität eines Sachverhalts zu erfassen. Man verwendet daher Modelle: vereinfachte, mathematische Beschreibungen, die eine ausreichende Grundlage bieten, um Vorhersagen über das Verhalten eines bestimmten Systems treffen zu können. Ein besonders wichtiges Beispiel für ein physikalisches Modell ist der sogenannte getriebene und gedämpfte harmonische Oszillator. Mit diesem Modell kann man zahlreiche Phänomene näherungsweise beschreiben, zum Beispiel die Bewegung des Planeten um einen Stern, die Bewegung einer Schaukel oder den Strom in einem RLC-Stromkreis. Mathematisch kann der Oszillator beschrieben werden als

$$\ddot{x} + \Gamma \dot{x} + \omega_0^2 x = \frac{F(t)}{m} . \quad (2.1.1)$$

Hierbei ist  $x$  die Auslenkung,  $\Gamma$  die Abklingkonstante,  $\omega_0$  die Resonanzfrequenz multipliziert mit  $2\pi$  (die sogenannte Kreisfrequenz),  $F$  die Kraft,  $t$  die Zeit und  $m$  die Masse. Wenn die Kraft durch eine cosinusförmige Oszillation mit konstanter Amplitude und Phase gegeben ist,

$$F(t) = F_0 \cos(\omega t) . \quad (2.1.2)$$

können wir für jede Treibfrequenz  $\omega$  eine zeitunabhängige Lösung erhalten:

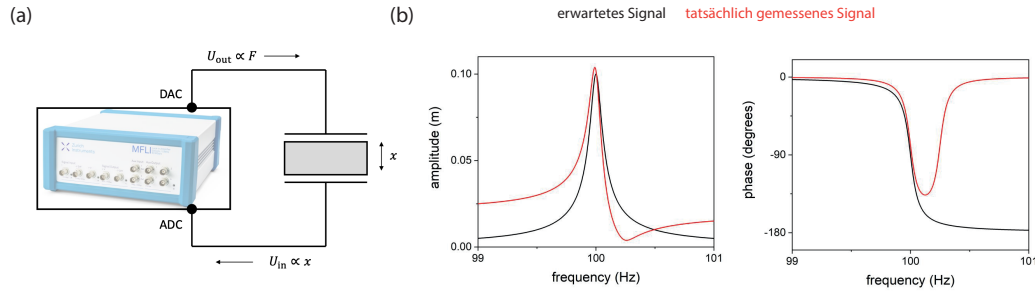
$$x(\omega) = \frac{F(\omega)/m}{\omega_0^2 - \omega^2 + i\omega\Gamma} . \quad (2.1.3)$$

Daraus lassen sich schliesslich die Amplitude  $x_0(\omega) = |x|$  und die Phase  $\phi(\omega) = \frac{\text{Im}[x]}{\text{Re}[x]}$  des Systems berechnen.

Um die Korrektheit eines Modells zu überprüfen, wird ein Experiment benötigt. Als einfaches Beispiel betrachten wir hier einen mit einem Quarzkristall gefüllten Kondensator. Ein Quarzkristall ist ein sogenanntes Piezomaterial, in dem das Anlegen einer elektrischen Spannung  $U_{\text{out}}$  eine mechanische Spannung erzeugt. Eine periodische Spannung nahe der mechanischen Resonanzfrequenz des Quarzkristalles führt deshalb zu einer Schwingung. Diese Schwingung ihrerseits induziert wiederum eine Spannung  $U_{\text{in}}$  im Kondensator, die wir messen können. Dieses System wollen wir als einen getriebenen und gedämpften harmonischen Oszillator mit  $U_{\text{in}} = x$  modellieren.

Um unser Modell zu verifizieren, führen wir ein einfaches Experiment durch, bei dem wir die Treibfrequenz einer angelegten Spannung über die Resonanzfrequenz des Quarzkristalls streichen und die Amplitude und Phase von  $U_{\text{in}}$  aufzeichnen. Ein Vergleich der theoretisch erwarteten und gemessenen Kurven zeigt auf, dass wir das erwartete Verhalten des Oszillators nicht vollständig

reproduzieren können (siehe Figur ??). Der Grund hierfür ist, dass das aufgezeichnete Signal zusätzlich zu der durch die Schwingung des Kristalls induzierten Spannung auch eine zweite Spannungskomponente enthält. Die zweite Komponente entsteht durch die direkte kapazitive Kopplung zwischen den Kondensatorplatten und ist unabhängig von der Quartzschwingung. Dieses zusätzliche Signal können wir korrekt modellieren, indem wir unser Modell durch eine komplexe, konstante Zahl  $C$  erweitern. Häufig realisieren wir durch ein Experiment also, dass wir unser Modell anpassen müssen, um das Verhalten eines Systems adäquat beschreiben zu können.



**Abbildung 2.1:** (a): Schematische Visualisierung des Versuchsaufbaus. (b): Erwartetes (schwarz) und tatsächlich gemessenes Signal (rot) für Amplitude (links) und Phase (rechts) von  $U_{in}$ .

Ein Modell kann allerdings niemals bewiesen, sondern immer nur widerlegt werden. Soll heißen: Auch wenn unser Experiment keine Lücken im Modell aufdeckt, könnte es immer noch ein anderes, alternatives Modell geben, das die gleiche Beobachtung erklären würde. Deckt unser Experiment hingegen eine Lücke auf, ist das Modell widerlegt worden und muss verworfen, beziehungsweise angepasst werden. Ein wichtiger Bestandteil dieses Prozesses, des Überprüfens eines Modells anhand eines Experiments, ist selbstverständlich die Analyse der gemessenen Daten. Kein Experiment ist komplett fehlerfrei, doch idealerweise sollten wir zwischen Fehlern des Modells und Fehlern des Experiments unterscheiden können. In diesem Kapitel wollen wir uns daher im Detail mit Messfehlern beschäftigen.

## 2.2 Messfehler

Für dieses Unterkapitel ist ein Jupyter-Notebook verfügbar. Siehe [Messfehler.ipynb](#)

### 2.2.1 Was ist ein Messfehler?

Um zu verdeutlichen, was ein Messfehler ist, wollen wir zunächst ein einfaches Beispiel betrachten. Wir messen mit dem Beschleunigungssensor eines Smartphones die Erdbeschleunigung  $g$ .<sup>1</sup> Da der Wert der Erdbeschleunigung bereits mit sehr hoher Genauigkeit gemessen wurde, erwarten wir schon vor der Messung ein bestimmtes Messergebnis,  $\tilde{x}$ . Dieser erwartete Wert  $\tilde{x}$  ist eine Abschätzung des tatsächlichen Werts von  $g$ , den wir (grundsätzlich) nicht kennen. Während unserer ersten Messung von  $g$  erhalten wir nun einen Wert  $x_1 \neq \tilde{x}$ . Der Abstand zwischen  $x_1$  und  $\tilde{x}$  ist der Fehler  $e$  unserer Messung:

$$e = \tilde{x} - x_1. \quad (2.2.1)$$

Ein solcher Messfehler kann verschiedene Ursachen haben. Entweder kann bei der Messung selbst ein Fehler auftreten, z.B. durch Ablesungenauigkeit, der Messung des falschen Parameters oder

<sup>1</sup>Dafür kann zum Beispiel die App Phyphox verwendet werden, Download möglich auf [phyphox.org](http://phyphox.org)

der ungenügenden Kalibration eines Messgerätes. Oder aber die Messung wird beeinflusst von (unbekannten) äusseren Faktoren wie z.B. Temperatur oder Luftfeuchtigkeit.

In jedem Fall gibt es bei einem Messvorgang für die gemessene Grösse einen tatsächlichen Wert, der - wie oben bereits erwähnt - grundsätzlich unbekannt ist. Der erwartete Wert  $\tilde{x}$  kann wie z.B. hier ein Literaturwert sein, oder ein theoretisch vorhergesagter Wert. Manchmal haben wir jedoch keine genaue Erwartung an den zu messenden Wert und können somit auch nicht direkt erkennen, ob unsere Messung stark vom erwarteten Wert abweicht. Somit müssen wir eine Möglichkeit finden, die Genauigkeit unserer Messung abzuschätzen und den Messfehler experimentell zu charakterisieren.

### 2.2.2 Systematische und zufällige Fehler

Im Folgenden wollen wir zwischen systematischen und zufälligen Messfehlern unterscheiden. Systematische Abweichungen sind konstante oder sich auf der Zeitskala unseres Experimentes nur sehr langsam ändernde Abweichungen des gemessenen Wertes vom tatsächlichen Wert. Solche systematischen Fehler können z.B. durch die falsche Kalibration einer Messapparatur zustande kommen. Sie können manchmal mittels einer Kontrollmessung einer bekannten Grösse in derselben Messapparatur aufgedeckt und eliminiert werden. Zufällige Fehler hingegen sind schnell fluktuierende Abweichungen, die bei jeder Durchführung des Experiments variieren und in der Regel nicht durch frühere Beobachtungen vorhergesagt werden können. Im Gegensatz zu systematischen Messfehlern sind zufällige Messfehler deutlich schwieriger auszuschliessen. Sie können lediglich reduziert, aber nicht vollständig eliminiert werden. Eine Messung, die zu Daten führt, die keine zufälligen Fehler zeigen, sollte misstrauisch machen.

### 2.2.3 Statistische Analyse durch wiederholtes Messen

Der Messfehler lässt sich durch wiederholtes Messen  $x_1, x_2, x_3, \dots, x_N$  unserer Messgrösse statistisch analysieren. Aus unseren Messergebnissen, unserer sogenannten Stichprobe, können wir einen **empirischen Mittelwert**

$$\bar{x} = \frac{1}{N} \sum_{n=1}^N x_n \quad (2.2.2)$$

und eine **empirische Varianz**

$$\Delta_x^2 = \frac{1}{N-1} \sum_{n=1}^N (x_n - \bar{x})^2 \quad (2.2.3)$$

berechnen. Häufig wird auch die empirische Standardabweichung  $\Delta_x$ , die Quadratwurzel der empirischen Varianz, verwendet. Sie ist somit ein Mass für systematische Fehler. Die **Präzision** einer Messung beschreibt die zufällige Verteilung der Messwerte um deren Mittelwert und ist somit ein Mass für zufällige Fehler. Beide Fehler lassen sich anschaulich in einem **Histogramm** darstellen, indem wir den Messbereich in gleich grosse “Bins” diskretisieren und die Messergebnisse innerhalb jedes Bins zählen (siehe Fig. ??).

Kehren wir nun zu unserem anfänglichen Beispiel zurück. Wir führen  $N$  Messungen von  $g$  durch und speichern die Ergebnisse in einem Numpy-Array  $g$  ab. Ebenfalls speichern wir die zugehörigen Zeitpunkte  $t$  der Messungen. In einem ersten Schritt visualisieren wir diese Daten in einem sehr einfachen Plot

```
plt.plot(t, g)
```

und bestimmen den empirischen Mittelwert und die empirische Varianz der Stichprobe. Dies kann sowohl mithilfe der in ?? und ?? angegebenen Formeln

```
mean = np.sum(g)/N
var = np.sum((g-mean)**2)/(N-1)
```

oder aber durch Verwendung der Funktionen der numpy-Bibliothek.

```
mean = np.mean(g)
var = np.var(g)
```

geschehen. Hierbei ist zu beachten, dass die so berechnete Varianz nicht die empirische Varianz ist. Um die empirische Varianz zu erhalten muss die Varianz mit einem Faktor  $\frac{N}{N-1}$  multipliziert werden.

Wir generieren nun unsere “Bins”, um den Messbereich zu diskretisieren, und zählen die Messergebnisse pro Bin. Zur Auswahl der Breite der Bins ist die anfängliche Visualisierung der Daten hilfreich. Da die vertikale Auflösung unserer Messung ohnehin durch die Präzision unseres Messgerätes limitiert ist, wählen wir relativ breite Bins.

```
n_bins = 14+1 # Anzahl an Bins
bin_size = 0.005 # Breite des Bins
bins = np.linspace(9.76, 9.76+bin_size*(n_bins-1), n_bins)
# Initialisiere Numpy Array um Anzahl Messergebnisse pro Bin
# zu speichern
occurrence = np.zeros(n_bins)
offset = np.min(bins)//bin_size + 1
# Iteriere ueber alle Datenpunkte
# Bestimme den Index des Bins, in den sie fallen
# Inkrementiere die Anzahl Ereignisse in diesem Bin um 1
for gx_i in gz:
    index = int(gx_i//bin_size-offset)
    occurrence[index] = occurrence[index] + 1
```

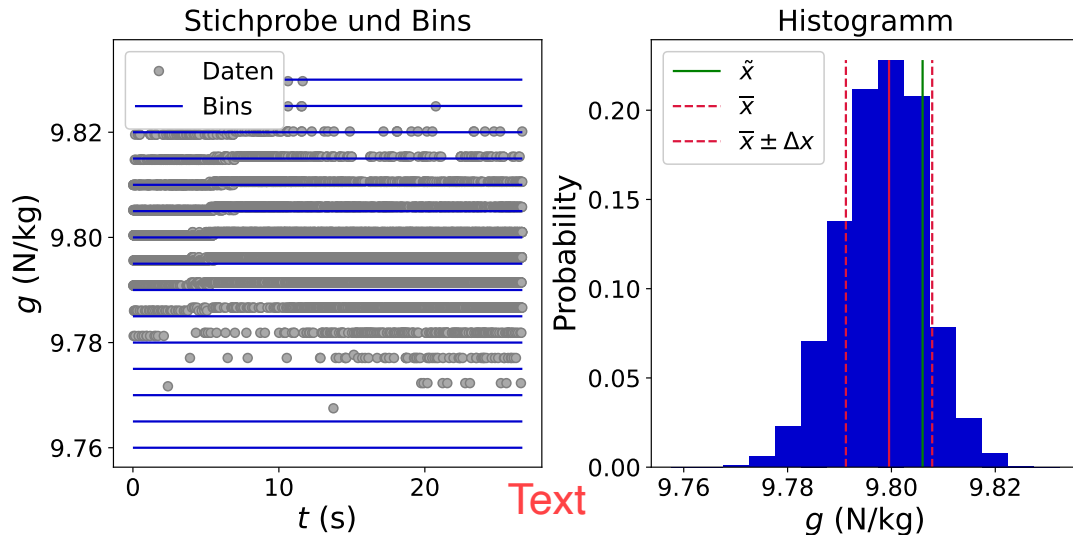
Nun können wir sehr einfach das Histogramm erstellen und den empirischen Mittelwert, die empirische Standardabweichung sowie den erwarteten Wert einzeichnen. Hierbei normalisieren wir unser Histogramm bereits entsprechend, um eine prozentuelle Wahrscheinlichkeit zu erhalten.<sup>2</sup>

```
A = N
g_theo = 9.78
plt.bar(bins, occurrence/A, width = bin_size)
plt.vlines(g_theo, 0, np.max(occurrence)/A)
plt.vlines(mean, 0, np.max(occurrence)/A)
plt.vlines(mean + np.sqrt(var), 0, np.max(occurrence)/A)
plt.vlines(mean - np.sqrt(var), 0, np.max(occurrence)/A )
```

Der von uns bestimmte Mittelwert weicht vom erwarteten Wert von  $g$  ab, unsere Messung ist also nicht genau, allerdings liegt der erwartete Wert noch innerhalb der empirischen Standardabweichung der Messung.

---

<sup>2</sup>“Normalisieren” bezeichnet die Multiplikation mit einem Faktor (in diesem Fall  $A$ ), um der berechneten Grösse eine konkrete Bedeutung zuweisen zu können. In unserem Fall soll die Normierung erreichen, dass die Summe der Histogrammwerte 1 (oder 100 Prozent) ergibt.



**Abbildung 2.2:** Statistische Analyse einer Messung von  $g$ . **Links:** Gemessene Datenpunkte (graue Punkte) als Funktion der Zeit  $t$ . Die Kanten der “Bins” sind als horizontale Linien indiziert. Aufgrund der limitierten vertikalen Auflösung der Messung arbeiten wir mit einer relativ geringen Anzahl von Bins. **Rechts:** Das resultierende Histogramm der Stichprobe. Die Anzahl der Messergebnisse pro Bin wurden für diese Darstellung bereits der Anzahl der Datenpunkte  $N$  normalisiert, sodass die Summe aller Histogrammwerte 1 ergibt. Der erwartete Wert  $\tilde{x}$  für die Erdbeschleunigung, sowie empirischer Mittelwert und empirische Standardabweichung sind eingezeichnet.

## 2.3 Wahrscheinlichkeitsverteilungen

Im bisherigen Verlauf dieses Kapitels haben wir gelernt, wie wir zufällige Messfehler anhand der wiederholten Durchführung einer Messung charakterisieren können. Ein wichtiges Resultat dieser Analyse war hierbei das diskrete Histogramm der Ereignishäufigkeiten (siehe Fig. ??), die wir intuitiv als Wahrscheinlichkeiten, die entsprechenden Werte für unsere Messgrösse zu erhalten, interpretiert haben. Im Folgenden wollen wir diesen Zusammenhang systematischer ergründen und hierzu den frequentistischen Wahrscheinlichkeitsbegriff und Wahrscheinlichkeitsverteilungen diskutieren. Ein alternativer Ansatz zur Interpretation von Wahrscheinlichkeiten, die sogenannte Bayesische Datenanalyse, wird in Kapitel ?? eingeführt.

### 2.3.1 Frequentistischer Wahrscheinlichkeitsbegriff

Der frequentistische Wahrscheinlichkeitsbegriff entspricht der Definition, die wir intuitiv für unsere erste Analyse des Histogramms aus Fig. ?? angenommen haben: Wir interpretieren den wiederholten Messvorgang einer experimentell zugänglichen Grösse als eine Reihe voneinander unabhängiger Zufallsexperimente und definieren die Wahrscheinlichkeit eines bestimmten Ergebnisses anhand der relativen Häufigkeit, mit der es in einer grossen Anzahl an Wiederholungen des Experiments auftritt. Im Grenzfall unendlich vieler Wiederholungen des Zufallsexperiment ist demnach die Wahrscheinlichkeit  $P(A)$ , dass ein bestimmtes Ereignis  $A$  eintritt, mathematisch definiert als

$$P(A) = \frac{n_A}{n_S} \quad (2.3.1)$$

wobei  $n_S$  die Anzahl der möglichen Ergebnisse des Experiments ist und  $n_A$  die Anzahl der Ergebnisse, die zu  $A$  führen. Grundsätzlich gilt, dass die frequentistischen Wahrscheinlichkeiten  $P$  umso besser aus den experimentellen Daten abgeschätzt werden können, je häufiger das Zufallsexperiment wiederholt wurde. Die heutige Popularität des frequentistischen Ansatzes ist somit zu grossen Teilen auf die gestiegenen technischen Standards und gesteigerten zur Verfügung

stehenden Rechenleistungen zurückzuführen.

Ein typisches Beispiel zur Veranschaulichung der frequentistischen Wahrscheinlichkeit ist der Würfelwurf. Hier ist die Anzahl der möglichen Ergebnisse  $n_S = 6$ . Die Wahrscheinlichkeit, dass die Augenzahl grösser ist als 4 (d.h.  $n_A = 2$ ) wäre folglich  $P(A) = \frac{2}{6} = \frac{1}{3}$ . Sofern eine ausreichend hohe Anzahl an Würfelwürfen durchgeführt, sollte somit ungefähr in jeder dritten Messung eine Augenzahl, die grösser ist als 4, auftreten.

Da ein mögliches Ergebnis  $A$  immer eine Teilmenge der Gesamtmenge aller möglichen Ergebnisse  $S$  des Experiments darstellt, gilt stets

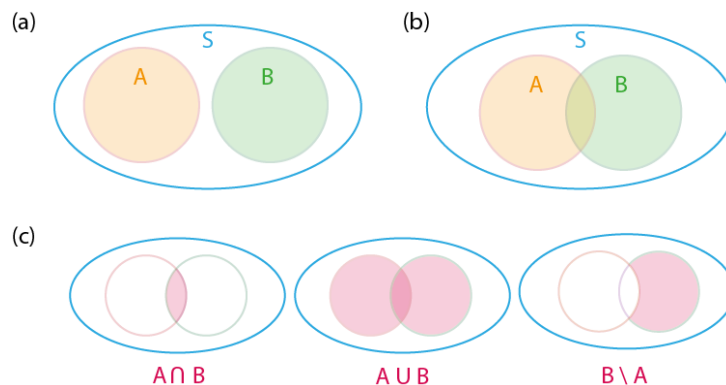
$$A \subseteq S : \quad 0 \leq P(A) \leq 1 \quad (2.3.2)$$

und zudem

$$P(S) = P(A) + P(\bar{A}) = 1, \quad (2.3.3)$$

wobei  $P(\bar{A})$  die Wahrscheinlichkeit ist, dass Ergebnis  $A$  *nicht* eintritt.

Häufig sind wir gleichzeitig an mehreren möglichen Ergebnissen, z.B.  $A$  und  $B$ , des Experiments interessiert und wollen zusätzlich zu deren individuellen frequentistischen Wahrscheinlichkeiten,  $P(A)$  und  $P(B)$ , auch kompliziertere Wahrscheinlichkeiten berechnen, die beide Ergebnisse betreffen. Zu diesem Zweck ist es hilfreich, sogenannte Venn-Diagramme (auch: Mengendiagramme) zu verwenden, die den Grad an Überlapp der Teilmengen mehrerer Ergebnisse und folglich deren Schnittmengen ( $A \cap B$ ), Vereinigungsmengen ( $A \cup B$ ) sowie Differenzmengen ( $A \setminus B$ ,  $B \setminus A$ ) graphisch darstellen können. In unserem anfänglichen Beispiel des Würfels könnten wir etwa zusätzlich zum Ergebnis  $A$  „Die Augenzahl ist grösser als 4“ auch das Ergebnis  $B$  „Die Augenzahl ist 6“ betrachten. Im entsprechenden Venn-Diagramm überlappen die Teilmengen dieser Ergebnisse, da sie gleichzeitig eintreten können - die Augenzahl ist schliesslich grösser als 4, wenn sie 6 beträgt.



**Abbildung 2.3:** Venn-Diagramme zur Veranschaulichung von Frequentistischen Wahrscheinlichkeiten. Die Ergebnisse  $A$  und  $B$  sind Teilmengen der Gesamtmenge  $S$  aller möglichen Ergebnisse des Experiments. (a) Wenn die Ergebnisse  $A$  und  $B$  nicht gleichzeitig eintreten können, überlappen ihre Teilmengen im Venn-Diagramm nicht. (b) Wenn die Ergebnisse  $A$  und  $B$  gleichzeitig eintreten können, gibt es einen gewissen Überlapp zwischen ihren Teilmengen. (c) Venn-Diagramme sind hilfreich, um Schnittmengen, Vereinigungsmengen und Differenzmengen darzustellen.

Die Wahrscheinlichkeit  $P(A \cup B)$ , dass entweder Ereignis  $A$  oder Ereignis  $B$  eintritt, ist

$$P(A \cup B) = P(A) + P(B) - P(A \cap B), \quad (2.3.4)$$

wobei wir die Wahrscheinlichkeit  $P(A \cap B)$  des gleichzeitigen Auftretens beider Ergebnisse subtrahieren müssen, um im Fall von überlappenden Ergebnismengen die entsprechende Fläche

des Venn-Diagramms nicht doppelt zu zählen. Weiterhin ist häufig die konditionelle (auch: bedingte) Wahrscheinlichkeit  $P(A|B)$ , dass Ergebnis  $A$  eintritt, wenn  $B$  eintritt, von Interesse. Sie ist gegeben als

$$P(A|B) = \frac{P(A \cap B)}{P(B)}. \quad (2.3.5)$$

Da  $P(A \cap B) = P(B \cap A)$  können wir folgend aus Gleichung ?? den Zusammenhang

$$P(A \cap B) = P(A|B)P(B) = P(B|A)P(A) \quad (2.3.6)$$

zwischen  $P(A|B)$  und  $P(B|A)$  herstellen. Aus diesem folgt schliesslich der sogenannte Satz von Bayes,

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}. \quad (2.3.7)$$

Die bisherige Betrachtung ist unabhängig davon, ob ein kausaler Zusammenhang zwischen  $A$  und  $B$  herrscht, korrekt. Falls kein solcher Zusammenhang besteht, d.h.  $A$  und  $B$  vollkommen unabhängig sind, entsprechen die bedingten Wahrscheinlichkeiten einfach den individuellen Wahrscheinlichkeiten der Ergebnisse, d.h.

$$P(A|B) = P(A). \quad (2.3.8)$$

Weiterhin gilt dann

$$P(A \cap B) = P(A)P(B). \quad (2.3.9)$$

### 2.3.2 Wahrscheinlichkeitsverteilungen - Grundlegende Charakteristiken

Dem frequentistischen Wahrscheinlichkeitsbegriff folgend stellt das aus den Ergebnissen der wiederholten Messung der Zufallsvariablen erhaltene Histogramm (siehe Fig. ??) in einer diskreten Form die Wahrscheinlichkeiten als Funktion des Messwertes da. Es bildet somit, sofern zum Einen eine ausreichend hohe Anzahl an Wiederholungen durchgeführt wurde und zum Anderen sowohl die vertikale als auch horizontalen Auflösungen der Messung es zulassen, näherungsweise die zugrundeliegende (kontinuierliche) Wahrscheinlichkeitsverteilung der Messgrösse ab. Im Folgenden werden wir zunächst einige grundlegende Charakteristiken von Wahrscheinlichkeitsverteilungen definieren und dann auf einige relevante, konkrete Wahrscheinlichkeitsverteilungen eingehen.

Eine Wahrscheinlichkeitsverteilung ist eine positive und normierte Funktion, die die Verteilung der Wahrscheinlichkeit einer Zufallsvariablen  $x$  beschreibt. Falls die Zufallsvariable nur diskrete Werte annehmen kann, spricht man von einer sogenannten **Probability Mass Function** (PMF), die jedem möglichen, diskreten Wert  $x_n$  der Zufallsvariable eine Wahrscheinlichkeit  $P(x_n)$  zuordnet. Es gilt

$$0 \leq P(x_n) \leq 1, \quad (2.3.10)$$

sowie

$$\sum_n P(x_n) = 1. \quad (2.3.11)$$

Im Falle einer kontinuierlichen Zufallsvariablen  $x$  sprechen wir von einer **Probability Density Function** (PDF) (zu deutsch: Wahrscheinlichkeitsdichtefunktion)  $f(x)$  die

$$0 \leq f(x) \quad (2.3.12)$$

und

$$\int_{-\infty}^{\infty} f(x) dx = 1 \quad (2.3.13)$$

erfüllt. Während im diskreten Fall die Wahrscheinlichkeit für das Auftreten eines bestimmten Wertes  $x_n$  direkt aus der PMF abgelesen werden kann, entspricht im kontinuierlichen Fall  $f(x)$  nicht direkt der Wahrscheinlichkeit für das Auftreten eines Wertes  $x$ . Stattdessen kann die Wahrscheinlichkeit dafür, dass das Ergebnis  $x$  in einem Intervall zwischen  $a$  und  $b$  liegt, durch Integration der PDF als

$$P(a \leq x \leq b) = \int_a^b f(x) dx \quad (2.3.14)$$

erhalten werden.

In Tabelle ?? sind einige der wichtigsten Kennzahlen von Wahrscheinlichkeitsdichtefunktionen aufgelistet.

**Tabelle 2.3.1:** Auflistung einiger wichtiger Kennzahlen von Wahrscheinlichkeitsdichtefunktionen.

| Name und Variable          | Formel                                                                                   | Bedeutung                                                                                                                                                                      |
|----------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modus $x_{\text{mode}}$    | $x_{\text{mode}} : \left. \frac{df(x)}{dx} \right _{x_{\text{mode}}} = 0 \quad (2.3.15)$ | Der Wert von $x$ , bei dem die Verteilung ihr Maximum erreicht. Die hier angegebene Formel zur Berechnung kann für eine PDF mit einem einzigen Maximum verwendet werden.       |
| Median $x_{\text{median}}$ | $\frac{1}{2} = \int_{-\infty}^{x_{\text{median}}} f(x) dx \quad (2.3.16)$                | Die Mitte der Verteilung. Der Median wird häufig bei der Darstellung von sozioökonomischen Daten verwendet, etwa zur Angabe des durchschnittlichen Gehalts einer Gesellschaft. |
| Halbwertsbreite FWHM       | $f(a) = f(b) = \frac{1}{2} f(x_{\text{mode}}) \quad (2.3.17)$ $\text{FWHM} = b - a.$     | Charakterisiert die Breite der Verteilung beim halben Wert ihres Maximalwertes.                                                                                                |
| Erwartungswert $\mu$       | $\mu = E(x)$ $= \int_{-\infty}^{+\infty} x f(x) dx \quad (2.3.18)$                       | Die Zahl, die die Zufallsvariable $x$ im Mittel annimmt. Häufig wird der Erwartungswert auch als Mittelwert bezeichnet.                                                        |



|                                |                                                                                                                                |                                                                                         |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Varianz $\text{var}(x)$        | $\begin{aligned}\text{var}(x) &= E((x - \mu)^2) \\ &= \int_{-\infty}^{+\infty} (x - \mu)^2 f(x) dx\end{aligned}\quad (2.3.19)$ | Die mittlere quadratische Abweichung der Zufallsvariablen $x$ von ihrem Erwartungswert. |
| Standardabweichung<br>$\sigma$ | $\sigma = \sqrt{\text{var}(x)} \quad (2.3.20)$                                                                                 |                                                                                         |

Zusätzlich zu diesen Charakteristika lassen sich für viele PDF sogenannte Momente berechnen. Das  $m$ -te Moment  $M_m$  einer Wahrscheinlichkeitsverteilung  $f(x)$  ist definiert als

$$M_m = \int_{-\infty}^{+\infty} x^m f(x) dx. \quad (2.3.21)$$

Das 0-te Moment

$$M_0 = \int_{-\infty}^{+\infty} f(x) dx = 1 \quad (2.3.22)$$

entspricht somit der Normierungsbedingung der Wahrscheinlichkeitsverteilung, während das 1-te Moment

$$M_1 = \int_{-\infty}^{+\infty} x f(x) dx = E(x) \quad (2.3.23)$$

ihren Mittelwert liefert.

Ebenfalls ist das  $m$ -te zentrale Moment einer Wahrscheinlichkeitsverteilung definiert als

$$\tilde{M}_m = \langle (x - M_1)^m \rangle. \quad (2.3.24)$$

Damit ist die Varianz einer PDF gegeben durch das zweite zentrale Moment:

$$\sigma^2 = \langle (x - M_1)^2 \rangle = \langle x^2 \rangle - \langle x \rangle^2 = M_2 - M_1^2. \quad (2.3.25)$$

Das dritte zentrale Moment quantifiziert die Schiefe und das vierte zentrale Moment die Wölbung der PDF. Es ist hilfreich, die momenterzeugende Funktion zu definieren und unter Verwendung der Taylorreihe von  $\exp(\zeta x)$  zu formulieren

$$\begin{aligned}M(\zeta) &= \int_{-\infty}^{+\infty} \exp(\zeta x) f(x) dx \\ &= \int_{-\infty}^{+\infty} \left( 1 + \zeta x + \frac{\zeta^2 x^2}{2!} + \dots \right) f(x) dx \\ &= M_0 + \zeta M_1 + \frac{\zeta^2}{2!} M_2 + \dots\end{aligned} \quad (2.3.26)$$

Höhere Momente  $m \geq 1$  können iterativ aus der Ableitung der momenterzeugenden Funktion berechnet werden:

$$M_m = \left. \frac{\partial^m M(\zeta)}{\partial \zeta^m} \right|_{\zeta=0}. \quad (2.3.27)$$

Darüber hinaus ist es hilfreich, die charakteristische Funktion  $\phi(\nu)$

$$\phi(\nu) = \int_{-\infty}^{+\infty} \exp(i\nu x) f(x) dx. \quad (2.3.28)$$

einer Wahrscheinlichkeitsverteilung  $f(x)$  zu definieren, wobei  $i = \sqrt{-1}$  die imaginäre Einheit ist. Die charakteristische Funktion ist die inverse Fouriertransformierte der Wahrscheinlichkeitsverteilung. Die Fourier-Transformation werden wir in Kapitel ?? zur Spektralanalyse genauer kennenlernen. Indem wir die Exponentialfunktion erweitern, können wir die charakteristische Funktion als Funktion der Momente der PDF ausdrücken:

$$\begin{aligned} \phi(\nu) &= \int_{-\infty}^{+\infty} \left( 1 + i\nu x + \frac{1}{2!}(i\nu)^2 x^2 + \frac{1}{3!}(i\nu)^3 x^3 + \dots \right) f(x) dx \\ &= 1 + i\nu M_1 + \frac{1}{2!}(i\nu)^2 M_2 + \frac{1}{3!}(i\nu)^3 M_3 + \dots \end{aligned} \quad (2.3.29)$$

Ausserdem können wir die Wahrscheinlichkeitsverteilung  $f(x)$  durch Fouriertransformation von  $\phi(\nu)$  berechnen:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} \exp(-i\nu x) \phi(\nu) d\nu. \quad (2.3.30)$$

Die Kombination dieser beiden Ergebnisse ergibt:

$$f(x) = \frac{1}{2\pi} \int_{-\infty}^{+\infty} \exp(-i\nu x) \left( 1 + i\nu M_1 + \frac{1}{2!}(i\nu)^2 M_2 + \frac{1}{3!}(i\nu)^3 M_3 + \dots \right) d\nu. \quad (2.3.31)$$

### 2.3.3 Die Normalverteilung

In der Praxis finden wir sehr häufig normalverteilte Messergebnisse vor. Insbesondere liegt eine Normalverteilung immer dann vor, wenn der Messfehler aus der additiven Überlagerung zahlreicher unabhängiger, zufälliger Ereignisse resultiert und somit selbst tatsächlich vollkommen zufällig ist. Der Grund hierfür ist der sogenannte **zentrale Grenzwertsatz**:

Die Summe vieler (identisch verteilter) Zufallsvariablen ist eine normalverteilte Zufallsvariable.

Beispiel: Während die Augenzahl beim Wurf eines einzelnen Würfels gleichverteilt ist, ist die Summe der Augenzahlen mehrerer Würfel normalverteilt.

Die Normalverteilung wird daher häufig als Modell für die Wahrscheinlichkeitsverteilung von Messdaten angenommen, sofern keine Kenntnis über das Vorhandensein einer anderen, physikalisch motivierten Verteilung besteht. Sie wird durch den Erwartungswert  $\mu$  und die Standardabweichung (die Quadratwurzel der Varianz)  $\sigma = \sqrt{\text{var}(x)}$  parametrisiert.

$$f(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right). \quad (2.3.32)$$

Die Standardabweichung gibt hierbei das Gewicht der PDF um den Erwartungswert an: Ein Messergebnis liegt mit 68 % Wahrscheinlichkeit innerhalb des Intervalls  $[\mu - \sigma, \mu + \sigma]$ . Je breiter

das Intervall, desto mehr Gewicht, d.h. umso wahrscheinlicher ist es, ein Messergebnis im Intervall zu finden:

$$\begin{aligned}
 1\sigma &\rightarrow 68\% \\
 2\sigma &\rightarrow 95\% \\
 3\sigma &\rightarrow 99.7\% \\
 &\dots \\
 5\sigma &\rightarrow 99.99994\%
 \end{aligned}
 \tag{2.3.33}$$

Selbstverständlich sind die Parameter  $(\mu, \sigma)$  der Normalverteilung, die einer gemessenen Stichprobe zugrunde liegt, zunächst unbekannt und müssen anhand der experimentell gemessenen Daten möglichst gut abgeschätzt werden. Wir wollen dafür den Erwartungswert  $E(x) = \mu$  mit dem empirischen Mittelwert  $\bar{x}$  und die Standardabweichung  $\sigma$  mit der empirischen Standardabweichung  $\Delta_x$  annähern. Aber dürfen wir das?

Um zu überprüfen, ob der empirische Mittelwert und die empirische Varianz gute Schätzwerte für den tatsächlichen Erwartungswert und die tatsächliche Varianz sind, bilden wir deren Erwartungswerte,  $E(\bar{x})$  und  $E(\Delta_x^2)$ . Zunächst berechnen wir den Erwartungswert des empirischen Mittelwertes:

$$E(\bar{x}) = E\left(\frac{1}{N} \sum_{n=1}^N x_n\right) = \frac{1}{N} \sum_{n=1}^N E(x_n). \tag{2.3.34}$$

wobei wir verwendet haben, dass der Erwartungswert linear ist und somit für unabhängige Zufallsvariablen  $X, Y$  sowie Konstanten  $a, b$  gilt:

$$\begin{aligned}
 E(a) &= a \\
 E(aX) &= aE(X) \\
 E(aX + b) &= aE(X) + b \\
 E(aX + bY) &= aE(X) + bE(Y).
 \end{aligned}
 \tag{2.3.35}$$

Wenn wir nun weiterhin beachten, dass jede einzelne Messung der Zufallsvariable den gleichen Erwartungswert besitzt wie die Zufallsvariable selbst (d.h.  $E(x_n) = E(x)$ ) erhalten wir

$$E(\bar{x}) = \frac{1}{N} \sum_{n=1}^N E(x_n) = \frac{1}{N} \sum_{n=1}^N E(x) = \frac{1}{N} N E(x) = E(x). \tag{2.3.36}$$

Der Erwartungswert des Mittelwertes entspricht also tatsächlich dem Erwartungswert der Zufallsvariablen. Der aus den Stichproben erhaltene empirische Mittelwert ist somit in der Tat ein guter Schätzwert für den tatsächlichen Erwartungswert:

$$\mu \approx \bar{x} = \frac{1}{N} \sum_{n=1}^N x_n, \tag{2.3.37}$$

Wir gehen nun analog vor für die Varianz und berechnen ihren Erwartungswert. Hierfür verwenden wir den Zusammenhang

$$\text{var}(x) = E(x^2) - E(x)^2 \rightarrow E(x^2) = \text{var}(x) + E(x)^2 \tag{2.3.38}$$

sowie

$$\text{var}(\bar{x}) = \frac{\text{var}(x)}{N} \tag{2.3.39}$$

und können damit folgende Umformung vornehmen:

$$\begin{aligned}
E(\Delta_x^2) &= E\left(\frac{1}{N-1} \sum_{n=1}^N (\bar{x} - x_n)^2\right) \\
&= E\left(\frac{1}{N-1} \sum_{n=1}^N (\bar{x}^2 - 2\bar{x}x_n + x_n^2)\right) \\
&= E\left(\frac{1}{N-1} \left(\sum_{n=1}^N \bar{x}^2 - 2\sum_{n=1}^N \bar{x}x_n + \sum_{n=1}^N x_n^2\right)\right) \\
&= E\left(\frac{1}{N-1} \left(N\bar{x}^2 - 2\bar{x} \sum_{n=1}^N x_n + \sum_{n=1}^N x_n^2\right)\right) \\
&= E\left(\frac{1}{N-1} \left(N\bar{x}^2 - 2N\bar{x}^2 + \sum_{n=1}^N x_n^2\right)\right) \\
&= E\left(\frac{1}{N-1} \left(\sum_{n=1}^N x_n^2 - N\bar{x}^2\right)\right) \tag{2.3.40} \\
&= \frac{1}{N-1} \left(\sum_{n=1}^N E(x_n^2) - NE(\bar{x}^2)\right) \\
&= \frac{1}{N-1} \left(\sum_{n=1}^N (\text{var}(x) + E(x)^2) - NE(\bar{x}^2)\right) \\
&= \frac{1}{N-1} \left(\sum_{n=1}^N (\text{var}(x) + E(x)^2) - N\left(\frac{\text{var}(x)}{N} + E(\bar{x})^2\right)\right) \\
&= \frac{1}{N-1} (N\text{var}(x) - NE(x)^2 - \text{var}(x) + NE(\bar{x})^2) \\
&= \frac{1}{N-1} (\text{var}(x)(N-1) - NE(x)^2 + NE(x)^2) \\
&= \frac{1}{N-1} \text{var}(x)(N-1) \\
&= \text{var}(x).
\end{aligned}$$

Somit ist auch die empirische Standardabweichung ein guter Schätzwert der Standardabweichung.

$$\sigma \approx \Delta_x = \sqrt{\frac{1}{N-1} \sum_{n=1}^N (x_n - \bar{x})^2}. \tag{2.3.41}$$

Jetzt, da wir die empirische Standardabweichung und Varianz als gute Schätzwerte für Erwartungswert und Varianz identifiziert haben, wollen wir zu unserem Datensatz an Messdaten für  $g$  aus Abschnitt ?? zurückkehren. Zunächst wollen wir überprüfen, ob die von uns gemessenen Daten tatsächlich einer Normalverteilung folgen. Wir berechnen, welcher Anteil unserer Messwerte im  $1\sigma$ -Intervall liegt und vergleichen das Resultat mit dem für die Normalverteilung erwarteten Wert.

```

n_sigma = 0
width = 1
# Iteriere durch alle Bins und addiere ihre Werte auf, wenn sie im
# entsprechenden Intervall liegen
i = 0
for b in bins:

```

```

if (b >= (mean - np.sqrt(var) * width)) and (b <= (mean + np.sqrt(var) *
width)):
    n_sigma = n_sigma + occurrence[i]
i = i + 1
print('Occurences within the {:d}-sigma-interval: {:.1f}%'.format(width, n_sigma
/N*100))

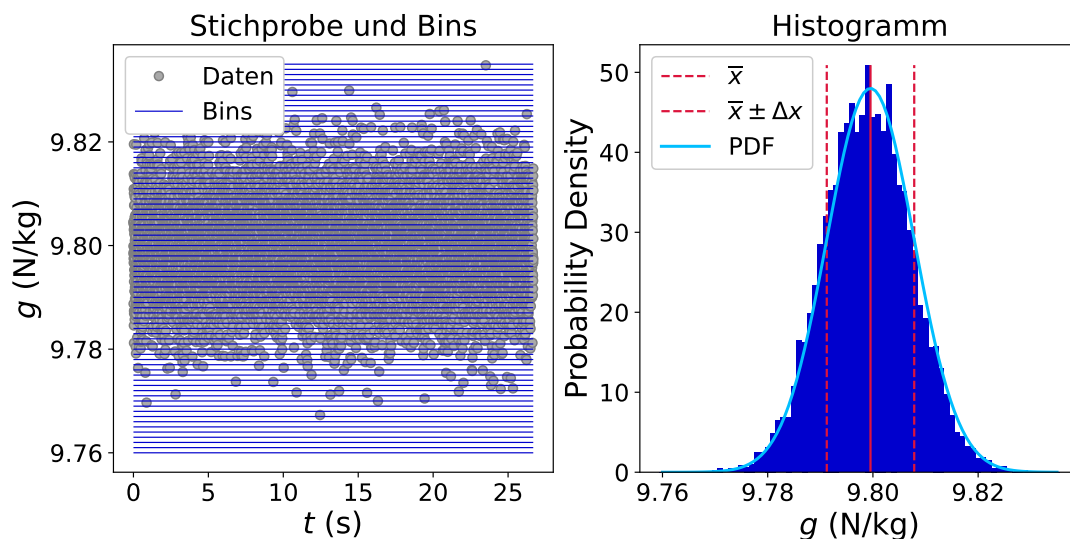
```

Wir finden, dass 64.7 Prozent unserer Messung im  $1\sigma$  Intervall liegen, was sehr nah am erwarteten Wert für die Gaussverteilung liegt. Auch entsprechende Berechnungen für die grösseren Intervalle liefern Ergebnisse, die sehr ähnlich zu den erwarteten Werten sind. Unsere Stichprobe scheint tatsächlich normalverteilt zu sein.

In einem weiteren Schritt simulieren wir nun unter der Annahme einer Normalverteilung und unter Verwendung unserer empirisch gemessenen Mittelwerts und Standardabweichung als Schätzungen der Parameter einen Datensatz mit höherer vertikaler Auflösung. Hierzu verwenden wir das random-Modul in Python:

```
g_sim = np.random.normal(mean, np.sqrt(var), N)
```

Wir analysieren die so erhaltenen Werte analog zu unserem ursprünglichen Datensatz, verwenden nun allerdings deutlich schmalere Bins und normalisieren die erhaltenen Ereignishäufigkeiten nicht nur mit der Anzahl  $N$  der Messungen, sondern ebenfalls mit der Bin-Breite, um eine Wahrscheinlichkeitsdichte (*Probability Density*) zu erhalten. Ebenfalls zeichnen wir die zugehörige Normalverteilung als Graph ein (siehe Figur ??). Dieser Vergleich versinnbildlicht, dass eine Messung mit ausreichender Anzahl Datenpunkten und entsprechender Auflösung die zugrundeliegende Wahrscheinlichkeitsverteilung sehr gut abbilden kann.



**Abbildung 2.4:** Statistische Analyse einer simulierten Messung von  $g$  mit höherer Auflösung. **Links:** Auftragung der gemessenen Datenpunkte (graue Punkte) gegen die Zeit  $t$ . Die Kanten der “Bins” sind als horizontale Linien indiziert. **Rechts:** Das resultierende Histogramm der Stichprobe. Die Anzahl der Messergebnisse pro Bins wurden für diese Darstellung bereits mit dem Produkt aus Anzahl der Datenpunkte  $n$  und Bin-Breite normalisiert, um eine Wahrscheinlichkeitsdichte zu erhalten. Der empirische Mittelwert und die empirische Standardabweichung (rote Linien), sowie die aus diesen erhaltene Normalverteilung (blaue Linie) sind eingezeichnet.

### 2.3.4 Binomial- und Poissonverteilung

Die Binomialverteilung ist eine der wichtigsten, diskreten Wahrscheinlichkeitsverteilungen. Sie liegt immer dann vor, wenn eine Serie von  $n$  unabhängigen und gleichartigen Versuchen zur Messung einer Zufallsvariablen mit lediglich zwei möglichen Ergebnissen,  $d$  und  $u$ , durchgeführt

wird. Ein typisches Alltagsbeispiel für eine Binomialverteilung wäre etwa der Münzwurf, ein Beispiel der statistischen Physik die Besetzung eines Zustandes mit einer Reihe von Spins, die sich entweder in einem „up“ (d.h.  $S_z = +\frac{1}{2}\hbar$ ) oder „down“ (d.h.  $S_z = -\frac{1}{2}\hbar$ ) Zustand befinden können. Die Wahrscheinlichkeiten für die beiden Ergebnisse sind in jeder Durchführung des Versuchs identisch und

$$P(u) = p, \quad (2.3.42)$$

$$P(d) = q = 1 - p. \quad (2.3.43)$$

Die Binomialverteilung

$$B(k|p, n) = \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k} \quad (2.3.44)$$

gibt die Wahrscheinlichkeit  $B(k|p, n)$  dafür an, dass in  $n$  Versuchen  $k$ -mal das Ergebnis  $u$  auftritt. Sie setzt sich zusammen aus der der Wahrscheinlichkeit  $p^k (1-p)^{n-k}$  für das Auftreten der  $k$  Ereignisse in einer bestimmten Reihenfolge und dem Binomialkoeffizienten

$$\binom{n}{k} \equiv \frac{n!}{k!(n-k)!}, \quad (2.3.45)$$

der angibt, auf wie viele verschiedene Arten man aus einer Menge von  $n$  Objekten  $k$  Objekte auswählen kann. Als Wahrscheinlichkeitsverteilung muss dies natürlich normiert sein, sodass wir als 0-tes Moment

$$M_0 = \sum_{k=0}^n \binom{n}{k} p^k q^{n-k} = 1 \quad (2.3.46)$$

erhalten. Wir wollen nun die höheren Momente ( $m \geq 1$ ) mithilfe der Ableitung der Momenterzeugenden Funktion

$$M_m = \left. \frac{\partial^m M(\zeta)}{\partial \zeta^m} \right|_{\zeta=0} \quad (2.3.47)$$

berechnen. Wir berechnen die Ableitung als

$$\begin{aligned} \frac{\partial M_m}{\partial p} &= \frac{\partial}{\partial p} \left( \sum_{k=0}^n k^m \binom{n}{k} p^k (1-p)^{n-k} \right) \\ &= \sum_{k=0}^n k^{m+1} \binom{n}{k} p^{k-1} (1-p)^{n-k} - \sum_{k=0}^n k^m (n-k) \binom{n}{k} p^k (1-p)^{n-k-1} \\ &= \frac{1}{p} \sum_{k=0}^n k^{m+1} \binom{n}{k} p^k q^{n-k} - \frac{n}{q} \sum_{k=0}^n k^m \binom{n}{k} p^k q^{n-k} + \frac{1}{q} \sum_{k=0}^n k^{m+1} \binom{n}{k} p^k q^{n-k} \\ &= \frac{1}{p} M_{m+1} - \frac{n}{1} M_m + \frac{1}{q} M_{m+1}. \end{aligned} \quad (2.3.48)$$

und können somit die höheren Momente mittels

$$M_{m+1} = np M_m + pq \frac{\partial M_m}{\partial p} \quad (2.3.49)$$

berechnen. Wir erhalten somit das erste Moment, den Mittelwert der Binomialverteilung, als

$$\mu = M_1 = np M_0 + pq \frac{\partial M_0}{\partial p} = np. \quad (2.3.50)$$

Für das zweite Moment folgt

$$M_2 = npM_1 + pq \frac{\partial M_1}{\partial p} = np(np) + pq \frac{\partial}{\partial p}(np) = n^2 p^2 + npq, \quad (2.3.51)$$

sodass wir für Standardabweichung und die Varianz schlussendlich folgende Ausdrücke erhalten:

$$\sigma^2 = M_2 - M_1^2 = npq, \quad (2.3.52)$$

$$\sigma \propto \sqrt{n}. \quad (2.3.53)$$

Sofern ein Grenzfall vorliegt, in dem gilt

- Die Anzahl der Messungen  $n$  ist sehr gross.
- Die Anzahl der Ereignisse pro Messung  $k$  ist klein, insbesondere  $n \gg k$ .
- Die Wahrscheinlichkeit für ein Event  $p$  ist sehr klein ( $p \rightarrow 0$ ).

geht die Binomialverteilung in die Poissonverteilung über. Die Poissonverteilung stellt somit einen Grenzfall der Binomialverteilung dar. Um die Poissonverteilung zu erhalten, müssen wir die Binomialverteilung wie folgt umformen

$$\lim_{n \gg k} \binom{n}{k} p^k (1-p)^{n-k} = \lim_{n \gg k} \frac{n!}{k!(n-k)!} p^k (1-p)^{n-k} = \frac{n^k}{k!} p^k (1-p)^n \quad (2.3.54)$$

und erhalten schliesslich durch Einsetzen des Mittelwertes  $\mu = np$  und Bilden des Grenzwertes  $\lim_{p \rightarrow 0} (1-p)^{\frac{1}{p}} = e^{-1}$  die Poisson-Verteilung

$$P_\mu(k) = \frac{\mu^k}{k!} \exp(-\mu). \quad (2.3.55)$$

Analog zur Binomialverteilung können wir nun ausgehend von  $M_0 = 1$  iterativ mittels

$$M_{m+1} = \mu M_m + \mu \frac{\partial M_m}{\partial \mu} \quad (2.3.56)$$

die höheren Momente der Poissonverteilung berechnen. Wir erhalten somit für den Mittelwert der Poissonverteilung

$$M_1 = \mu M_0 + \mu \frac{\partial M_0}{\partial \mu} = \mu. \quad (2.3.57)$$

Für die Varianz erhalten wir:

$$M_2 = \mu M_1 + \mu \frac{\partial M_1}{\partial \mu} = \mu^2 + \mu, \quad (2.3.58)$$

$$\sigma^2 = M_2 - M_1^2 = \mu. \quad (2.3.59)$$

### 2.3.5 $\chi^2$ Verteilung

Die  $\chi^2$  Verteilung ist eine Verteilung, die insbesondere bei der Analyse der Güte von Fits Verwendung findet.

Wir betrachten zunächst zwei unkorrelierte, normalverteilte Zufallsvariablen  $\epsilon_1$  und  $\epsilon_2$  mit ihren Erwartungswerten  $\mu_1 = \mu_2 = 0$  und Varianzen  $\sigma_1$  und  $\sigma_2$ . Dann ist die Wahrscheinlichkeit, Werte in den Intervallen  $[\epsilon_1, \epsilon_1 + d\epsilon_1]$  und  $[\epsilon_2, \epsilon_2 + d\epsilon_2]$  zu finden, gegeben durch:

$$f(\epsilon_1, \epsilon_2) d\epsilon_1 d\epsilon_2 = \frac{1}{\sqrt{2\pi}\sigma_1} \exp\left(-\frac{\epsilon_1^2}{2\sigma_1^2}\right) \frac{1}{\sqrt{2\pi}\sigma_2} \exp\left(-\frac{\epsilon_2^2}{2\sigma_2^2}\right) d\epsilon_1 d\epsilon_2. \quad (2.3.60)$$

Wir definieren neue Variablen  $x_1 = \frac{\epsilon_1}{\sigma_1}$  und  $x_2 = \frac{\epsilon_2}{\sigma_2}$ , damit wird ?? zu:

$$f(\epsilon_1, \epsilon_2) dx_1 dx_2 = \frac{1}{2\pi} \exp\left(-\frac{1}{2}(x_1^2 + x_2^2)\right) dx_1 dx_2. \quad (2.3.61)$$

Weiterhin definieren wir die Summe der Quadrate:  $S = x_1^2 + x_2^2$ .

Die Wahrscheinlichkeit, dass  $S$  in einem Intervall  $[S, S + dS]$  liegt, entspricht also der Fläche eines Rings um den Ursprung mit Radius  $\chi$  und Breite  $d\chi$ . Um dies zu berechnen, wechseln wir in Polarkoordinaten:

$$x_1 = \chi \cos(\theta) \quad (2.3.62)$$

$$x_2 = \chi \sin(\theta) \quad (2.3.63)$$

$$\chi^2 = S = x_1^2 + x_2^2 \quad (2.3.64)$$

$$(2.3.65)$$

Damit finden wir:

$$f(\epsilon_1, \epsilon_2) dx_1 dx_2 = f(\chi, \theta) \chi d\chi d\theta = \frac{1}{2\pi} \exp\left(-\frac{1}{2}\chi^2\right) \chi d\chi d\theta, \quad (2.3.66)$$

und nach Integration über  $\theta$ :

$$f(\chi) \chi d\chi = \exp\left(-\frac{1}{2}\chi^2\right) \chi d\chi. \quad (2.3.67)$$

Dies können wir ausdrücken als Funktion von  $\chi^2$ :

$$f(\chi^2) d\chi^2 = \frac{1}{2} \exp\left(-\frac{1}{2}\chi^2\right) d\chi^2, \quad (2.3.68)$$

und ohne die Differentiale finden wir die PDF der Zufallsvariable  $\chi^2$ :

$$f_2(\chi^2) = \frac{1}{2} \exp\left(-\frac{1}{2}\chi^2\right). \quad (2.3.69)$$

Dies ist die  $\chi^2$  Verteilung für zwei Variablen, also für zwei Freiheitsgrade. Verallgemeinert für  $k$  Freiheitsgrade (bzw.  $k$  unabhängige, normalverteilte Zufallsvariablen) ergibt sich:

$$f_k(\chi^2) = \frac{(\chi^2)^{\frac{k-2}{2}}}{s^{\frac{k}{2}} (\frac{k}{2} - 1)!} \exp\left(-\frac{1}{2}\chi^2\right). \quad (2.3.70)$$

Diese Funktion ist mühsam zu berechnen. Man findet ihre Werte in Tabellen oder in Python in der Bibliothek SciPy:

```
from scipy.stats import chi2
```

Um  $f_n(\chi^2)$  zu berechnen, kann dann die folgende Funktion verwendet werden:

```
f_X2_k = chi2.pdf(X2, k)
```



Wir listen hier einige Eigenschaften der  $\chi^2$  Verteilung auf, ohne diese herzuleiten. Die Mode der  $\chi^2$  Verteilung, also ihr Maximum, wird erreicht für:

$$\chi_{\text{mode}}^2 = k - 2. \quad (2.3.71)$$

Der Erwartungswert der Verteilung ist:

$$E(\chi^2) = k, \quad (2.3.72)$$

und ihre Varianz ist

$$\text{var}(\chi^2) = 2k. \quad (2.3.73)$$

Oft ist es praktisch, die reduzierte  $\chi^2$ -Verteilung zu verwenden:  $\chi_{\text{red}}^2 = \frac{1}{k}\chi^2$ . Dies hat den Vorteil, dass  $E(\chi_{\text{red}}^2) = 1$  ist, unabhängig von  $k$ . Dies gilt jedoch nicht für ihre Varianz:  $\text{var}(\chi_{\text{red}}^2) = \frac{2}{k}$ .

## 2.4 Fehler des Mittelwertes und die Integrationszeit

Nachdem wir nun den frequentistischen Wahrscheinlichkeitsbegriff eingeführt und damit unsere erste intuitive Interpretation des Histogramms als Annäherung der Wahrscheinlichkeitsverteilung unserer Messgrösse fundiert haben, wollen wir uns einem weiteren Aspekt des Histogramms in Fig. ?? zuwenden. Bei näherer Betrachtung ziehen wir folgende Vermutung: Während die Unsicherheit einer einzelnen Messung durch die Standardabweichung  $\Delta_x$  gegeben ist, können wir aus einer grossen Anzahl Datenpunkten den Mittelwert viel genauer abschätzen. Im Histogramm sehen wir beispielsweise bereits sehr deutlich, in welchen Bin der Erwartungswert sein sollte. Die Standardabweichung scheint also die Präzision unserer Messung des Mittelwerts nicht fundamental zu begrenzen, wenn wir bereit sind, genug Zeit für viele Messungen aufzubringen. Um diese Vermutung mathematisch beweisen, berechnen wir den zufälligen Fehler  $\Delta\mu$ , den wir bei der Abschätzung des Erwartungswertes  $\mu$  aus dem Mittelwert  $\bar{x}$  einer begrenzten Anzahl von Messwerten machen. Wir bezeichnen diesen Schätzfehler als “Fehler des Mittelwertes” (engl: Error of the Mean). Wir führen also eine Messung mit  $N$  Werten durch und berechnen daraus einen Mittelwert. Diese Messreihe wiederholen wir nun viele Male und bilden die Varianz der einzelnen Mittelwerte. Daraus können wir erschliessen, wie unterschiedlich die verschiedenen Mittelwerte voneinander sind. Für diese Varianz gilt:

$$\Delta\mu^2 = \text{var}(\bar{x}) = \text{var}\left(\frac{1}{N} \sum_{n=1}^N x_n\right) = \frac{1}{N^2} \text{var}\left(\sum_{n=1}^N x_n\right) = \frac{1}{N^2} N\sigma^2 = \frac{1}{N} \sigma^2. \quad (2.4.1)$$

Die Wurzel der Varianz zeigt uns die Unsicherheit des Mittelwerts, und somit den Fehler, den wir bei der Abschätzung des Erwartungswertes aus dem Mittelwert von  $N$  Messungen machen. Dieser Fehler ist somit  $\Delta\mu = \frac{\sigma}{\sqrt{N}}$ . Wir können durch wiederholtes Messen also die Genauigkeit des Messergebnisses verbessern. Generell halten wir fest: Die Werte von  $\mu$  und  $\sigma$  ändern sich nicht systematisch durch wiederholtes Messen. Die zugehörigen Schätzfehler, die wir bei der Annäherung dieser Werte mit den empirischen Werten machen, ändern sich allerdings sehr wohl. Im Limit unendlich vieler Messungen ( $N \rightarrow \infty$ ) entsprechen empirischer Mittelwert und empirische Standardabweichungen den tatsächlichen Parametern der zugrundeliegenden Verteilungen, solange keine systematischen Fehler vorliegen.

## 2.5 Fehlerfortpflanzung

Wir haben nun gelernt, wie wir den Fehler (die Unsicherheit) der Messung einer Variablen charakterisieren können. Häufig können wir allerdings die Grösse, an der wir interessiert sind,

nicht direkt messen. Stattdessen messen wir experimentell zugängliche Parameter und berechnen aus diesen die eigentliche Zielgrösse. Zum Beispiel kann die Erdbeschleunigung, die wir im vorangegangenen Beispiel betrachtet haben, auch experimentell mit einem Fadenpendel bestimmt werden

$$g = \frac{4\pi^2 l}{T_{\text{Periode}}^2} . \quad (2.5.1)$$

Hierbei werden die Fadenlänge  $l$  und Periodendauer  $T_{\text{Periode}}$  experimentell gemessen und die Erdbeschleunigung entsprechend berechnet. Um zu verstehen, wie sich die Fehler der Messungen von  $l$  und  $T_{\text{Periode}}$  auf den Fehler der Bestimmung von  $g$  auswirken, müssen wir eine sogenannte Fehlerfortpflanzung durchführen. Die Fehlerfortpflanzung ermöglicht es uns, den Einfluss von Fehlern in den einzelnen gemessenen Variablen auf den Fehler des berechneten Resultates zu bestimmen. Die exakte Formel hierfür kann grundsätzlich sehr komplex sein. Im Folgenden wollen wir eine Methode zur Fehlerfortpflanzung näher betrachten, die es uns ermöglicht, den Fehler zumindest im Fall von normalverteilten PDF auf eine einfache Weise anzunähern.

### 2.5.1 Fehlerfortpflanzung nach Gauss

Wir betrachten zunächst den einfachsten Fall, in dem unsere Zielgrösse  $f(x)$  nur von einer einzigen, fehlerbehafteten Messgrösse  $x$  abhängt. Um zu analysieren, wie sich eine Unsicherheit  $\Delta x$  in  $x$  auf die Unsicherheit  $\Delta f(x)$  von  $f(x)$  auswirkt, bilden wir die Taylorentwicklung von  $f(x)$  am Ort  $\bar{x}$  und brechen diese nach dem ersten Glied ab (lineare Näherung, Taylorentwicklung erster Ordnung):

$$f(x) = f(\bar{x} + \Delta x) \approx f(\bar{x}) + \left. \frac{\partial f}{\partial x} \right|_{\bar{x}} (x - \bar{x}) = f(\bar{x}) + \left. \frac{\partial f}{\partial x} \right|_{\bar{x}} \Delta x. \quad (2.5.2)$$

Wir erhalten somit den folgenden Zusammenhang zwischen dem Fehler der Zielgrösse,  $\Delta f(x)$ , der aus der Abweichung  $\Delta x$  entsteht:

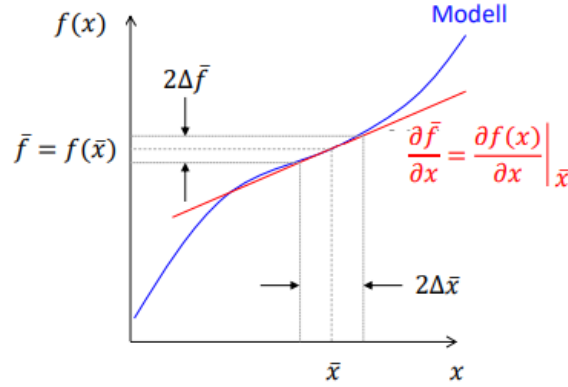
$$\Delta f(x) = f(\bar{x} + \Delta x) - f(\bar{x}) = \left. \frac{\partial f}{\partial x} \right|_{\bar{x}} \Delta x. \quad (2.5.3)$$

Unter der Annahme, dass die Messung von  $x$  einer Normalverteilung folgt, können wir die Standardabweichung  $\sigma_x$  als typischen Fehler  $\Delta x$  einsetzen und wollen auch  $\Delta f(x)$  als Standardabweichung  $\sigma_f$  interpretieren.

$$\sigma_f = \left. \frac{\partial f}{\partial x} \right|_{\bar{x}} \sigma_x. \quad (2.5.4)$$

Folglich erhalten wir für den Zusammenhang der Varianzen

$$\sigma_f^2 = \left( \left. \frac{\partial f}{\partial x} \right|_{\bar{x}} \right)^2 \sigma_x^2. \quad (2.5.5)$$



**Abbildung 2.5:** Veranschaulichung des Prinzips der Gaußschen Fehlerfortpflanzung im eindimensionalen Fall.

Wir betrachten nun den etwas komplizierteren Fall, in dem unsere Zielgrösse  $f(x, y)$  von zwei fehlerbehafteten Messgrössen  $x, y$  abhängt. Analog zum eindimensionalen Fall versuchen wir, einen Zusammenhang zwischen der Varianz der Zielgrösse und den Varianzen der Messgrössen  $\sigma_x, \sigma_y$  herzustellen. Hierbei verwenden wir die Taylorentwicklung erster Ordnung für beide Variablen. Der Übersichtlichkeit halber verwenden wir  $\bar{f} = f(\bar{x}, \bar{y})$ ,  $f_n = f(x_n, y_n)$  und  $\left. \frac{\partial f}{\partial x} \right|_{\bar{x}, \bar{y}} = \frac{\partial \bar{f}}{\partial x}$ :

$$\begin{aligned} \sigma_f^2 &= \frac{1}{N-1} \sum_{n=1}^N (f_n - \bar{f})^2 = \frac{1}{N-1} \sum_{n=1}^N \left( \frac{\partial \bar{f}}{\partial x} (x - \bar{x}) + \frac{\partial \bar{f}}{\partial y} (y - \bar{y}) \right)^2 \\ &= \frac{1}{N-1} \sum_{n=1}^N \left[ \left( \frac{\partial \bar{f}}{\partial x} \right)^2 (x - \bar{x})^2 + \left( \frac{\partial \bar{f}}{\partial y} \right)^2 (y - \bar{y})^2 + 2 \frac{\partial \bar{f}}{\partial x} \frac{\partial \bar{f}}{\partial y} (x - \bar{x})(y - \bar{y}) \right] \\ &= \left( \frac{\partial \bar{f}}{\partial x} \right)^2 \sigma_x^2 + \left( \frac{\partial \bar{f}}{\partial y} \right)^2 \sigma_y^2 + 2 \frac{\partial \bar{f}}{\partial x} \frac{\partial \bar{f}}{\partial y} \sigma_{xy}^2 \end{aligned} \quad (2.5.6)$$

Wie aus Gleichung ?? ersichtlich wird, hängt der Fehler der Zielgrösse im mehrdimensionalen Fall zusätzlich zu den Varianzen der einzelnen Messgrössen ( $\sigma_x \sigma_y$ ) auch noch von der sogenannten Kovarianz  $\sigma_{xy}^2$  ab, die wir im folgenden Kapitel ?? noch genauer betrachten wollen. Den in Gleichung ?? erhaltenen Ausdruck können wir auch unter Verwendung der sogenannten Kovarianzmatrix schreiben als:

$$\sigma_f^2 = (\partial_x, \partial_y) f \begin{pmatrix} \sigma_x^2 & \sigma_{xy}^2 \\ \sigma_{xy}^2 & \sigma_y^2 \end{pmatrix} \begin{pmatrix} \partial_x \\ \partial_y \end{pmatrix} f \quad (2.5.7)$$

Dieser Formalismus kann auf beliebig viele Variablen erweitert werden, mit entsprechend höherer Dimensionalität der Kovarianzmatrix. Für unkorrelierte Variablen (d.h. alle Kovarianzen sind null, die Kovarianzmatrix ist diagonal) erhalten wir mit diesem Verfahren der Fehlerfortpflanzung:

$$\sigma_f^2 = \sum_{n=1}^N \left( \frac{\partial f}{\partial x_n} \right)^2 \sigma_{x_n}^2, \quad (2.5.8)$$

$$\sigma_f = \sqrt{\sum_{n=1}^N \left( \frac{\partial f}{\partial x_n} \right)^2 \sigma_{x_n}^2}. \quad (2.5.9)$$

Folglich gilt für unkorrelierte Variablen, dass die Varianz der Summe gleich der Summe der Varianzen ist.

Dies ist die sogenannte **Gauss-Methode** der Fehlerrechnung. Sie setzt voraus, dass:

- die  $\sigma_n$  klein genug sind, um eine lineare Taylor-Approximation annehmen zu dürfen.
- die  $x_n$  statistisch unabhängig sind, so dass gilt:  $\text{var}(x_1 + x_2) = \text{var}(x_1) + \text{var}(x_2)$ . Falls die  $x_n$  nicht statistisch unabhängig sind, nennen wir sie korreliert und es treten zusätzliche Kovarianzen auf (siehe Kapitel ??)

Wir wollen die Gaussche Fehlerfortpflanzung nun für unser Beispiel der Bestimmung der Erdbeschleunigung mittels der Messung der Schwingungsperiode und Länge eines Fadenpendels verwenden. Wir messen die Schwingungsperiode  $T_{\text{Periode}}$  zu:

$$T_{\text{Periode}} = (2.50 \pm 0.08)\text{s}$$

$\uparrow$   
 $\bar{T}$

$\uparrow$   
 $\sigma_T$

und die Fadenlänge  $l$  zu:

$$l = (1.55 \pm 0.02)\text{m}.$$

$\uparrow$   
 $\bar{l}$

$\uparrow$   
 $\sigma_l$

Mit der Fehlerfortpflanzung nach Gauss ergeben sich folgende partielle Ableitungen:

$$\frac{\partial g}{\partial T_{\text{Periode}}} = -\frac{8\pi^2 l}{T_{\text{Periode}}^3}, \quad (2.5.10)$$

$$\frac{\partial g}{\partial l} = \frac{4\pi^2}{T_{\text{Periode}}^2}. \quad (2.5.11)$$

Damit folgt letztendlich:

$$\sigma_g = \sqrt{\left(-\frac{8\pi^2 l}{T_{\text{Periode}}^3}\right)^2 \sigma_T^2 + \left(\frac{4\pi^2}{T_{\text{Periode}}^2}\right)^2 \sigma_l^2}, \quad (2.5.12)$$

wobei wir  $\bar{T}$  und  $\bar{l}$  einsetzen.

### 2.5.2 Signifikante Stellen

Aus den vorangegangenen Diskussionen dieses Kapitels haben Sie bereits gelernt, dass keine experimentell gemessene Zahl absolut genau ist. Dieser Umstand sollte sich auch in der Anzahl Stellen widerspiegeln, mit denen ein numerisches Resultat angegeben wird. Es ist wichtig, Resultate mit der richtigen Anzahl Stellen anzugeben, denn zu viele angegebene Stellen suggerieren eine falsche Genauigkeit. Die Anzahl Stellen vor und hinter dem Komma, die man mit Sicherheit angeben kann, wird als *Anzahl signifikanter Stellen* bezeichnet.

Wir runden Zahlen auf die letzte Ziffer, die vom berechneten Messfehler betroffen ist. Der Messfehler wiederum wird üblicherweise auf eine signifikante Stelle gerundet. Zum Beispiel würden wir das Ergebnis der Messung einer Masse mit Mittelwert  $m = 3.25118$  kg mit Standardabweichung  $\sigma_m = 0.02$  kg angeben als  $m = 3.25$  kg  $\pm$  0.02 kg. Wir haben folgende Optionen, um Standardfehler (Fehler des Mittelwerts) anzugeben:

- Absoluter Fehler  $x + \Delta x$ , z.B. Die Masse beträgt  $m = 3.25 \text{ kg} \pm 0.02 \text{ kg}$
- Relativer Fehler  $x + \frac{\Delta x}{x}$ , z.B. Die Masse beträgt  $m = 3.25 \text{ kg} \pm 1\%$
- Implizite Angabe, z.B. Die Masse beträgt  $m = 3.25 \text{ kg}$ . Hier wird angenommen, dass die Zahl nicht genauer bekannt ist als die Hälfte der letzten angegebenen Ziffer (d.h. hier  $\pm 0.005$ ).

**Verständnisfragen:**

- 1 Wodurch entstehen Abweichungen zwischen dem gemessenen Mittelwert und dem «Literaturwert»?
- 2 Wodurch entsteht eine Standardabweichung in der gemessenen Verteilung von Messwerten?
- 3 Wie verändert sich nach wiederholtem Messen der Fehler in der Abschätzung des Erwartungswertes ( $\Delta\mu$ ) und die Standardabweichung ( $\sigma$ )?
- 4 Kann der Fehler in der Abschätzung des Erwartungswertes  $\Delta\mu$  kleiner werden als die Standardabweichung  $\sigma$ ?

**Antworten:**

- 1 Für unendlich viele Messpunkte tragen nur systematische Fehler zu dieser Abweichung bei. Für endlich viele Messpunkte werden auch statistische Fehler wichtig.
- 2 Die Standardabweichung ist ein gängiges Mass für die statistische Streuung von Messwerten.
- 3  $\sigma$  bleibt unverändert, aber  $\Delta\mu$  wird kleiner.
- 4 Ja.

## Kapitel 3

# Korrelierte Messgrößen

Für dieses Kapitel ist ein Jupyter-Notebook verfügbar. Siehe Kovarianz.ipynb

Bislang haben wir in unseren Beispielen immer nur eine einzige Zufallsvariable gemessen oder Fälle betrachtet, in denen alle Zufallsvariablen voneinander unabhängig waren. In realen Experimenten treten allerdings häufig Korrelationen der Messgrößen auf. In diesem Fall geht eine Änderung in einer Variable mit einer Änderung der anderen Variable einher. Zum Beispiel könnte die Frequenz eines Resonators mit der Raumtemperatur korreliert sein. Eine derartige Korrelation muss in die Fehleranalyse einbezogen werden, wie wir in Gleichung ?? sehen.

Bei der Analyse und Interpretation der Korrelation ist allerdings Vorsicht geboten: Korrelation wird häufig verwendet, um einen kausalen Zusammenhang aufzuzeigen. Allerdings bedeutet das Vorliegen einer Korrelation zweier Variablen  $x$  und  $y$  nicht automatisch, dass ein kausaler Zusammenhang zwischen ihnen besteht. Zwei Variablen  $x$  und  $y$  können etwa indirekt durch eine dritte Variable  $z$  verbunden sein. Wenn  $z$  sowohl  $x$  als auch  $y$  beeinflusst, können letztere eine Korrelation aufweisen, ohne dass sie direkt kausal verbunden sind. In unserem Beispiel des Resonators könnte zum Beispiel die Luftfeuchtigkeit im Raum der Grund dafür sein, dass sowohl die Temperatur als auch die Frequenz ansteigen oder abfallen.

### 3.1 Kovarianz

Um einen linearen Zusammenhang zwischen zwei Variablen aufzuzeigen, verwenden wir die geschätzte Kovarianz

$$\text{cov}(x, y) = \frac{1}{N-1} \sum_{n=1}^N (x_n - \bar{x})(y_n - \bar{y}). \quad (3.1.1)$$

In Messungen mit vielen Datenpunkten (mit  $\frac{1}{N-1} \rightarrow \frac{1}{N}$ ) wird diese Formel häufig vereinfacht zu:

$$\sigma_{xy}^2 = \text{cov}(x, y) = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(y_n - \bar{y}). \quad (3.1.2)$$

Die Kovarianz hat die Einheit  $[\sigma_{xy}^2] = [x] \times [y]$ . Damit hängt der Wert der Kovarianz von den gewählten Einheiten ab und hat keine universelle Bedeutung. Beispielsweise ändert sich der Zahlenwert einer Kovarianz, wenn wir Nanometer statt Meter als Längeneinheit festlegen. Um eine einheitenlose Zahl mit einer universellen Bedeutung zu erhalten, definieren wir den

normalisierten Korrelationskoeffizienten:

$$\rho_{xy} = \frac{\text{cov}(x, y)}{\sigma_x \sigma_y} \in [-1, 1]. \quad (3.1.3)$$

Hierbei bedeutet:

$$\rho_{xy} = \begin{cases} 0, & \text{unkorrelierte Variablen} \\ \pm 1, & \text{maximal (anti-)korrelierte Variablen für } y = ax. \end{cases} \quad (3.1.4)$$

Die Varianzen und Kovarianzen können kompakt in der sogenannten Kovarianz-Matrix zusammengefasst werden, die Sie bereits in Abschnitt ?? kennengelernt haben:

$$\begin{pmatrix} \sigma_x^2 & \sigma_{xy}^2 & \cdots \\ \sigma_{yx}^2 & \sigma_y^2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}. \quad (3.1.5)$$

Wie oben bereits erwähnt, misst die Kovarianz allerdings nur lineare Korrelationen, nichtlineare Fälle müssen anders behandelt werden. Um das zu verdeutlichen, berechnen wir die Kovarianz für die nichtlineare Funktion  $y = x^2$ . Hier sei  $x$  eine Zufallsvariable mit einer symmetrischen Verteilungsfunktion, das heisst  $-x$  ist gleich wahrscheinlich wie  $x$ . Für die Kovarianz erhalten wir

$$\begin{aligned} \text{cov}(x, y) &= \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(y_n - \bar{y}) \\ &= \frac{1}{N} \sum_{n=1}^N x_n (x_n^2 - \bar{y}) \\ &= \frac{1}{N} \sum_{n=1}^N x_n^3 - \frac{1}{N} \sum_{n=1}^N x_n \bar{y} = 0 \quad \text{für } N \rightarrow \infty, \end{aligned} \quad (3.1.6)$$

was verdeutlicht, dass nichtlineare Korrelationen mit erweiterten Methoden untersucht werden müssen.

Wir wollen lineare Korrelationen nun am Beispiel von einfachen trigonometrischen Funktionen veranschaulichen. Hierzu generieren wir zunächst drei künstliche, quasi analoge Signale  $U_1$ ,  $U_2$  und  $U_3$ . Die Signale  $U_1$  und  $U_3$  sind Sinusfunktionen verschiedener Amplituden, aber gleicher Frequenzen. Signal  $U_2$  ist eine Kosinusfunktion mit gleicher Amplitude und Frequenz wie  $U_1$ . Alle Signale werden mit identischem Rauschen überlagert.

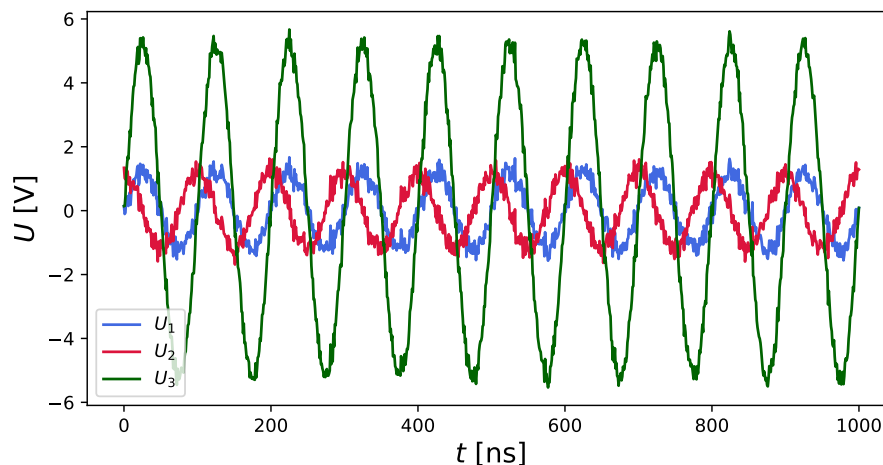
```
N = 1001 # Laenge des Datenvektors
t = np.linspace(0, N-1, N) # Zeitvektor von Punkten mit Abstand 1 in Einheit von
    Nanosekunden (als konkretes Beispiel)

f_sig = 0.01 # Frequenz
Amp1 = 1.2 # Amplitude 1
Amp3 = 5.2 # Amplitude 2
sig1 = Amp1*np.sin(2*np.pi*f_sig*t) # Signal 1
sig2 = Amp1*np.cos(2*np.pi*f_sig*t) # Signal 2
sig3 = Amp3*np.sin(2*np.pi*f_sig*t) # Signal 3
noise_amp = 0.2 # Amplitude des Rauschens
noise1 = np.random.normal(0, noise_amp, N) # normalverteiltes Rauschen mit
    Standardabweichung noise_amp
noise2 = np.random.normal(0, noise_amp, N) # normalverteiltes Rauschen mit
    Standardabweichung noise_amp
```



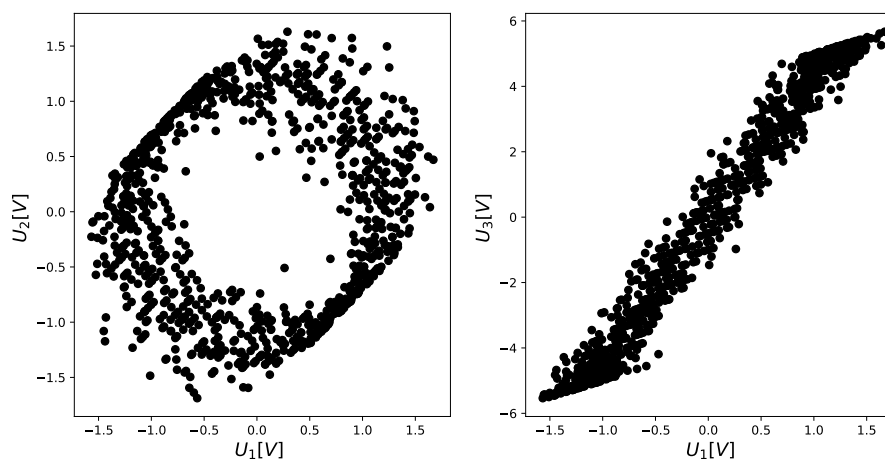
```
noise3 = np.random.normal(0, noise_amp, N) # normalverteiltes Rauschen mit
      Standardabweichung noise_amp
U1 = sig1 + noise1 # addiere Rauschen zu Signal 1
U2 = sig2 + noise1 # addiere Rauschen zu Signal 2
U3 = sig3 + noise1 # addiere Rauschen zu Signal 3
```

In einem ersten Schritt können wir diese Daten nun einfach visualisieren (siehe Figur ??). Diese Darstellung lässt bereits erahnen, dass  $U_1$  und  $U_3$  korreliert sind, doch vor allem für  $U_1$  und  $U_2$  ist aus dieser Darstellung nur schwer ersichtlich, ob tatsächlich eine Korrelation vorliegt.



**Abbildung 3.1:** Visualisierung der künstlich generierten, quasi-analogen und rauschbehafteten Signale  $U_1$ ,  $U_2$  und  $U_3$  als Funktion der Zeit  $t$ .

Zur graphischen Überprüfung linearer Korrelation stellen wir nun  $U_2$  und  $U_3$  als Funktion von  $U_1$  dar (siehe Figur ??). Aus dieser Darstellung ist direkt ersichtlich, dass  $U_1$  und  $U_3$  stark korreliert sind,  $U_1$  und  $U_2$  hingegen nur sehr schwach. Diese schwache Korrelation von  $U_1$  und  $U_2$  kommt dadurch zustande, dass beiden Signalen identisches Rauschen anhaftet.



**Abbildung 3.2:** Um eine mögliche Korrelation zwischen zwei Variablen graphisch zu überprüfen, stellt man die eine Variable als Funktion der anderen dar. **Links:**  $U_2$  als Funktion von  $U_1$ . Die Punkte liegen nicht auf einer Geraden, es liegt also keine starke lineare Korrelation vor. **Rechts:**  $U_3$  als Funktion von  $U_1$ . Wir sehen eine starke, lineare Korrelation der beiden Variablen.

Wir berechnen nun die oben eingeführte Kovarianz sowie die normalisierten Korrelationskoeffizienten.

```
# U1 U3
```

```

C = np.stack((U1,U3), axis=0)
cov13 = np.cov(C)
corcov13 = np.corrcoef(C)

# U1 U2
D = np.stack((U1, U2), axis = 0)
cov12 = np.cov(D)
corcov12 = np.corrcoef(D)

```

Die so erhaltenen Korrelationskoeffizienten  $\rho_{U_1,U_2} = 0.06$  und  $\rho_{U_1,U_3} = 0.98$  stützen unsere anfängliche, qualitative Analyse.

Als nächstes simulieren wir die Messung unserer quasi-analogen Signale mit einem rauschbehafteten Analog-Digital-Konvertierer (engl: Analog-Digital-Converter ADC). Das so hinzugefügte Rauschen ist nun für jedes Signal unterschiedlich.

```

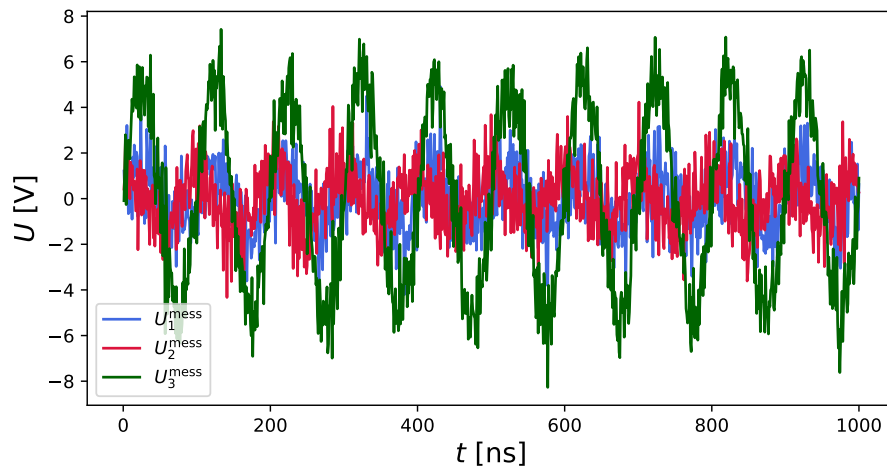
Delta_t = 1 # zeitl. Abstand der Messpunkte in den gewaehlten Einheiten: fuer
            # einen Zeitvektor t mit Abstaenden von je 1 Nanosekunde misst unser ADC einen
            # Datenpunkt alle Delta_t Nanosekunden
U_max = 10 # maximal messbarer Wert: alle groesseren Werte werden als U_max
           # angezeigt (clipping)
U_min = 0.01 # minimaler messbarer Spannungsunterschied der Signalwerte:
             # Spannungsaufloesung des ADC
U_noise = 1 # Standardabweichung des Spannungsrauschens, das dem Signal durch
            # den Messprozess hinzugefuegt wird

# Initialisierung der Messvektoren
n_mess = math.floor(N/Delta_t)-1 # Anzahl gemessener Punkte. floor rundet ab
n = range(n_mess) # wird fuer for loop benoetigt, enthaelt 0 und n_mess als
                  # untere und obere Grenze von n
t_mess = np.zeros(n_mess) # leerer Vektor, um die Zeitwerte zu erfassen
U1_mess = np.zeros(n_mess) # leerer Vektor, um die Spannungswerte zu erfassen
U2_mess = np.zeros(n_mess) # leerer Vektor, um die Spannungswerte zu erfassen
U3_mess = np.zeros(n_mess) # leerer Vektor, um die Spannungswerte zu erfassen

for i in n:
    t_mess[i] = t[(i+1)*Delta_t] # jeder (i+1)*Delta_t-te Punkt wird gemessen
    U1_mess[i] = np.clip(U_min*round((U1[(i+1)*Delta_t]+np.random.normal(0,
        U_noise))/U_min,0),-U_max,U_max) # numpy.clip limitiert die maximalen
        Werte auf [-U_max,U_max]
    U2_mess[i] = np.clip(U_min*round((U2[(i+1)*Delta_t]+np.random.normal(0,
        U_noise))/U_min,0),-U_max,U_max) # numpy.clip limitiert die maximalen
        Werte auf [-U_max,U_max]
    U3_mess[i] = np.clip(U_min*round((U3[(i+1)*Delta_t]+np.random.normal(0,
        U_noise))/U_min,0),-U_max,U_max) # numpy.clip limitiert die maximalen
        Werte auf [-U_max,U_max]

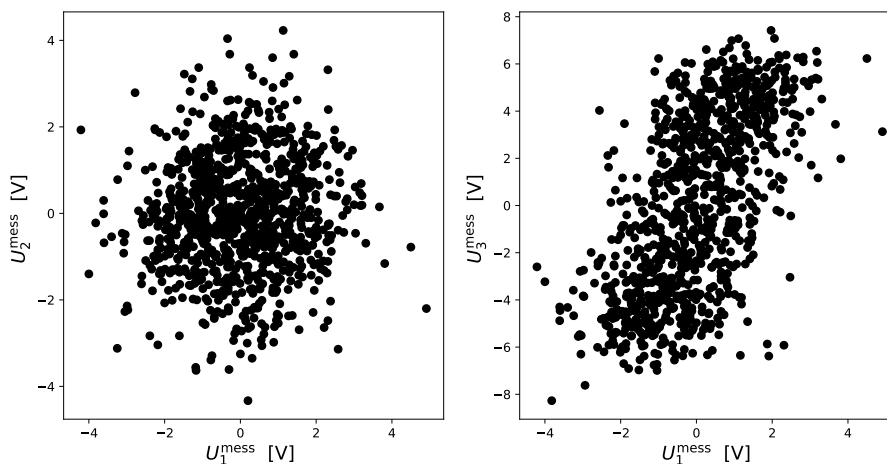
```

Wir können die so erhaltenen, “gemessenen” Signale  $U_1^{\text{mess}}$ ,  $U_2^{\text{mess}}$  und  $U_3^{\text{mess}}$  nun erneut als Funktionen der Zeit  $t$  als auch als Funktionen von  $U_1^{\text{mess}}$  darstellen.



**Abbildung 3.3:** Visualisierung der künstlich generierten Signale  $U_1^{\text{mess}}$ ,  $U_2^{\text{mess}}$  und  $U_3^{\text{mess}}$  nach der Messung mit einem rauschbehafteten ADC als Funktion der Zeit  $t$ .

Sowohl die Darstellung in Abbildung ?? als auch die erneute Berechnung der Korrelationskoeffizienten zu  $\rho_{U_1^{\text{mess}}, U_2^{\text{mess}}} = 0.05$  und  $\rho_{U_1^{\text{mess}}, U_3^{\text{mess}}} = 0.63$  ergeben, dass das Hinzufügen dieses unkorrelierten Rauschens die Korrelation der Signale reduziert hat. Dieses einfache Beispiel verdeutlicht, dass in einem realen Messvorgang Korrelationen einzelner Variablen durch zu starkes Rauschen reduziert, bzw. vollkommen maskiert werden können.



**Abbildung 3.4:** Korrelationen zwischen den Signalen, nachdem sie von einem simulierten ADC (mit Rauschen) gemessen wurden. **Links:**  $U_2^{\text{mess}}$  als Funktion von  $U_1^{\text{mess}}$ . **Rechts:**  $U_3^{\text{mess}}$  als Funktion von  $U_1^{\text{mess}}$ .

Wir fassen zusammen: Ob zwei Signale korreliert sind oder nicht, ist nicht davon abhängig, ob sie Rauschen enthalten. Korrelation (selbst starke) kann beispielsweise durch korreliertes Rauschen entstehen. Gleichzeitig müssen rauschfreie Signale nicht korreliert sein, selbst wenn sie, wie in Abbildung ??, dieselbe Frequenz haben.

## 3.2 Auto-Kovarianz

Häufig sollen nicht zwei verschiedene Messgrößen miteinander korreliert werden, sondern eine Messgröße mit sich selbst in Abhängigkeit einer Zeitverschiebung. Durch Einsetzen von  $y = x_{i+\Delta}$  in Gleichung ?? erhält man die diskrete Auto-Kovarianz, also die Korrelation der Funktion  $x$

mit sich selber in Abhängigkeit einer Indexverschiebung  $\Delta$

$$R_{xx}(\Delta) = \frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(x_{n+\Delta} - \bar{x}). \quad (3.2.1)$$

Analog zum Korrelationskoeffizienten definieren wir den Autokorrelationskoeffizienten

$$\rho_{xx} = \frac{R_{xx}(\Delta)}{\sigma_x^2} \in [-1, 1]. \quad (3.2.2)$$

In vielen Anwendungen sind wir an der diskreten Autokovarianz als Funktion der Zeitverschiebung  $\tau = \Delta \times \delta t$  interessiert und verwenden diese Zeitverschiebung anstelle der Indexverschiebung als Variable, sodass wir

$$R_{xx}(\tau) = \frac{1}{N} \sum_{t=t_1}^{t_N} (x(t) - \bar{x})(x(t + \tau) - \bar{x}) \quad (3.2.3)$$

mit dem entsprechenden Autokorrelationskoeffizienten:

$$\rho_{xx} = \frac{R_{xx}(\tau)}{\sigma_x^2} \in [-1, 1]. \quad (3.2.4)$$

erhalten.

Theoretisch existiert für kontinuierliche Funktionen auch die kontinuierliche Auto-Kovarianz

$$R_{xx}(\tau) = \lim_{T \rightarrow \infty} \frac{1}{T} \int_0^T x(t)x(t + \tau)dt = \langle x(t)x(t + \tau) \rangle. \quad (3.2.5)$$

Diese kann allerdings nicht für diskrete Signale verwendet werden und ist daher in realen Anwendungen nicht von Relevanz. Wichtige Eigenschaften der Auto-Kovarianz sind wie folgt:

- Die Auto-Kovarianz ist symmetrisch um  $\Delta = 0$  (bzw.  $\tau = 0$ ).
- Die Samplingzeit  $\delta t$  (zeitlicher Abstand zwischen aufeinanderfolgenden Messpunkten) setzt ein unteres Limit für die zeitliche Auflösung der Messung, und damit für die ‘periodische Zeit’  $\tau$ .
- Die Auto-Kovarianz bei  $\Delta = 0$  ( $\tau = 0$ ) entspricht der Varianz und ist das Maximum der Funktion.
- Die Auto-Kovarianz von Gauss-verteilterm weissen Rauschen ist Null für alle  $\tau \neq 0$ , da weisses Rauschen per Definition von Punkt zu Punkt vollkommen unkorreliert ist.
- Die Auto-Kovarianz einer perfekten Sinuswelle ist als Funktion von  $\tau$  perfekt periodisch, da Punkte über jede Periode (und damit über jedes ganzzahlige Mehrfache der Periode) perfekt korreliert sind.

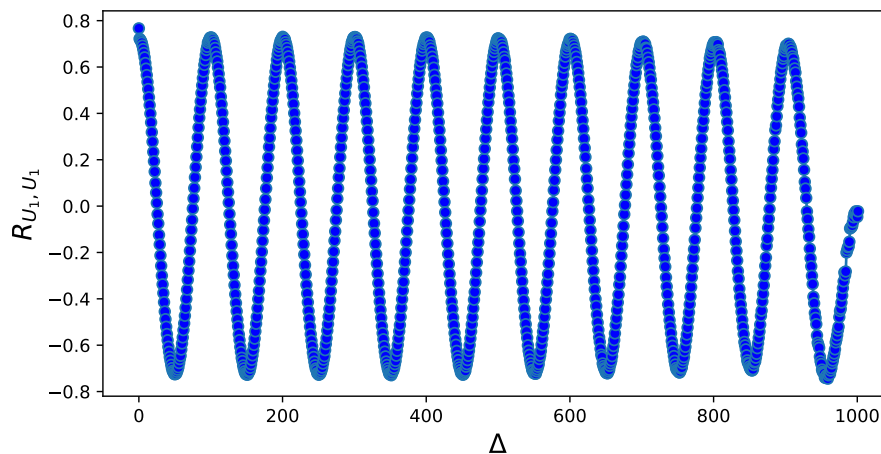
Die Auto-Kovarianz ist geeignet, um periodische zeitliche Zusammenhänge zu sehen. Ein typisches Beispiel ist die Bestimmung der Korrelationszeit (“Kohärenzzeit”) aus der Breite des Maximums um  $\tau = 0$ . Aperiodische zeitliche Information (wie z.B. ein kurzer Signalpuls) ist darin aber nicht mehr sichtbar.

Wir wollen nun für unser im Kovarianz-Kapitel eingeführtes Test-Signal  $U_1$  die Auto-Kovarianz berechnen. Hierfür definieren wir zunächst die entsprechende Auto-Kovarianzfunktion und wenden diese auf  $U_1$  an.

```
# definiere Auto-Kovarianzfunktion
def Rxx(x, delta):
    xm = np.mean(x)
    dev_sum = 0
    for i in range(len(x) - delta):
        dev_sum += (x[i] - xm) * (x[i + delta] - xm)
    return dev_sum / (len(x) - delta)

# wende Funktion auf Beispiele an
delta_max = len(U1)
Rxx_vs_delta = [Rxx(U1, i) for i in range(delta_max)]
lags = np.linspace(0, delta_max, num=delta_max)
```

Da  $U_1$  eine Sinusfunktion ist, erreicht die Auto-Kovarianzfunktion in periodischen Abständen Maxima und Minima (siehe ??). Wenn  $\Delta$  dem Vielfachen einer Periode von  $U_1$  entspricht, liegt eine hohe Korrelation vor, nach ungeraden Vielfachen einer halben Periode von  $U_1$  eine hohe Anti-Korrelation.



**Abbildung 3.5:** Autokorrelationsfunktion  $R_{U_1 U_1}(\Delta)$  von  $U_1$  als Funktion der Indexverschiebung  $\Delta$ .

#### Anwendungsbeispiel: Ultraschnelle Laserphysik

Ein reales Anwendungsgebiet der Autokorrelation ist die ultraschnelle Laserphysik (Engl: Ultrafast Laser Physics). Die Charakterisierung von ultraschnellen optischen Pulsen gestaltet sich häufig schwierig, weil die verfügbaren elektrischen Techniken nur über eine limitierte Zeitauflösung (1-10 Picosekunden) verfügen, die Dauer der Pulse allerdings nur wenige Femtosekunden beträgt. Um solche kurzen Pulse dennoch zu messen, wird das Verfahren der sogenannten *Optical Intensity Autocorrelation* angewandt. Der Laserstrahl wird zunächst in zwei Laserstrahlen gleicher Intensität geteilt. Einer der beiden Strahlen wird gegenüber dem anderen Strahl um eine Zeit  $\tau$  verzögert, indem die Länge des optischen Pfades dieses Strahles entsprechend angepasst wird. Anschliessend werden die beiden Strahlen mithilfe eines nichtlinearen Kristalls wieder überlagert und detektiert. Die Intensität des gemessenen Signales ist abhängig von der Verzögerung  $\tau$  und entspricht aus mathematischer Sicht einer Autokorrelationsfunktion. Mit diesem experimentellen Aufbau kann mit höherer Zeitauflösung das Intensitätsprofil des Eingangsstrahls bestimmt werden.

**Verständnisfragen:**

- 1 Welche Funktion ist nützlich, um einen systematischen Zusammenhang zwischen zwei Variablen aufzuzeigen?
- 2 In einem Experiment tauchen zwei Quellen von statistischen Fehlern auf ( $\sigma_x$  und  $\sigma_y$ ). Unter welchen Bedingungen kann der gemessene Fehler  $\sigma_f$  kleiner sein als die beiden einzelnen Fehlerquellen?

**Antworten:**

- 1  $\frac{1}{N} \sum_{n=1}^N (x_n - \bar{x})(y_n - \bar{y})$
- 2 Wenn die beiden Messgrößen korreliert sind.

## Kapitel 4

# Die Spektrale Leistungsdichte und Rauschen

Zur Fouriertransformation und Spektralen Leistungsdichte ist ein Jupyter-Notebook verfügbar. Siehe `Fouriersynthese.ipynb`

### 4.1 Information im Frequenzraum

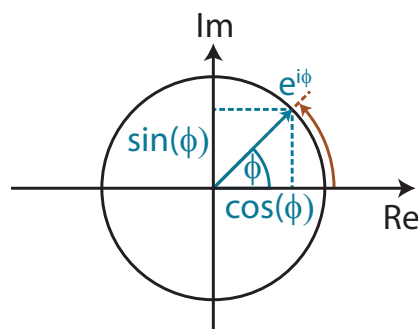
#### 4.1.1 Komplexe Zahlen und Oszillationen

Im Folgenden benötigen wir Exponentialfunktionen mit komplexen Argumenten, um Oszillationen darzustellen. Wir erinnern uns an die folgenden Reihenentwicklungen:

$$\begin{aligned}e^{\phi} &= 1 + \phi + \frac{\phi^2}{2!} + \frac{\phi^3}{3!} + \dots \\ \cos \phi &= 1 - \frac{\phi^2}{2!} + \frac{\phi^4}{4!} - \dots \\ \sin \phi &= \phi - \frac{\phi^3}{3!} + \frac{\phi^5}{5!} - \dots\end{aligned}\tag{4.1.1}$$

Hier nennen wir  $\phi$  eine “Phase”. Durch Einsetzen können wir überprüfen, dass ausserdem gilt

$$e^{i\phi} = \cos \phi + i \sin \phi.\tag{4.1.2}$$



**Abbildung 4.1:** Komplexe Zahl  $e^{i\phi}$  dargestellt als Vektor in der komplexen Zahlenebene. Wir können diese Zahl in Real- und Imaginärteil zerlegen,  $e^{i\phi} = \cos(\phi) + i \sin(\phi)$ . Wenn die Phase  $\phi$  linear zeitabhängig ist, rotiert der Zahlenvektor gegen den Uhrzeigersinn.

Wenn die Phase linear von der Zeit abhängig ist, setzen wir  $\phi = \omega t$  ein und erhalten eine Rotation im Gegenuhrzeigersinn in der komplexen Ebene:

$$e^{i\omega t} = \cos \omega t + i \sin \omega t. \quad (4.1.3)$$

Generell führen zeitabhängige reelle Argumente in der Exponentialfunktion entweder zu einem Wachstum oder einer Abnahme, je nach Vorzeichen. Imaginäre Argumente dagegen führen zu einer Oszillation. Die Form in ?? ist sehr praktisch, um harmonische Oszillationen darzustellen, da man den Realteil der Funktion mit der Auslenkung  $x(t)$  identifizieren kann und den Imaginärteil mit der entsprechenden zeitlichen Ableitung  $\dot{x}/\omega$  (die Teilung durch  $\omega$  ist nötig, um dieselbe Einheit wie für  $x$  zu erhalten). Damit das Vorzeichen der Ableitung stimmt, benutzen wir für die Darstellung harmonischer Oszillatoren oft

$$e^{-i\omega t} = \cos \omega t - i \sin \omega t. \quad (4.1.4)$$

Das negative Vorzeichen entspricht einer Rotation in der komplexen Ebene im Uhrzeigersinn.

### 4.1.2 Die Diskrete Fouriertransformation (DFT)

Sei  $x(t)$  eine Timetrace (d.h. gemessene Werte als Funktion der Zeit). Können wir die verschiedenen Frequenz-Komponenten dieses Signals einfach darstellen? Dafür ist folgendes Theorem wichtig:

Fouriers Theorem: Jede integrierbare Funktion kann als Summe von trigonometrischen Funktionen dargestellt werden.

Dieses Theorem führt zur **Fouriertransformation**. In dieser Vorlesung beschäftigen wir uns insbesondere mit der diskreten Fouriertransformation (DFT), da alle gemessenen Signale sowohl in der Zeit als auch in der Auslenkung diskret sind (d.h., nur gewisse Werte annehmen). Mit der Exponentialschreibweise nimmt die DFT eines Signals  $x(t)$  die folgende Form an:

$$x(t_k) = \sum_{n=-N/2}^{N/2} X(f_n) e^{i2\pi f_n t_k} \quad (4.1.5)$$

wobei  $\Delta_f = 1/t_{\text{tot}}$  die kleinste Frequenzdifferenz ist, die in der Messzeit  $t_{\text{tot}}$  aufgelöst werden kann, und  $f_n = n\Delta_f$  ist die  $n$ -te diskrete Frequenz, die wir als Funktion der diskreten Zeit  $t_k = k\Delta_t$  betrachten.

Die komplexen Koeffizienten  $X(f)$  geben an, wie gross die Amplitude der DFT an einer bestimmten Frequenz ist. Diese Koeffizienten können mit einer Rücktransformation bestimmt werden:

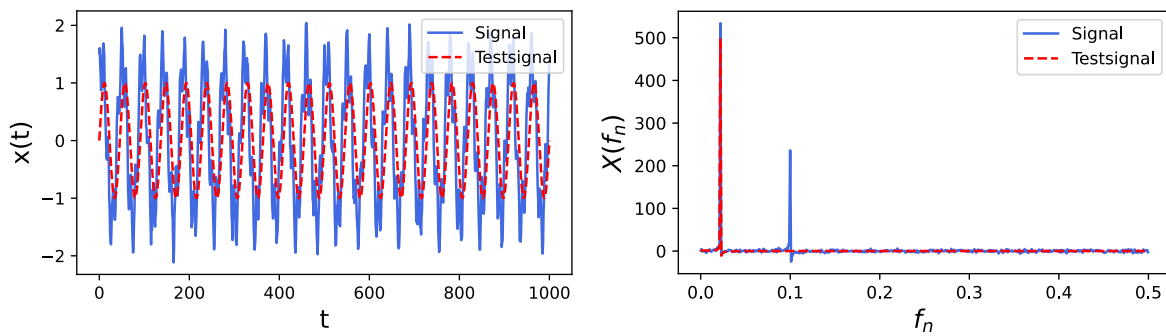
$$X(f_n) = \frac{1}{N} \sum_{k=0}^{N-1} x(t_k) e^{-i2\pi f_n t_k} \quad (4.1.6)$$

Wir können die  $X(f_n)$  als Kovarianz zwischen dem Signal  $x(t)$  und einem Testsignal  $e^{-i2\pi f_n t_k}$  an der Frequenz  $f_n$  vorstellen.  $X(f_n)$  gibt also an, wie gut die beiden Signale übereinstimmen. Ob die Normierung  $1/N$  in der Hin- oder in der Rücktransformation erfolgt, ist frei wählbar (es müssen aber andere Formeln entsprechend angepasst werden, siehe unten). Abb. ?? zeigt ein Beispiel für eine Fouriertransformation.

Man beachte, dass die rein mathematische Definition der DFT für eine einheitenlose Zahlenfolge  $x_n$  einfacher aussieht:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}. \quad (4.1.7)$$





**Abbildung 4.2:** Beispiel Fouriertransformation: Das gemessene Signal enthält zwei dominante Frequenzkomponenten, von denen eine mit dem Testsignal übereinstimmt. **Links:** Signal  $x(t)$  und Testsignal  $e^{i2\pi f_n t_k}$  **Rechts:** Fouriertransformation von  $x(t_k)$  und vom Testsignal.

Hier sind sowohl  $x_n$  als auch die transformierte Zahlenfolge  $X_k$  Zahlen ohne physikalische Bedeutung. Die Schrittgröße, welche die Rolle einer diskreten Integrationsvariablen einnimmt, reduziert sich hier auf 1 und wird unsichtbar. Diese Formel kann also nicht direkt auf physikalische Signale mit Einheiten angewendet werden. In Python, MATLAB oder anderen Programmen, wo die Fouriertransformation mit Hilfe der “fast Fourier transform” FFT (einer effizienten Variante der DFT) berechnet wird, muss man daher das Resultat der FFT nachträglich mit der Schrittgröße normieren. Generell ist immer Vorsicht geboten, wenn man einen Software-Algorithmus zum ersten Mal verwendet, da die Normierung nicht immer gleich implementiert ist. Im numpy-Paket von Python entspricht beispielsweise die funktion `numpy.fft` in unserer Notation nicht  $X(f_n)$ , sondern  $NX(f_n)$ . Man berechnet dann den spektralen Koeffizienten  $X(f_n)$  einer Messreihe  $x(t)$  mit dem folgenden Code:

```
FFT_x = np.fft.fft(x)/N # fast Fourier transform von x(t) mit N Messwerten
X_n = FFT_x[n] # Fourierkoeffizient mit Index n (für Details siehe Dokumentation)
```

Die Fouriertransformation kann entweder nur für positive (einseitig, single-sided) oder für positive und negative Frequenzen (zweiseitig, double-sided) definiert werden. In der zweiseitigen DFT eines Signals mit  $N$  Punkten in der Zeit liegen im Frequenzraum ebenfalls  $N$  Punkte vor, die zwischen den maximalen Frequenzen  $\pm f_{max}$  (siehe unten) verteilt sind. Negative Frequenzen haben keine zusätzliche Bedeutung; die Fouriertransformation ist symmetrisch im Realteil (bzw. antisymmetrisch im Imaginärteil). Aus diesem Grund werden die negativen Frequenzanteile oft weggelassen und man erhält  $N/2$  Punkte zwischen null und  $f_{max}$ .

### 4.1.3 Eigenschaften der DFT

- **Maximale Frequenz:** Die maximale Frequenz, die ohne Aliasing gesampelt werden kann, ist die sogenannte Nyquist-Frequenz:

$$f_{max} = \frac{1}{2\Delta_t} \quad (4.1.8)$$

Signale höherer Frequenz können mit derselben Samplingrate nicht richtig interpretiert werden und erzeugen im Spektrum Spitzen bei falschen Frequenzwerten.

Diese Limite für korrekt gemessene Frequenzen nennt man das Nyquist-Kriterium.

- **Frequenzauflösung:** Die DFT hat  $N$  Frequenzwerte von  $-f_{max}$  bis  $+f_{max}$ . Diese Frequenzwerte  $f_n$  entsprechen einer ganzen Zahl  $n$  mal der Auflösung:

$$\Delta f = \frac{1}{t_{tot}} = \frac{1}{N\Delta_t} = \frac{2f_{max}}{N} \quad (4.1.9)$$

also

$$f_n = n \times \Delta_f. \quad (4.1.10)$$

Anschaulich können wir uns merken, dass über die gesamte Messzeit  $t_{\text{tot}} = N\Delta t$  mindestens eine volle Oszillation Unterschied zwischen den Frequenzwerten entsteht, also  $\Delta_t \leq 1/t_{\text{tot}}$ .

- Die PSD einer perfekten, unendlich lange gemessenen Sinuswelle hat einen einzelnen Peak bei der Wellenfrequenz und ist überall sonst gleich null. Perfektes weisses Rauschen hat über das gesamte Spektrum denselben Wert.

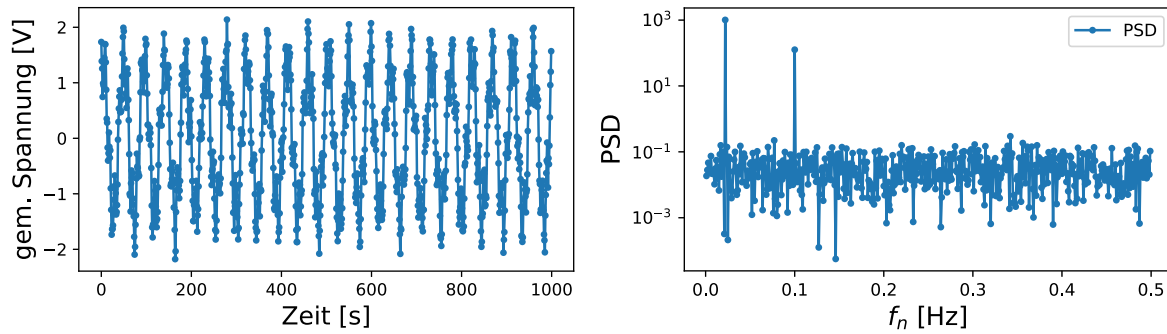
## 4.2 Die Spektrale Leistungsdichte

In vielen Anwendungen sind wir nicht an der Phase oder dem Vorzeichen einer Fourierkomponente interessiert, sondern nur daran, wieviel Leistung in einem bestimmten Frequenzintervall enthalten ist. Beispielsweise kann es wichtig sein, wieviel Leistung eine Lichtquelle im sichtbaren Teil des elektromagnetischen Spektrums abstrahlt. In derartigen Fällen wird das normierte Quadrat der Fouriertransformation verwendet – die sogenannte Leistungsdichte oder power spectral density (PSD):

$$S_{xx}(f_n) = \frac{\Delta_t}{N} \left| \sum_{k=0}^{N-1} x(t_k) e^{-i2\pi f_n t_k} \right|^2 = \Delta_t N |X(f_n)|^2, \quad (4.2.1)$$

Mit Python lässt sich die gesamte PSD aller Frequenzen beispielsweise so berechnen:

```
FFT_x = np.fft.fft(x)/N # normierte FFT von x(t), d.h. Liste von
                        Fourierkoeffizienten von x(t) mit N Messwerten
PSD = dt*N*np.abs(FFT_x)**2
```



**Abbildung 4.3:** Beispiel PSD. **Links:** Signal  $x(t)$  mit gemessenen Punkten angenommen wir messen einmal pro s **Rechts:** PSD der gemessenen Punkte (siehe .ipynb zu dieser Sektion)

Die PSD kann auch anders berechnet werden: Wenn wir die Fouriertransformation der Auto-Kovarianz  $R_{xx}(t)$  aus Gl. (??) bilden, erhalten wir die spektrale Leistungsdichte (power spectral density, PSD)  $S_{xx}$ :

$$S_{xx}(f) = \sum_{n=0}^N R_{xx}(\tau_n) e^{-i2\pi f \tau_n} \Delta\tau \quad (4.2.2)$$

mit der entsprechenden Rücktransformation

$$R_{xx}(\tau) = \frac{1}{2\pi} \sum_{n=0}^N S_{xx}(f_n) e^{i2\pi f_n \tau} \Delta f. \quad (4.2.3)$$

Dieser Zusammenhang zwischen der Auto-Kovarianz und der spektralen Leistungsdichte ist bekannt als das Wiener-Khinchin Theorem.

Abb. ?? zeigt, wie wir das Signal  $x(t)$  aus dem vorherigen Kapitel mit einem Analog-Digital-Konverter messen und dann die PSD dieser Messung berechnen.

Die Normierung mit  $\Delta_t$  in Gl. (??), bzw.  $\Delta_f$  in Gl. (??) macht aus einem Power Spectrum (mit derselben Einheit wie  $x^2$ ) eine PSD mit der Einheit  $x^2/\text{Hz}$  (zum Beispiel  $\text{m}^2/\text{Hz}$ ,  $\text{V}^2/\text{Hz}$ , ...).

#### 4.2.1 Eigenschaften der spektralen Leistungsdichte (PSD)

- Die PSD bildet das Spektrum der Leistung, welche im Gegensatz zur Amplitude der Welle nicht von der Wellenphase abhängt. Die PSD ist also überall positiv.
- Die Normierung der PSD ist so gewählt, dass ihre numerische Integration über alle Frequenzen der Varianz  $\sigma_x^2$  entspricht (siehe “Parseval-Theorem” in Sektion ??). Deshalb muss die PSD Einheiten von  $x^2/\text{Hz}$  haben. Im Gegensatz zum Leistungsspektrum, welches auch oft verwendet wird, ist der Wert der PSD an einer bestimmten Frequenz unabhängig von der Messzeit (abgesehen von statistischen Fehlern).
- Die PSD ist generell für positive und negative Werte von  $f$  bestimmt und symmetrisch um  $f = 0$ . Oft wird nur der positive Teil der PSD gezeigt. Dann muss die PSD mit einem Faktor 2 multipliziert werden, um die Normierung zu erhalten. Es wird deshalb immer angegeben, ob man die “single-sided” oder “double-sided PSD” benutzt.

### 4.3 Rauschen und Parsevals Theorem

Zu Rauschen und Parseval-Theorem ist ein Jupyter-Notebook verfügbar. Siehe Parseval theorem and filtering.ipynb

#### 4.3.1 Parseval-Theorem

Die wichtige Normierungseigenschaft der PSD, welche einen Zusammenhang zwischen der Varianz einer Variable  $x(t)$  und dessen PSD herstellt, ist bekannt als das Parseval-Theorem. Für den kontinuierlichen, doppelseitig definierten Fall lautet das Parseval-Theorem:

$$\sigma_x^2 = \int_{-\infty}^{\infty} S_{xx}(f) df, \quad (4.3.1)$$

Für den diskreten, doppelseitig definierten Fall entspricht dies:

$$\sigma_x^2 = \Delta_f \sum_{f=-f_{max}}^{f_{max}} S_{xx}(f). \quad (4.3.2)$$

Einseitig definierte PSDs haben einen um einen Faktor 2 grösseren Wert an jeder Stelle, damit das Integral (bzw. die Summe) von null bis  $f_{max}$  immer noch der Varianz entspricht.

Die PSDs unkorrelierter Variablen sind additiv, also gilt für ein Signal  $x(t) = a(t) + b(t)$

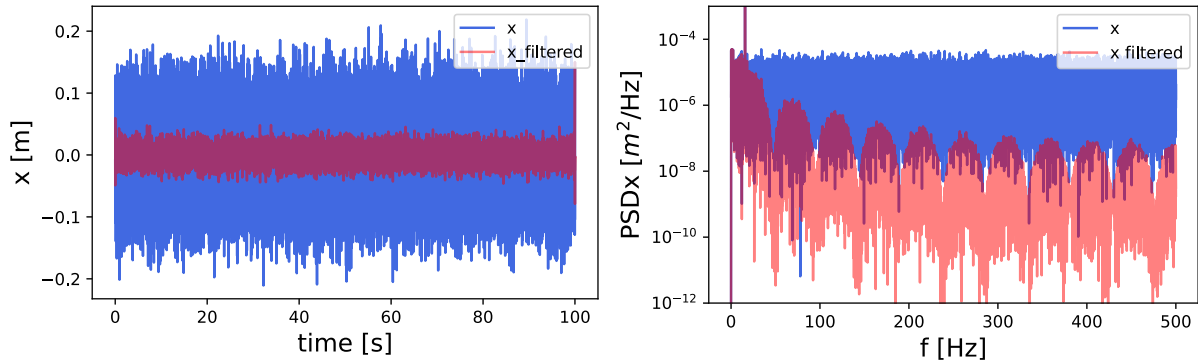
$$S_{xx}(f_n) = S_{aa}(f_n) + S_{bb}(f_n). \quad (4.3.3)$$

Aus dem Parseval-Theorem und Gl. (??) folgt dann die bereits bekannte Tatsache, dass auch die Varianzen unkorrelierter Variablen additiv sind, also

$$\sigma_x^2 = \sigma_a^2 + \sigma_b^2. \quad (4.3.4)$$

Diese Eigenschaft können wir uns zunutze machen, um zum Beispiel ein bekanntes Detektorrauschen von einem unbekannten Signal abzuziehen, wie in der Vorlesung gezeigt.

### 4.3.2 Rauschen in der PSD



**Abbildung 4.4:** Beispiel Rauschen und Tiefpassfilter in der PSD. **Links:** Signal  $x(t)$  mit unbekannten Frequenzkomponenten in blau und gefiltertes Signal in rot/ **Rechts:** PSD von  $x(t)$  in blau und vom gefilterten Signal in rot. Die Frequenzkomponente bei 15,75 Hz ist besonders ausgeprägt und der Tiefpassfilter macht diese Frequenz besser sichtbar.

In Abb. ?? ist eine Messdatenreihe und ihre PSD zu sehen. Eine Frequenzkomponente bei 15,75 Hz sticht im Spektrum heraus. Wir können einen *Tiefpassfilter* anwenden, um das Rauschen bei allen anderen Frequenzen zu dämpfen und so das Signal auch im Zeitraum besser sichtbar zu machen.

Wir implementieren einen einfachen digitalen Tiefpassfilter, indem wir jeden Punkt in  $x(t)$  mit seinen 10 Nachbarn zu jeder Seite mitteln. Wie erwartet finden wir, dass die PSD bei höheren Frequenzen deutlich reduziert wurde.

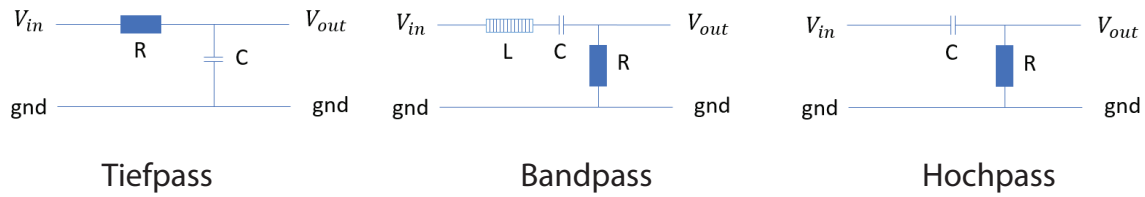
Nach Gl. (??) wurde durch das Filtern auch die Varianz des Signals verringert, was wir in Abb. ?? (links) anhand der Breite des roten gegenüber des blauen Signals qualitativ abschätzen können.

In der gemittelten Messung entspricht jeder Punkt dem Mittelwert aus mehreren ursprünglichen Messwerten. Dadurch werden die statistischen Fehler an jedem Punkt zu einem Fehler des Mittelwerts, und die Standardabweichung reduziert sich um  $1/\sqrt{N}$ . Das bedeutet auch, dass benachbarte Punkte (innerhalb der Filterzeit) miteinander korreliert werden, also nicht mehr unabhängige Information enthalten. Dieser Verlust an (hochfrequenten) Informationen ist genau der Effekt, den wir durch das Filtern erzielen wollen. Eine Reduktion des Rauschens geht also automatisch einher mit Informationsverlust.

### 4.3.3 Filterarten

Wie in der letzten Sektion gezeigt, kann man Filter einsetzen, um verschiedene Frequenzanteile eines Signals herauszufiltern. Im Wesentlichen unterscheidet man drei Filterarten:

- Tiefpass (low-pass): Hohe Frequenzanteile werden abgeschwächt, niedrige Frequenzanteile werden transmittiert.
- Bandpass (band-pass): Frequenzanteile ausserhalb eines bestimmten Frequenzbereiches werden abgeschwächt, der Band-Frequenzbereich wird transmittiert.
- Hochpass (high-pass): Niedrige Frequenzanteile werden abgeschwächt, hohe Frequenzanteile werden transmittiert.



**Abbildung 4.5:** Analoge Implementierung von verschiedenen Filtern. **Links:** Tiefpass aus paralleler Kapazität **Mitte:** Bandpass aus serieller Induktivität und Kapazität **Rechts:** Hochpass aus serieller Kapazität

Derartige Filter lassen sich digital als Algorithmen oder analog durch elektronische Bauteile implementieren. Die einfachsten analogen, elektronischen Varianten sind, siehe Abb. ??:

- Low-pass: Widerstand und Kondensator
- Band-pass: Spule, Kondensator und Widerstand (ein RLC Resonator)
- High-pass: Kondensator und Widerstand

Wir betrachten als Beispiel den typischen RC-Tiefpassfilter, der aus einem Widerstand  $R$  in der Signalleitung und einer Kapazität  $C$  zur Erdung besteht (siehe Zeichnung in Vorlesungs-Slides). Die Impedanzen (“komplexen Widerstände”) der Elemente  $R$  und  $C$  sind in Abhängigkeit der Signalfrequenz  $\omega$  gegeben als:

$$\begin{aligned} Z_R &= R \\ Z_C &= \frac{1}{i\omega C}. \end{aligned} \quad (4.3.5)$$

Das Verhältnis  $T_{filter}$  der Ein- und Ausgangsspannungen  $V_{in} = I_{in}(Z_R + Z_C)$  bzw.  $V_{out} = I_{in}Z_C$ , lässt sich wie folgt bestimmen:

$$\begin{aligned} T_{filter} &= \frac{|V_{out}|^2}{|V_{in}|^2} \\ &= \frac{|Z_C|^2}{|R + Z_C|^2} \\ &= \frac{1/(\omega C)^2}{R^2 + 1/(\omega C)^2} \\ &= \frac{1}{1 + (\omega RC)^2} \\ &= \frac{1}{1 + \omega^2/\omega_{3dB}^2}. \end{aligned} \quad (4.3.6)$$

Hierbei ist  $\omega_{3dB} = 1/RC$  die Frequenz, bei der  $|V_{out}|^2$  den halben Wert von  $|V_{in}|^2$  erreicht.

**Verständnisfragen:**

- 1 Was besagt Fouriers Theorem?
- 2 Wann würden wir typischerweise die kontinuierliche/diskrete Fouriertransformation benutzen?
- 3 Was ist die maximale Frequenz, die ohne Aliasing gesampelt werden kann?
- 4 Was besagt das Parseval-Theorem und warum ist es wichtig?
- 5 Wie berechnet man die Standardabweichung  $\sigma_x$  eines rauschbehafteten Signals  $x(t)$ ?
- 6 Welche Frequenzanteile werden durchgelassen wenn erst ein Hochpass, und dann ein Tiefpassfilter angewendet wird?

**Antworten:**

- 1 Fouriers Theorem besagt das jede integrierbare Funktion als Summe von trigonometrischen Funktionen dargestellt werden kann.
- 2 Beim Herleiten von analytischen Zusammenhängen verwendet man oft die kontinuierliche Fouriertransformation, und beim analysieren von experimentellen Daten verwendet man im Regelfall die diskrete Fouriertransformation, da eine gespeicherte Messung aus diskreten Datenpunkten besteht.
- 3 Die Nyquist-Frequenz,  $f_{max} = 1/2\delta t$
- 4 Das Parseval-Theorem besagt, dass das Integral über die PSD der Varianz entspricht. Dies ist wichtig zur Normierung der PSD.
- 5 Über das Parseval-Theorem lässt sich die Varianz  $\sigma_x^2$  aus der PSD berechnen. Die Standardabweichung ist dann  $\sqrt{\sigma_x^2}$ .
- 6 Wenn sich die Frequenzbereiche des Hoch- und Tiefpasses überschneiden, werden diejenigen Frequenzen durchgelassen, die in dem Überschneidungsbereich liegen, ähnlich wie bei einem Bandpass. Wenn sich die Frequenzen der beiden Filter nicht überschneiden, wird kein Signal durchgelassen.

# Kapitel 5

## Abschätzen von Parametern

### 5.1 Bayessche Datenanalyse

Die Bayessche Analyse präsentiert ein vollkommen anderes Verständnis von Wahrscheinlichkeiten als der frequentistische Ansatz, der in Kapitel ?? behandelt wurde. In diesem Ansatz wird die Unsicherheit als eine subjektive Unsicherheit anstelle einer objektiven Unsicherheit des Experiments interpretiert. Während der frequentistische Ansatz eine hohe Anzahl an Wiederholungen eines Experimentes erfordert, um Wahrscheinlichkeiten zu berechnen und aufgrund derer Aussagen über die Zukunft zu treffen, bilden im Bayesschen Ansatz subjektive Erfahrungen die Grundlage dafür. Im Fokus der Bayesschen Analyse stehen daher bedingte Wahrscheinlichkeiten und der Satz von Bayes (siehe Gleichung ??) nimmt eine zentrale Stellung ein. Im Folgenden definieren wir

|                                                                                      |                                 |
|--------------------------------------------------------------------------------------|---------------------------------|
| Die gemessenen Daten des Experiments                                                 | $A \rightarrow D$               |
| Ein endlicher Satz von Ergebnissen, die wir mithilfe der Daten unterscheiden möchten | $B \rightarrow B_1, \dots, B_n$ |
| Unser gesamtes zusätzliches Wissen (wird oft vernachlässigt)                         | I                               |

und schreiben den Satz von Bayes damit als

$$P(B_i|D, I) = \frac{P(D|B_i, I)P(B_i|I)}{P(D|I)} \quad (5.1.1)$$

oder vereinfacht zu

$$P(B_i|D) = \frac{P(D|B_i)P(B_i)}{P(D)}. \quad (5.1.2)$$

Wenn  $D$  impliziert, dass eines der  $B_i$  eintritt, gilt weiterhin

$$P(D) = \sum_k P(D|B_k)P(B_k). \quad (5.1.3)$$

In Gleichung ?? ist  $P(B_i)$  die *a-priori* Wahrscheinlichkeit für das Eintreten eines Ergebnisses  $B_i$  die auf dem ursprünglichen Wissen *vor* der Durchführung eines Experiments basiert. Es werden nun eine Reihe von Daten  $D$  gemessen und die konditionellen Wahrscheinlichkeiten  $P(D|B_i)$  für deren Auftreten in Abhängigkeit des Vorliegens eines bestimmten Ergebnisses  $B_i$  berechnet. Die Kenntnis der  $P(B_i)$  und  $P(D|B_i)$  ermöglicht es nun, sogenannte *a-posteriori* Wahrscheinlichkeiten  $P(B_i|D)$  zu bestimmen, mit denen ein Ergebnis  $B_i$  basierend auf der Grundlage des Datensatzes  $D$  auftritt. Es wird also eine subjektive Erfahrung, die Kenntnis des Datensatzes  $D$ , zur Berechnung der Wahrscheinlichkeiten verwendet.

**Beispiel 1:** Die Infektionskrankheit

Nehmen wir an, es gibt eine Infektionskrankheit mit einer Inzidenzrate von 100. Das bedeutet, dass 100 von 100000 Menschen, also 0.1 %, infiziert sind. Die *a-priori* Wahrscheinlichkeiten dafür, ob eine Person infiziert ( $B_I$ ) oder gesund ( $B_G$ ) ist, sind somit

$$P(B_I) = 0.001 \quad (5.1.4)$$

$$P(B_G) = 0.999 \quad (5.1.5)$$

Eine Person lässt sich nun testen und der Test ist positiv. Wie hoch ist die Wahrscheinlichkeit, dass die Person tatsächlich infiziert ist, wenn der Test in 99% der Fälle das korrekte Ergebnis liefert?

Um diese Frage zu beantworten, berechnen wir zuerst die bedingten Wahrscheinlichkeiten

$$P(D|B_I) = 0.99 \quad (5.1.6)$$

$$P(D|B_G) = 0.01 \quad (5.1.7)$$

und verwenden dann den Satz von Bayes. Dieser liefert

$$P(B_I|D) = \frac{P(D|B_I)P(B_I)}{P(D|B_I)P(B_I) + P(D|B_G)P(B_G)} = 0.1, \quad (5.1.8)$$

die Wahrscheinlichkeit, dass die Person tatsächlich infiziert ist, liegt somit „nur“ bei 10%.

Wenn sich die Person nun ein weiteres Mal testen lässt und der Test erneut positiv ist, verwenden wir nun die aus dem ersten Testergebnis resultierenden Wahrscheinlichkeiten  $P(B_I) = 0.1$  und  $P(B_G) = 0.9$  als neue *a-priori* Wahrscheinlichkeiten und verwenden auf dieser Grundlage erneut den Satz von Bayes. Nach der zweiten positiven Testrunde ergibt diese Berechnung, dass die Person zu 92% infiziert ist.



**Beispiel 2:** Das Ziegenproblem

Hinter lediglich einem von drei Toren befindet sich eine Ziege. Ziel eines Spiels ist es nun, zu erraten, hinter welchem der Tore sich die Ziege befindet. Der Kandidat des Spiels wird dazu aufgefordert, sich für ein Tor  $i = 1, 2$  oder  $3$  zu entscheiden. Anschliessend öffnet der Moderator des Spiels eins der beiden Tore, für das sich der Spieler nicht entschieden hat und hinter dem sich die Ziege *nicht* befindet. Der Kandidat erhält nun die Möglichkeit, entweder auf seiner ursprünglichen Wahl zu bestehen oder seine Meinung nochmals zu ändern. Mit welcher Entscheidung gewinnt er das Spiel mit höherer Wahrscheinlichkeit?

Diese Frage lässt sich mithilfe des Satz von Bayes beantworten. Die Wahrscheinlichkeiten  $P(B_i)$ , das sich die Ziege hinter dem Tor  $i$  befindet ist für alle Tore identisch und gleich  $\frac{1}{3}$ . Nehmen wir nun ein konkretes Szenario an, indem sich der Spieler für ein Tor  $k = 1$  entscheidet und der Moderator anschliessend das Tor  $j = 3$  öffnet. Die bedingten Wahrscheinlichkeiten  $P(D|B_i)$ , dass dieses Ergebnis  $D$  eintritt, wenn sich die Ziege hinter einem Tor  $i$  befindet, sind somit

$$P(D|B_1) \cdot P(B_1) = \frac{1}{2} \cdot \frac{1}{3} = \frac{1}{6} \quad (5.1.9)$$

$$P(D|B_2) \cdot P(B_2) = 1 \cdot \frac{1}{3} = \frac{1}{3} \quad (5.1.10)$$

$$P(D|B_3) \cdot P(B_3) = 0 \cdot \frac{1}{3} = 0 \quad (5.1.11)$$

Mithilfe des Satzes von Bayes können wir nun die Wahrscheinlichkeiten  $P(B_1|D)$  und  $P(B_2|D)$  berechnen:

$$P(B_1|D) = \frac{P(D|B_1)P(B_1)}{P(D|B_1)P(B_1) + P(D|B_2)P(B_2) + P(D|B_3)P(B_3)} = \frac{1}{3} \quad (5.1.12)$$

$$P(B_2|D) = \frac{P(D|B_2)P(B_2)}{P(D|B_1)P(B_1) + P(D|B_2)P(B_2) + P(D|B_3)P(B_3)} = \frac{2}{3} \quad (5.1.13)$$

Der Spieler erhöht (verdoppelt) somit seine Gewinnchancen, wenn er seine Wahl nochmals ändert.

## 5.2 Likelihood Funktion und Maximum Likelihood

Zur Likelihood Funktion und Maximum Likelihood ist ein Jupyter-Notebook verfügbar. Siehe Likelihood Function.ipynb

### 5.2.1 Likelihood-Funktion

Wir haben gesehen, dass eine einzelne Messung äquivalent ist zu einer Ziehung einer Zufallszahl aus einer Wahrscheinlichkeitsverteilung. Die Wahrscheinlichkeitsverteilung einer Zufallsvariable  $\zeta$

sei:

$$f(\zeta, a_0, a_1, \dots, a_m) = f(\zeta, \mathbf{a}). \quad (5.2.1)$$

Hierbei ist  $\mathbf{a}$  ein Vektor der die Wahrscheinlichkeitsverteilung parametrisiert, wie z. B. in Gl. (??) und (??). Für mehrere Variablen schreiben wir:

$$f(\zeta_0, \zeta_1, \dots, \zeta_n, a_0, a_1, \dots, a_m) = f(\boldsymbol{\zeta}, \mathbf{a}). \quad (5.2.2)$$

In einem Experiment sind die Werte der  $a_i$  unbekannt, aber wir kennen (aus der Messung) einige  $\zeta_i$ , zum Beispiel messen wir die Werte  $\zeta_i = x_i$ . Also können wir schreiben:

$$f(x_0, x_1, \dots, x_n, \mathbf{a}) = f(\mathbf{x}, \mathbf{a}) \stackrel{\text{def}}{=} L(\mathbf{x}, \mathbf{a}). \quad (5.2.3)$$

Hierbei ist  $L$  die Likelihood-Funktion. In der Likelihood-Funktion sind die  $x_i$  bekannt und die  $a_i$  sind die Variablen.

*Beispiel:* Wir messen zwei unabhängige Datenpunkte  $x_1$  und  $x_2$  aus einer Gaussverteilung:

$$f(\zeta, a_0, a_1) = \frac{1}{\sqrt{2\pi a_1}} \exp\left(-\frac{1}{2} \frac{(\zeta - a_0)^2}{a_1}\right). \quad (5.2.4)$$

Die Parameter  $a_0$  (der Erwartungswert) und  $a_1$  (die Varianz) sind unbekannt. Da die Messungen unabhängig sind, ist die gemeinsame Wahrscheinlichkeitsverteilung das Produkt der Einzelwahrscheinlichkeiten:

$$f(x_1, x_2, a_0, a_1) = f(x_1, a_0, a_1) f(x_2, a_0, a_1) = \frac{1}{2\pi a_1} \exp\left(-\frac{1}{2} \frac{\sum_{i=1}^2 (x_i - a_0)^2}{a_1}\right). \quad (5.2.5)$$

$$L(x_1, x_2, a_0, a_1) = f(x_1, x_2, a_0, a_1) \quad (5.2.6)$$

In diesem Beispiel sieht die Likelihood-Funktion der Wahrscheinlichkeitsverteilung sehr ähnlich. Aber Vorsicht:

Die Likelihood-Funktion ist **keine** Wahrscheinlichkeitsverteilung. Die Likelihood-Funktion ist eine Plausibilitätsfunktion.

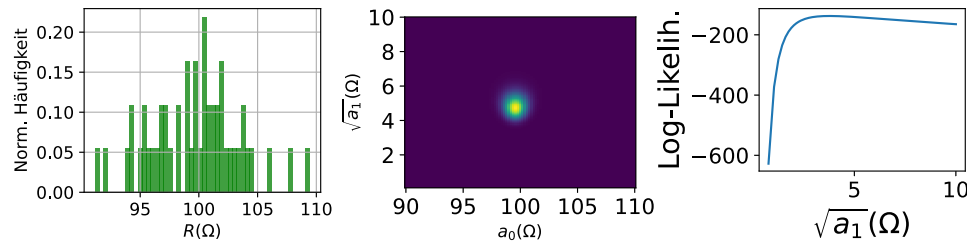
Insbesondere ist sie nicht normiert.

*Beispiel:* Wir haben einen Satz von Widerständen mit einer unbekannten Toleranz, wie in Abb. ??, links als Histogramm gezeigt. Wir nehmen an, dass die Widerstandswerte normalverteilt sind. Für  $n$  Datenpunkte  $\mathbf{x} = (x_1, x_2, \dots, x_n)$  gilt dann:

$$L(\mathbf{x}, a_0, a_1) = \prod_{i=1}^n f(x_i, a_0, a_1) = \left(\frac{1}{2\pi a_1}\right)^{\frac{n}{2}} \exp\left(-\frac{1}{2} \frac{\sum_{i=1}^n (x_i - a_0)^2}{a_1}\right). \quad (5.2.7)$$

Wir berechnen die Likelihood-Funktion für “alle” Werte von  $a_0$  und  $a_1$ . Das Ergebnis ist in Abb. ??, Mitte zu sehen. Wir finden die maximale Likelihood für eine Verteilung bei  $\hat{a}_0 = 100 \Omega$  und  $\sqrt{\hat{a}_1} = 5 \Omega$ .

In diesem Fall, weil die Daten normalverteilt sind, hätten wir auch die Ausdrücke für die Abschätzung von Erwartungswert und Standardabweichung benutzen können. Aber die Analyse der Likelihood-Funktion funktioniert auch bei beliebigen anderen Verteilungen.



**Abbildung 5.1:** Beispiel zur Likelihood Funktion **Links:** Histogramm von "gemessenen" Widerstandswerten. **Mitte:** Likelihood Funktion für verschiedene Parameter  $a_{0,1}$  der zugrunde liegenden Normalverteilung. **Rechts:** Log-Likelihood Funktion der gleichen Daten.

### 5.2.2 Maximum Likelihood

Der genaue Wert der Likelihood-Funktion ist nicht besonders aussagekräftig. Weil sie nicht normiert sein muss, gibt sie uns keine Wahrscheinlichkeit bzw. Wahrscheinlichkeitsdichte. Aber die Likelihood-Funktion beinhaltet Information über die Parameter  $\mathbf{a}$  der Wahrscheinlichkeitsverteilung. Sie gibt uns eine Plausibilität für die Werte der Parameter  $\mathbf{a}$ . Daher sind Werte von  $\mathbf{a}$ , die eine grosse Likelihood haben, zu bevorzugen.

Wir können Likelihoods über ihr Verhältnis vergleichen:

$$LR = \frac{L(\mathbf{x}, \mathbf{a})}{L(\mathbf{x}, \mathbf{b})}. \quad (5.2.8)$$

Dies ist das Verhältnis der "Plausibilitäten", dass die Parameter  $\mathbf{a}$  und  $\mathbf{b}$  die gemessenen Daten erklären. Insbesondere wollen wir die Werte für  $\mathbf{a}$  finden, die die Likelihood-Funktion maximieren.

Dies ist das sogenannte **Maximum Likelihood-Prinzip**.

Wir beginnen wieder mit gaussverteilten Daten  $\mathbf{x} = (x_1, x_2, \dots, x_n)$ . Die zugrundeliegende Likelihood-Funktion ist:

$$f(\zeta, a_0, a_1) = \frac{1}{\sqrt{2\pi a_1}} \exp\left(-\frac{1}{2} \frac{(\zeta - a_0)^2}{a_1}\right). \quad (5.2.9)$$

Die Parameter  $a_0$  (der Erwartungswert) und  $a_1$  (die Varianz) sind wieder unbekannt und wir wollen diese Werte abschätzen. Die kombinierte Likelihood-Funktion ist dann das Produkt der einzelnen Likelihood-Funktionen:

$$L(\mathbf{x}, a_0, a_1) = \left(\frac{1}{2\pi a_1}\right)^{\frac{n}{2}} \exp\left(-\frac{1}{2} \frac{\sum_{i=1}^n (x_i - a_0)^2}{a_1}\right). \quad (5.2.10)$$

Wir sind ausschliesslich an den Werten von  $a_0, a_1$  interessiert die diese Funktionen maximieren, aber nicht am Wert des Maximums. Wir können also den Logarithmus nehmen (Damit lässt sich leichter rechnen.):

$$l(\mathbf{x}, a_0, a_1) = -\frac{n}{2} \ln(2\pi a_1) - \frac{1}{2} \frac{\sum_{i=1}^n (x_i - a_0)^2}{a_1}. \quad (5.2.11)$$

Dies ist die sogenannte **Log-Likelihood-Funktion**.

In Abb. ??, rechts, ist die Log-Likelihood Funktion für das Beispiel im vorigen Abschnitt zu sehen.

Wir wollen also die Log-Likelihood-Funktion (??) maximieren. Also berechnen wir die Ableitung von  $l$  nach  $a_0$  und  $a_1$  an den Stellen  $\hat{a}_0$  und  $\hat{a}_1$  und setzen diese gleich Null, sodass  $\hat{a}_0$  und  $\hat{a}_1$  dann die Werte sind, die  $l$  maximieren:

$$\begin{aligned}\frac{\partial l}{\partial a_0} &= \frac{\partial}{\partial a_0} \Big|_{\hat{a}_0, \hat{a}_1} - \frac{n}{2} \ln(2\pi a_1) - \frac{1}{2} \frac{\sum_{i=1}^n (x_i - a_0)^2}{a_1} = 0, \\ \rightarrow 0 &= \frac{1}{\hat{a}_1} \sum_{i=1}^n (x_i - \hat{a}_0), \\ &= \frac{1}{\hat{a}_1} \left( \sum_{i=1}^n x_i - n\hat{a}_0 \right)\end{aligned}\tag{5.2.12}$$

Der Erwartungswert:  $\rightarrow \hat{a}_0 = \frac{1}{n} \sum_{i=1}^n x_i$

und:

$$\begin{aligned}\frac{\partial l}{\partial a_1} &= \frac{\partial}{\partial a_1} \Big|_{\hat{a}_0, \hat{a}_1} - \frac{n}{2} \ln(2\pi a_1) - \frac{1}{2} \frac{\sum_{i=1}^n (x_i - a_0)^2}{a_1} = 0, \\ \rightarrow 0 &= -\frac{n}{2} \frac{1}{\hat{a}_1} + \frac{1}{2\hat{a}_1^2} \sum_{i=1}^n (x_i - \hat{a}_0)^2,\end{aligned}\tag{5.2.13}$$

Die Varianz:  $\rightarrow \hat{a}_1 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{a}_0)^2.$

Bemerkung: Der Wert der Varianz hat einen Bias im Vergleich zu dem Wert für die empirische Varianz.

### 5.2.3 Der gewichtete Mittelwert

Wir betrachten nun ein häufiges Szenario. Wir messen mit einer uns bekannten und charakterisierten Messapparatur. Das heisst die Messunsicherheit (die Standardabweichung)  $\sigma_i$  jeder Messung ist bekannt. Wir messen den Datensatz  $\mathbf{x} = (x_1, x_2, \dots, x_n)$  und wollen den Mittelwert bestimmen.

Die zugrundeliegende Likelihood-Funktion jeder Messung ist:

$$f_i(\zeta_i, \sigma_i, a) = \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left(-\frac{1}{2} \frac{(\zeta_i - a)^2}{\sigma_i^2}\right).\tag{5.2.14}$$

Damit ist die Likelihood-Funktion:

$$L(\mathbf{x}, \boldsymbol{\sigma}, a) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma_i} \exp\left(-\frac{1}{2} \frac{(x_i - a)^2}{\sigma_i^2}\right)\tag{5.2.15}$$

und die Log-Likelihood-Funktion:

$$l(\mathbf{x}, \boldsymbol{\sigma}, a) = \sum_{i=1}^n \ln\left(\frac{1}{\sqrt{2\pi}\sigma_i}\right) - \frac{1}{2} \sum_{i=1}^n \frac{(x_i - a)^2}{\sigma_i^2}.\tag{5.2.16}$$

Wir bestimmen das Maximum:

$$\begin{aligned}
 \left. \frac{\partial l}{\partial a} \right|_{\hat{a}} &= \left. \frac{\partial}{\partial a} \right|_{\hat{a}} \left( -\frac{1}{2} \sum_{i=1}^n \frac{(x_i - a)^2}{\sigma_i^2} \right) = 0, \\
 \rightarrow \sum_{i=1}^n \frac{x_i - \hat{a}}{\sigma_i^2} &= \sum_{i=1}^n \frac{x_i}{\sigma_i^2} - \hat{a} \sum_{i=1}^n \frac{1}{\sigma_i^2}, \\
 \rightarrow \hat{a} &= \frac{\sum_{i=1}^n \frac{x_i}{\sigma_i^2}}{\sum_{i=1}^n \frac{1}{\sigma_i^2}} = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}.
 \end{aligned} \tag{5.2.17}$$

Hier definieren wir das Gewicht der einzelnen Messwerte als  $w_i = \frac{1}{\sigma_i^2}$ . Letzteres ist der **gewichtete Mittelwert**. Das heisst die einzelnen Werte  $x_i$  tragen zum Wert des Mittelwertes mehr bei, wenn sie eine kleinere Unsicherheit haben. Werte mit grosser Unsicherheit haben dementsprechend einen kleineren Beitrag zum Mittelwert.



# Kapitel 6

## Fitten von Parametern

### 6.1 Lineares Fitten von Polynomen

Zum linearen Fitten ist ein Jupyter-Notebook verfügbar. Siehe `linear_fitting.ipynb`

#### 6.1.1 Die Methode der kleinsten Quadrate

Bisher haben wir die Likelihood-Funktion maximiert, um Parameter zu bestimmen, die die Wahrscheinlichkeitsverteilung (PDF) eines Datensatzes beschreiben. Insbesondere die Log-Likelihood-Funktion ist einfach zu maximieren wenn die PDF eine Gaussverteilung ist:

$$\left. \frac{\partial l}{\partial a} \right|_{\hat{a}} = \left. \frac{\partial}{\partial a} \right|_{\hat{a}} \left( -\frac{1}{2} \sum_{i=1}^n \frac{(x_i - a)^2}{\sigma_i^2} \right) = 0. \quad (6.1.1)$$

In diesem Fall ist die Maximierung der Log-Likelihood-Funktion äquivalent zur Minimierung der Summe der Abstandsquadrate der einzelnen Messpunkte zum Erwartungswert (also dem Modell). Dies ist die Methode der kleinsten Quadrate. Wir definieren die Summe der Abstandsquadrate  $S = \sum_{i=1}^n \frac{(x_i - a)^2}{\sigma_i^2}$  und damit schreiben wir:

$$\left. \frac{\partial S}{\partial a} \right|_{\hat{a}} = \left. \frac{\partial}{\partial a} \right|_{\hat{a}} \sum_{i=1}^n \frac{(x_i - a)^2}{\sigma_i^2} = 0. \quad (6.1.2)$$

Bemerkung: Wenn die  $\sigma_i$  bekannt sind, dann ist dies äquivalent zur Minimierung von  $\chi^2$  und damit ist  $S = \chi^2$ . Das ist jedoch oft nicht der Fall, wir bleiben also zunächst bei  $S$ .

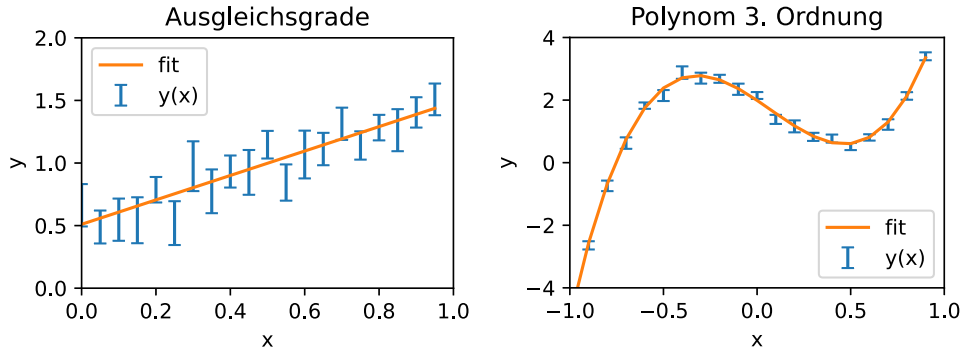
#### 6.1.2 Lineares Fitten von Polynomen

##### 6.1.2.1 Fitten einer Ausgleichsgeraden

Wir haben die Werte  $y_1, y_2, \dots, y_n$  als Funktion der Kontrollparameter  $x_1, x_2, \dots, x_n$  gemessen. Die  $x_i$  haben keine Unsicherheit, während die Unsicherheiten der  $y_i$  durch  $\sigma_i$  gegeben sind. Das bedeutet, dass die Wahrscheinlichkeit, dass der Wert  $y$  gemessen wird gegeben ist durch:

$$f(y) \sim \exp \left( -\frac{(y - \mu)^2}{2\sigma^2} \right), \quad (6.1.3)$$

wobei beide Werte  $y$  und  $\sigma$  von  $x$  abhängen. Wir nehmen an, dass  $\mu = a_0 + a_1 x$ , also dass die Erwartungswerte von  $y$  linear von  $x$  abhängen, wie in Abb. ??, links zu sehen ist. Wir wollen die



**Abbildung 6.1:** Beispiel zum linearen Fitten. **Links:** Linearer Fit einer Ausgleichsgerade zu fehlerbehafteten Daten. **Rechts:** Linearer Fit eines Polynoms dritten Grades an fehlerbehafteten Daten.

Werte von  $a_0$  und  $a_1$  finden, die die Daten am besten wiedergeben. Das heisst:

$$f(y) \sim \exp \left( -\frac{(y - a_0 - a_1 x_i)^2}{2\sigma_i^2} \right) \quad (6.1.4)$$

ist die Wahrscheinlichkeitsverteilung der  $y_i$ . Dann ist die Likelihood-Funktion für den  $i$ -ten Datenpunkt:

$$L(x_i, y_i, \sigma_i, a_0, a_1) \sim \exp \left( -\frac{(y_i - a_0 - a_1 x_i)^2}{2\sigma_i^2} \right) \quad (6.1.5)$$

und die kombinierte Log-Likelihood-Funktion für alle Datenpunkte:

$$l(\mathbf{x}, \mathbf{y}, \boldsymbol{\sigma}, a_0, a_1) = -\frac{1}{2} \sum_{i=1}^n \frac{(y_i - a_0 - a_1 x_i)^2}{\sigma_i^2} + C \quad (6.1.6)$$

mit einer Konstanten  $C$ . Wir benutzen wieder die Gewichte  $w_i = \frac{1}{\sigma_i^2}$  und damit ist:

$$l(\mathbf{x}, \mathbf{y}, \mathbf{w}, a_0, a_1) = -\frac{1}{2} \sum_{i=1}^n w_i (y_i - a_0 - a_1 x_i)^2 + C. \quad (6.1.7)$$

Wir können die besten Abschätzungen für  $a_0$  und  $a_1$  finden, indem wir die partiellen Ableitungen von  $l$  nach  $a_0$  und  $a_1$  gleich Null setzen. Da die Unsicherheiten einer Normalverteilung zugrunde liegen, können wir auch die gewichtete Summe der Abstandsquadrate (der Residuen) minimieren:

$$S = \sum_{i=1}^n w_i (y_i - a_0 - a_1 x_i)^2. \quad (6.1.8)$$

Die Residuen sind die Abstände der gemessenen Datenpunkte vom Modell.

Wir haben also zwei Gleichungen:

$$\begin{aligned} \frac{\partial S}{\partial a_0} &= -2 \sum_{i=1}^n w_i (y_i - \hat{a}_0 - \hat{a}_1 x_i) = 0, \\ \frac{\partial S}{\partial a_1} &= -2 \sum_{i=1}^n w_i x_i (y_i - \hat{a}_0 - \hat{a}_1 x_i) = 0, \end{aligned} \quad (6.1.9)$$



mit den Lösungen:

$$\begin{aligned}\hat{a}_0 &= \frac{\sum w_i x_i^2 \sum w_i y_i - \sum w_i x_i \sum w_i x_i y_i}{\sum w_i \sum w_i x_i^2 - (\sum w_i x_i)^2}, \\ \hat{a}_1 &= \frac{\sum w_i \sum w_i x_i y_i - \sum w_i x_i \sum w_i y_i}{\sum w_i \sum w_i x_i^2 - (\sum w_i x_i)^2}.\end{aligned}\quad (6.1.10)$$

Wir können das auch in Matrixform schreiben (die sogenannte Normalform):

$$\begin{pmatrix} \sum w_i & \sum w_i x_i \\ \sum w_i x_i & \sum w_i x_i^2 \end{pmatrix} \begin{pmatrix} \hat{a}_0 \\ \hat{a}_1 \end{pmatrix} = \begin{pmatrix} \sum w_i y_i \\ \sum w_i x_i y_i \end{pmatrix}, \quad (6.1.11)$$

$$\mathbf{N}\hat{\mathbf{a}} = \mathbf{Y}, \quad (6.1.12)$$

mit der Lösung:

$$\hat{\mathbf{a}} = \mathbf{N}^{-1}\mathbf{Y}. \quad (6.1.13)$$

In Abb. ??, links, ist ein fehlerbehafteter Datensatz  $y(x)$  zu sehen, der einer linearen Funktion folgt. Minimieren der Abstandsquadrate ergibt eine Ausgleichsgerade, die die Datenpunkte beschreibt.

### 6.1.2.2 Fitten eines Polynoms $m$ -ter Ordnung

Dies kann nun sehr einfach auf Polynome beliebiger Ordnung erweitert werden:  $y = a_0 + a_1 x + a_2 x^2 + \dots + a_m x^m$  und hat dann folgende explizite Form:

$$\begin{pmatrix} \sum w_i & \sum w_i x_i & \cdots & \sum w_i x_i^m \\ \sum w_i x_i & \sum w_i x_i^2 & \cdots & \sum w_i x_i^{m+1} \\ \vdots & \vdots & \ddots & \vdots \\ \sum w_i x_i^m & \sum w_i x_i^{m+1} & \cdots & \sum w_i x_i^{2m} \end{pmatrix} \begin{pmatrix} \hat{a}_0 \\ \hat{a}_1 \\ \vdots \\ \hat{a}_m \end{pmatrix} = \begin{pmatrix} \sum w_i y_i \\ \sum w_i x_i y_i \\ \vdots \\ \sum w_i x_i^m y_i \end{pmatrix}, \quad (6.1.14)$$

$$\mathbf{N}\hat{\mathbf{a}} = \mathbf{Y}, \quad (6.1.15)$$

ebenfalls mit der Lösung:

$$\hat{\mathbf{a}} = \mathbf{N}^{-1}\mathbf{Y}, \quad (6.1.16)$$

was für  $m = 3$  in Abb. ??, rechts dargestellt ist.

Diese Methode wird Lineares Fitten genannt, weil das Gleichungssystem in Gl. (??) ein lineares Gleichungssystem ist, nicht nur wenn man eine Ausgleichsgerade fittet.

### 6.1.2.3 Unsicherheiten der Fitparameter

Auf diese Weise können wir analytisch die beste Abschätzung der Modellparameter  $\mathbf{a}$  bestimmen. Aber wir wollen auch wissen wie zuverlässig diese Werte sind; wir möchten **Varianz** und **Kovarianz** wissen. Das Polynom berechnet mit den  $\hat{a}$ , wird nicht durch alle Datenpunkte gehen. Also können wir schreiben:

$$y_i = \hat{a}_0 + \hat{a}_1 x_i + \dots + \hat{a}_m x_i^m + \epsilon_i, \quad (6.1.17)$$

wobei  $\epsilon_i$  der Abstand ist, den der  $i$ -te Punkt von der gefitteten Funktion hat: **Die Residuen**. Wir nehmen an, dass die Unsicherheiten der  $y_i$  durch zufällige Fehler zustande kommen und normalverteilt sind. Dann sind die  $\epsilon_i$  unkorreliert und ihr Erwartungswert ist Null:

$$\langle \epsilon_i \rangle = 0 \quad \langle \epsilon_i, \epsilon_j \rangle = \sigma_i^2 \delta_{ij}. \quad (6.1.18)$$

Wir definieren die Vektoren:

$$\mathbf{y} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} \quad \boldsymbol{\epsilon} = \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix} \quad \mathbf{a} = \begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix} \quad (6.1.19)$$

und die Matrizen:

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & \cdots & x_1^m \\ 1 & x_2 & \cdots & x_2^m \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^m \end{pmatrix} \quad \mathbf{W} = \begin{pmatrix} w_1 & \cdots & \cdots & 0 \\ \vdots & w_2 & \cdots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & \cdots & w_n \end{pmatrix}. \quad (6.1.20)$$

Damit wird ?? zu:

$$\mathbf{y} = \mathbf{X}\mathbf{a} + \boldsymbol{\epsilon}. \quad (6.1.21)$$

Wir können wieder die Normalmatrix und den gewichteten Ergebnisvektor finden:

$$\mathbf{N} = \mathbf{X}^T \mathbf{W} \mathbf{X} \quad \text{und} \quad \mathbf{Y} = \mathbf{X}^T \mathbf{W} \mathbf{y}. \quad (6.1.22)$$

Der Ausdruck für  $S$ , die gewichtete Summe der Quadrate der Residuen ist dann:

$$S = (\mathbf{y} - \mathbf{X}\mathbf{a})^T \mathbf{W} (\mathbf{y} - \mathbf{X}\mathbf{a}). \quad (6.1.23)$$

Die Normalform hat folgende Gestalt:

$$(\mathbf{X}^T \mathbf{W} \mathbf{X}) \hat{\mathbf{a}} = (\mathbf{X}^T \mathbf{W} \mathbf{y}) \quad (6.1.24)$$

mit der Lösung:

$$\hat{\mathbf{a}} = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y}. \quad (6.1.25)$$

Die Unsicherheiten der Parameter  $\hat{\mathbf{a}}$  sind gegeben durch deren Varianz:

$$\hat{\sigma}_i^2 = \langle (\hat{a}_i - a_i)^2 \rangle \quad (6.1.26)$$

und Kovarianz:

$$\hat{\sigma}_{ij}^2 = \langle (\hat{a}_i - a_j)(\hat{a}_j - a_i) \rangle \quad (6.1.27)$$

relativ zu den (unbekannten) tatsächlichen Werten  $\mathbf{a}$ . Das können wir auch als Matrix schreiben, in der Kovarianzmatrix:

$$\mathbf{C} = \begin{pmatrix} \hat{\sigma}_i^2 & \cdots & \hat{\sigma}_{0m}^2 \\ \vdots & \ddots & \vdots \\ \hat{\sigma}_{m0}^2 & \cdots & \hat{\sigma}_m^2 \end{pmatrix} = \langle (\hat{\mathbf{a}} - \mathbf{a})(\hat{\mathbf{a}} - \mathbf{a})^T \rangle. \quad (6.1.28)$$

Wir können Ausdruck ?? umformen:

$$\hat{\mathbf{a}} = (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \mathbf{y} = \quad (6.1.29)$$

$$= \mathbf{N}^{-1} \mathbf{X}^T \mathbf{W} (\mathbf{X} \mathbf{a} + \boldsymbol{\epsilon}) = \quad (6.1.30)$$

$$= \mathbf{N}^{-1} \mathbf{X}^T \mathbf{W} \mathbf{X} \mathbf{a} + \mathbf{N}^{-1} \mathbf{X}^T \mathbf{W} \boldsymbol{\epsilon} = \quad (6.1.31)$$

$$= \mathbf{N}^{-1} \mathbf{N} \mathbf{a} + \mathbf{N}^{-1} \mathbf{X}^T \mathbf{W} \boldsymbol{\epsilon} = \quad (6.1.32)$$

$$= \mathbf{a} + \mathbf{N}^{-1} \mathbf{X}^T \mathbf{W} \boldsymbol{\epsilon} = \quad (6.1.33)$$

$$\mathbf{a} + (\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \boldsymbol{\epsilon}. \quad (6.1.34)$$

Damit finden wir:

$$\mathbf{C} = \left\langle ((\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \boldsymbol{\epsilon}) ((\mathbf{X}^T \mathbf{W} \mathbf{X})^{-1} \mathbf{X}^T \mathbf{W} \boldsymbol{\epsilon})^T \right\rangle = \quad (6.1.35)$$

$$\left\langle \mathbf{N}^{-1} \mathbf{X}^T \mathbf{W} \boldsymbol{\epsilon} \boldsymbol{\epsilon}^T \mathbf{W}^T \mathbf{X} (\mathbf{N}^{-1})^T \right\rangle \quad (6.1.36)$$

und mit  $\langle \boldsymbol{\epsilon} \boldsymbol{\epsilon}^T \rangle = \mathbf{W}^{-1}$  (der einzigen verbleibenden Zufallsvariable) wird das zu:

$$\mathbf{C} = \mathbf{N}^{-1} \mathbf{X}^T \mathbf{W} \mathbf{W}^{-1} \mathbf{W}^T \mathbf{X} (\mathbf{N}^{-1})^T. \quad (6.1.37)$$

Da sowohl  $\mathbf{N}$ , als auch  $\mathbf{W}$  symmetrisch sind gilt:  $\mathbf{N} = \mathbf{N}^T$  und  $\mathbf{W} = \mathbf{W}^T$  und wir finden für die Kovarianzmatrix:

$$\mathbf{C} = \mathbf{N}^{-1}. \quad (6.1.38)$$

Bemerkung: Tatsächlich sind oft die  $\sigma_i$  nicht gut genug bekannt. Das heisst  $\sigma_i \neq \langle \epsilon_i^2 \rangle$ . Das kann korrigiert werden und man findet nach langer Rechnung:

$$\mathbf{C} = \frac{\hat{S}_{\min}}{n - m - 1} \mathbf{N}^{-1}. \quad (6.1.39)$$

Hierbei ist  $\hat{S}_{\min} = \sum_{i=1}^n w_i (y_i - \hat{a}_0 - \hat{a}_1 x_i - \dots - \hat{a}_m x_i^m)^2$ , der minimale Wert von  $S$  ausgewertet für die Werte von  $\hat{\mathbf{a}}$ . Dies entspricht der Skalierung der Kovarianzmatrix mit dem reduzierten  $\chi_{\text{red}}^2$ .

### 6.1.3 Güte des Fits - $\chi^2$ Test

Wir sehen also, dass auch der Wert von  $\hat{S}_{\min}$  eine Bedeutung hat. Im allgemeinen gilt

$$\hat{S}_{\min} = \sum_{i=1}^n w_i (y_i - f(x_i, \hat{\mathbf{a}}))^2. \quad (6.1.40)$$

$S$  ist also eine Zufallsvariable mit einer  $\chi^2$  Verteilung mit den Freiheitsgraden  $k = n - m - 1$ . Hier ist  $n$  die Anzahl der Datenpunkte und  $m - 1$  die Anzahl der Fitparameter. Wir erwarten also, dass  $E(\hat{S}_{\min}) = E(\chi_k^2) = k$ , mit einer Standardabweichung  $\sqrt{2k}$ .

Typischerweise wird ein 95% Konfidenzintervall  $[\chi_{0.025}^2, \chi_{0.975}^2]$  um den Erwartungswert angegeben, in dem der Wert  $\hat{S}_{\min}$  mit 95% Wahrscheinlichkeit zu finden ist. Die Grenzen des Intervalls berechnen sich nach

$$0.025 = \int_0^{\chi_{0.025}^2} f_k(\chi^2) d\chi^2, \quad (6.1.41)$$

$$0.975 = \int_{\chi_{0.975}^2}^{\infty} f_k(\chi^2) d\chi^2. \quad (6.1.42)$$

$$(6.1.43)$$

Achtung: Hier bezeichnen die Indizes der  $\chi^2$  den unteren und oberen Grenzwert, nicht die Anzahl der Freiheitsgrade.

Nun haben wir ein quantitatives Mass, anhand dessen wir entscheiden können, ob unser Modell die Daten gut beschreibt (die Nullhypothese): (A) Für  $\hat{S}_{\min} < \chi_{0.025}^2$  sind die Unsicherheiten ( $\sigma_i = \frac{1}{\sqrt{w_i}}$ ) der Daten zu gross, um sie als statistische Standardabweichung zu interpretieren.

Dies kann z.B. darauf hinweisen, dass ein einfacheres Modell die Daten beschreiben kann. Dieser Fall ist als Overfitting bekannt. (B) Für  $\hat{S}_{\min} > \chi_{0.975}^2$  weichen die Daten stärker vom Modell ab, als ihre Unsicherheiten erwarten lassen. Das Modell beschreibt also die Daten nicht gut.

Wie gross das Konfidenzintervall gewählt wird, hängt von der Situation ab und sollte im Rahmen der Analyse angegeben werden.

### 6.1.4 Zusammenfassung

- Wahrscheinlichkeitsverteilungen beschreiben die Wahrscheinlichkeiten von möglichen Ergebnissen von Zufallsexperimenten.
- Wir maximieren die (Log-)Likelihood-Funktion um die wahrscheinlichsten Werte für die Parameter der Wahrscheinlichkeitsverteilung zu finden.
- Die Likelihood-Funktion ist nicht normiert und somit selbst keine Wahrscheinlichkeitsverteilung.
- Wir können Polynome  $m$ -ter Ordnung fiten, indem wir die Normalmatrix  $\mathbf{N}$  berechnen.
- Die Unsicherheit (Varianz und Kovarianz) der Fitparameter ist gegeben durch die Kovarianzmatrix  $\mathbf{C} = \mathbf{N}^{-1}$ .

## 6.2 Fitten von nichtlinearen Funktionen

Zum nichtlinearen Fitten ist ein Jupyter-Notebook verfügbar. Siehe `nonlinear_fitting.ipynb`

### 6.2.1 Nichtlineare Schätzung der kleinsten Quadrate

Um die lineare Funktion  $a_0 - a_1 x_i$  an einen gemessenen Datensatz  $y_1, y_2, \dots, y_n$ , gemessen als Funktion der unabhängigen Variable  $x_1, x_2, \dots, x_n$  zu fiten, haben wir die Summe der Abstandsquadrate (Residuen) minimiert, vgl. ???. Im Allgemeinen heisst das also für eine Funktion  $f(x, a_0, a_1, \dots, a_m) = f(x, \mathbf{a})$  wir müssen

$$S = \sum_{i=1}^n w_i (y_i - f(x_i, \mathbf{a}))^2 \quad (6.2.1)$$

minimieren. Wir können nun  $S$  als Funktion von “allen”  $\mathbf{a}$  berechnen. Das ergibt uns eine Fläche im  $m + 2$  dimensionalen Raum.

Das Ziel ist nun das Minimum von  $S$  zu finden. Da es im Allgemeinen keine geschlossene Lösung gibt, sind iterative Ansätze notwendig.

#### 6.2.1.1 Gradientenverfahren

Wir starten mit einer initialen Schätzung der Werte von  $\mathbf{a}$ :  $\mathbf{a}_g$  und bewegen uns auf der Fläche  $S$  entlang des steilsten Gradienten, den wir aus der Linearisierung (Taylor-Entwicklung erster

Ordnung) erhalten. Dieser Methode folgen wir iterativ zum Minimum von  $S$ :

$$S \approx S(\mathbf{a}_g) + \sum_{i=0}^m \left. \frac{\partial S}{\partial a_i} \right|_{\mathbf{a}_g} \Delta a_i \quad (6.2.2)$$

mit  $\Delta a_i = a_i - a_{ig}$ . In Vektorschreibweise ist das:

$$S \approx S(\mathbf{a}_g) + \Delta \mathbf{a}^T \nabla S. \quad (6.2.3)$$

Hierbei ist

$$\nabla S = \begin{pmatrix} \frac{\partial S}{\partial a_0} \\ \vdots \\ \frac{\partial S}{\partial a_m} \end{pmatrix} \quad (6.2.4)$$

der Gradient von  $S$ . Damit ist die Korrektur von  $\mathbf{a}_g$  Richtung Minimum gegeben durch:

$$\Delta a_i = -\alpha \left. \frac{\partial S}{\partial a_i} \right|_{\mathbf{a}_g}, \quad (6.2.5)$$

wobei der Parameter  $\alpha$  die ‘Grösse’ der Korrektur skaliert. Damit sind die neuen Werte für die Fitparameter:

$$a_i = a_{ig} + \Delta a_i. \quad (6.2.6)$$

Diese Prozedur wird wiederholt bis die Fitfunktion die Daten hinreichend gut beschreibt. Dies kann zum Beispiel erreicht sein, wenn der Gradient sehr klein wird. Das tatsächliche Minimum ist erreicht wenn  $\nabla S = 0$ . Die initialen Schätzwerte  $\mathbf{a}_g$  sowie der Parameter  $\alpha$  müssen möglicherweise angepasst werden um die Konvergenz des Algorithmus zu erreichen.

Da die Schritte kleiner werden desto näher wir dem Optimum kommen, müssen wir  $\alpha$  bei jedem Schritt anpassen um die Konvergenz zu beschleunigen.

### 6.2.1.2 Newtonverfahren

Anstelle der einfachen Linearisierung von  $S$  um  $\mathbf{a}_g$  (Taylor-Entwicklung erster Ordnung), können wir auch den zweiten Term der Taylor-Entwicklung berücksichtigen:

$$S \approx S(\mathbf{a}_g) + \sum_{i=0}^m \left. \frac{\partial S}{\partial a_i} \right|_{\mathbf{a}_g} \Delta a_i + \frac{1}{2} \sum_{i=0}^m \sum_{j=0}^m \left. \frac{\partial^2 S}{\partial a_i \partial a_j} \right|_{\mathbf{a}_g} \Delta a_i \Delta a_j = S'(\Delta \mathbf{a}). \quad (6.2.7)$$

Wir approximieren  $S$  also durch ein Paraboloid. Der nächste Iterationsschritt ist gegeben durch das Minimum von  $S'$ . Das bedeutet, dass für alle  $\Delta a_k$ :  $\frac{\partial S'}{\partial \Delta a_k} = 0$ :

$$\left. \frac{\partial S}{\partial a_k} \right|_{\mathbf{a}_g} + \sum_{i=0}^m \left. \frac{\partial^2 S}{\partial a_k \partial a_i} \right|_{\mathbf{a}_g} \Delta \hat{a}_i = 0. \quad (6.2.8)$$

Hierbei ist  $\Delta \hat{\mathbf{a}}$  der Vektor mit den Korrekturen für die initialen Parameter. Dies können wir in Matrixform schreiben:

$$\underbrace{\begin{pmatrix} \frac{\partial^2 S}{\partial a_0^2} & \cdots & \frac{\partial^2 S}{\partial a_0 \partial a_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 S}{\partial a_m \partial a_0} & \cdots & \frac{\partial^2 S}{\partial a_m^2} \end{pmatrix}}_{\text{Hesse-Matrix } \mathbf{H}} \begin{pmatrix} \Delta \hat{a}_0 \\ \vdots \\ \Delta \hat{a}_m \end{pmatrix} = - \underbrace{\begin{pmatrix} \frac{\partial S}{\partial a_0} \\ \vdots \\ \frac{\partial S}{\partial a_m} \end{pmatrix}}_{\text{Gradient } \nabla S} \quad (6.2.9)$$

sowie auch den Ausdruck für  $S'$ :

$$S' = S(\mathbf{a}_g) + \Delta \mathbf{a}^T \nabla S + \frac{1}{2} \Delta \mathbf{a}^T \mathbf{H} \Delta \mathbf{a}. \quad (6.2.10)$$

Das heisst also:

$$\mathbf{H} \Delta \hat{\mathbf{a}} = -\nabla S. \quad (6.2.11)$$

Damit erhalten wir für die Korrekturen zu den initialen Parametern:

$$\Delta \hat{\mathbf{a}} = -\mathbf{H}^{-1} \nabla S. \quad (6.2.12)$$

Die neuen Parameter sind dann:  $\hat{\mathbf{a}} = \mathbf{a}_g + \Delta \hat{\mathbf{a}}$ . Auch hier fügen wir für bessere Konvergenz einen Skalierungsfaktor ( $\alpha < 1$ ) ein:

$$\hat{\mathbf{a}} = \mathbf{a}_g + \alpha \Delta \hat{\mathbf{a}}. \quad (6.2.13)$$

### 6.2.1.3 Vergleich des Gradienten- und Newtonverfahren

Wir berechnen die Korrektur als:

$$\begin{aligned} \text{Gradientenverfahren:} \quad \Delta \hat{\mathbf{a}} &= -\alpha \nabla S, \\ \text{Newtonverfahren:} \quad \Delta \hat{\mathbf{a}} &= -\alpha \mathbf{H}^{-1} \nabla S. \end{aligned} \quad (6.2.14)$$

Eine intuitive und einfache Verbesserung der Methode des Gradientenverfahrens ist also  $\alpha$  dynamisch anzupassen, indem wir es mit der inversen Krümmung von  $S$  skalieren, sodass wir eine schnellere Konvergenz erreichen, wie in Abb. ??, rechts zu sehen ist:

$$\Delta \hat{a}_i = -\alpha \left( \frac{\partial^2 S}{\partial a_i^2} \right)^{-1} \frac{\partial S}{\partial a_i}. \quad (6.2.15)$$

Grundsätzlich ist es wünschenswert die Vorteile von beiden Methoden zu kombinieren und ihre Nachteile zu kompensieren.

### 6.2.1.4 Marquardts Methode

Wir wollen die Verfahren 1 und 2 kombinieren um schnelle Konvergenz in allen Regimen zu erreichen.

- Wenn wir weit weg sind vom Optimum (wenn die 2te Ableitung von  $S$  gross ist) wollen wir ein konstantes  $\alpha$  benutzen.
- Wenn wir dem Optimum näher kommen (wenn die 2te Ableitung von  $S$  klein wird) möchten wir die Korrektur mit der inversen Hesse-Matrix skalieren.

Eine Methode dies zu erreichen wurde von D. Marquardt vorgeschlagen<sup>1</sup>, indem die Matrix  $\mathbf{H}$  ersetzt wird durch:

$$\mathbf{H}'_{i,j} = \begin{cases} \mathbf{H}_{i,j}(1 + \alpha) & i = j \\ \mathbf{H}_{i,j} & i \neq j \end{cases}. \quad (6.2.16)$$

Wir benutzen  $\mathbf{H}'$  um  $\hat{\mathbf{a}}$  zu berechnen:

$$\Delta \hat{\mathbf{a}} = -(\mathbf{H}')^{-1} \nabla S. \quad (6.2.17)$$

Falls  $\alpha$  sehr gross wird, vereinfacht sich dies zu:

$$\Delta \hat{a}_i = \frac{1}{(1 + \alpha)} \left( \frac{\partial^2 S}{\partial a_i^2} \right)^{-1} \frac{\partial S}{\partial a_i}. \quad (6.2.18)$$

Das ist sehr ähnlich wie unsere einfache Verbesserung des Gradientenverfahrens.

<sup>1</sup>Donald W. Marquardt, An Algorithm for Least-Squares Estimation of Nonlinear Parameters. Journal of the Society for Industrial and Applied Mathematics 11, 431 (1963)

### 6.2.2 Unsicherheit der Fitparameter

Mit diesen Methoden erhalten wir nicht automatisch die Kovarianzmatrix aus der Normalform, wie bei der linearen Minimierung der Summe der Abstandsquadrate. Aber wir können die Kovarianzmatrix aus der Hesse-Matrix berechnen. Hierzu nehmen wir an, dass wir die Werte  $\hat{\mathbf{a}}$ , die  $S$  minimieren, mit einer der vorher beschriebenen Methoden bestimmt haben:

$$S_{\min} = \sum_{i=0}^n w_i (y_i - f(x_i, \hat{\mathbf{a}}))^2. \quad (6.2.19)$$

Wir linearisieren  $f$  um  $\hat{\mathbf{a}}$  mittels einer Taylor-Entwicklung erster Ordnung:

$$f(x, \mathbf{a}) \approx f(x, \hat{\mathbf{a}}) + \sum_{j=0}^m \left. \frac{\partial f(x, \mathbf{a})}{\partial a_j} \right|_{\hat{\mathbf{a}}} (a_j - \hat{a}_j) = f(x, \hat{\mathbf{a}}) + \sum_{j=0}^m \frac{\partial f(x, \mathbf{a})}{\partial a_j} \Delta a_j, \quad (6.2.20)$$

mit  $\Delta a_j = a_j - \hat{a}_j$  und von nun an werden alle Ableitungen an der Position  $\hat{\mathbf{a}}$  ausgewertet. Hiermit können wir  $S$  abschätzen als:

$$\begin{aligned} S &\approx \sum_{i=0}^n w_i \left( y_i - f(x_i, \hat{\mathbf{a}}) - \sum_{j=0}^m \frac{\partial f(x_i, \mathbf{a})}{\partial a_j} \Delta a_j \right)^2 \\ &= \underbrace{\sum_{i=0}^n w_i (y_i - f(x_i, \hat{\mathbf{a}}))^2}_{=S_{\min}} - 2 \underbrace{\sum_{i=0}^n \left( w_i (y_i - f(x_i, \hat{\mathbf{a}})) \sum_{j=0}^m \frac{\partial f(x_i, \mathbf{a})}{\partial a_j} \Delta a_j \right)}_{\substack{= \frac{\partial S}{\partial \Delta a_j} \Big|_{\Delta a_j=0} = 0}} \\ &\quad + \sum_{i=0}^n w_i \underbrace{\left( \sum_{j=0}^m \frac{\partial f(x_i, \mathbf{a})}{\partial a_j} \Delta a_j \right)^2}_{= \sum_{j=0}^m \frac{\partial f(x_i, \mathbf{a})}{\partial a_j} \Delta a_j \sum_{k=0}^m \frac{\partial f(x_i, \mathbf{a})}{\partial a_k} \Delta a_k} \\ &= S_{\min} + \sum_{j=0}^m \sum_{k=0}^m \Delta a_j \Delta a_k \sum_{i=0}^n w_i \frac{\partial f(x_i, \mathbf{a})}{\partial a_j} \frac{\partial f(x_i, \mathbf{a})}{\partial a_k} \\ &= S_{\min} + \Delta \mathbf{a}^T \mathbf{N} \Delta \mathbf{a}. \end{aligned} \quad (6.2.21)$$

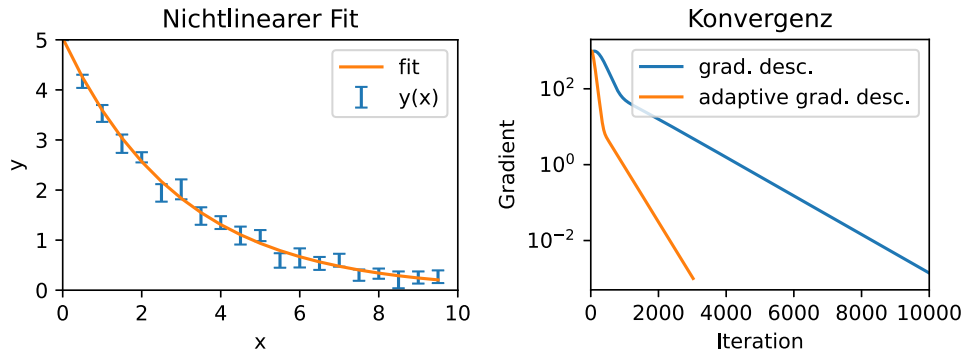
So haben wir das nichtlineare Problem auf ein lineares reduziert und können die Methoden anwenden, die wir für den linearen Fall etabliert haben. Somit können wir im Prinzip die Kovarianzmatrix  $\mathbf{C} = \mathbf{N}^{-1}$  berechnen.

Um die explizite Berechnung von  $\mathbf{N}$  zu vermeiden, können wir uns an ?? erinnern. Am Minimum von  $S$ , wo  $\mathbf{a}_g = \hat{\mathbf{a}}$  und  $\nabla S = 0$ , wird das zu:

$$S = S_{\min} + \frac{1}{2} \Delta \mathbf{a}^T \mathbf{H} \Delta \mathbf{a} \quad (6.2.22)$$

und somit  $\frac{1}{2} \Delta \mathbf{a}^T \mathbf{H} \Delta \mathbf{a} = \Delta \mathbf{a}^T \mathbf{N} \Delta \mathbf{a}$ , was bedeutet, dass  $\mathbf{N} = \frac{1}{2} \mathbf{H}$ . Wir können also die Kovarianzmatrix aus der Hesse-Matrix berechnen:

$$\mathbf{C} = 2\mathbf{H}^{-1}. \quad (6.2.23)$$



**Abbildung 6.2:** Beispiel zum nichtlinearen Fitten. **Links:** Nichtlinearer Fit einer Exponentialfunktion zu fehlerbehafteten Daten. **Rechts:** Konvergenz des Gradientenverfahrens und des Newton-Verfahrens.

### 6.2.3 Qualität des Fits

Mit diesen Methoden können wir ausgehend von den Anfangswerten für die Fit parameter ein Minimum der Kostenfunktion  $S$  bestimmen. Ob dieses Minimum jedoch das globale, oder ein lokales Minimum ist, ist nicht sicher. Wir können das Ergebnis des Fits zusammen mit den Daten plotten um beurteilen zu können, ob die Daten gut beschrieben werden und ob nach unserer Vorstellung, das globale Minimum gefunden wurde.

Darüber hinaus, bietet es sich an die Residuen zu betrachten, um zu evaluieren, wie gut der Fit die Daten beschreibt (Siehe auch Abschnitt ??). Dazu berechnen wir  $\epsilon_i = y_i - f(x_i, \hat{\mathbf{a}})$  für den gesamten Datensatz um die Modellparameter  $\hat{\mathbf{a}}$  und  $S$  zu minimieren. Idealerweise sollten die Abweichungen  $\epsilon_i$  der gemessenen Werte  $y_i$  von der Fitfunktion  $f(x_i, \hat{\mathbf{a}})$  nur durch zufällige (statistische), unkorrelierte Fehler zustande kommen. In diesem Fall beschreibt unser Modell die Daten gut und die Voraussetzungen für die Berechnung der Unsicherheiten der Fitparameter (siehe ??) sind erfüllt.

Wenn die Residuen nicht zufällig und symmetrisch um die Null verteilt sind, sondern systematische Abweichungen und Korrelationen aufweisen, dann beschreibt das Modell die Daten nur unzureichend (Wobei hier kleine Abweichungen oft nicht vermieden werden können und in Kauf genommen werden müssen). Diese Abweichung kann durch systematische Fehler in der Messung hervorgerufen werden, die durch eine Optimierung des Messaufbaus bzw. durch Anpassung des Modells reduziert oder vermieden werden können. Andererseits kann auch das physikalische Modell nicht das richtige sein, um den gemessenen Effekt zu beschreiben. Dann muss das Modell erweitert, oder ein anderes Modell gewählt werden. Es ist auf jeden Fall zu beachten, dass die Unsicherheiten der Fitparameter unzuverlässig sind wenn die Residuen zu starke Korrelationen aufweisen.

### 6.2.4 Zusammenfassung

- Wir können für jede Funktion die Kostenfunktion  $S$  (die Summe der Abstandsquadrate) definieren.
- Minimieren von  $S$  erlaubt uns die Funktionsparameter zu finden, für die die Funktion die Daten am besten beschreibt.
- Dazu folgen wir iterativ dem Gradienten von  $S$  zum Minimum.
- Die Unsicherheit der Fitparameter können wir mit Hilfe der Hesse-Matrix berechnen.
- Die meisten Datenanalysewerkzeuge stellen effiziente und zuverlässige Implementierungen dieser Methoden zu Verfügung, in Python z.B. "curve.fit" in der "scipy" Bibliothek.



**Verständnisfragen:**

- 1 Was ist die Likelihood Funktion?
- 2 Warum ist die Likelihood Funktion keine Wahrscheinlichkeitsverteilung?
- 3 Warum wollen wir die Likelihood Funktion maximieren?
- 4 Welchen Sinn hat die Log-Likelihood Funktion?
- 5 Was bedeuten die Residuen im Kontext eines Fit mit einer Ausgleichsgerade?
- 6 Was ist die Normalmatrix?
- 7 Warum funktioniert das invertieren der Normalmatrix nicht bei nichtlinearen Fitten?
- 8 Wie berechnet man die Unsicherheit der Fitparameter beim Nichtlinearen Fitten?

**Antworten:**

- 1 Die Likelihood Funktion gibt ein relatives Maß wie gut ein Modell mit Parametern  $a_i$  zu bekannten Datenpunkten  $x_i$  passt.
- 2 Die Likelihood Funktion ist keine Wahrscheinlichkeitsverteilung, weil sie nicht normiert ist.
- 3 Die Parameter  $a_i$ , die die Likelihood Funktion maximieren, beschreiben die gemessenen Daten am besten, sodass wir ein aussagekräftigeres Modell erhalten.
- 4 Da die Likelihood Funktion nicht normiert ist, sind für uns nur relative Änderungen relevant. Oft erstrecken sich die Werte der Likelihood Funktion über viele Größenordnungen, sodass der Logarithmus der Likelihood Funktion ihre numerischen Werte besser vergleichbar macht. Zusätzlich können über die Log-Likelihood Funktion verschiedene Ausdrücke wie der Erwartungswert und die Varianz hergeleitet werden.
- 5 Die Residuen beim Fit mit einer Ausgleichsgerade sind die Abstände der Datenpunkte vom Fit Modell. Die gewichtete Summe der Residuen gibt an wie weit unsere Gerade von der optimalen Ausgleichsgerade entfernt ist.
- 6 Die Normalmatrix ist die Matrixform des linearen Gleichungssystems das man erhält wenn man die Summe der Residuen minimiert. Invertieren der Normalmatrix lässt uns aus den gemessenen Datenpunkten die Parameter eines Polynoms berechnen, das diese Datenpunkte optimal beschreibt.
- 7 Beim Ableiten der Summe der Residuen einer nichtlinearen Funktion ergibt sich nicht unbedingt ein lineares Gleichungssystem welches wir als Normalmatrix schreiben könnten. Daher müssen wir die Summe der Residuen iterativ minimieren.
- 8 Die Unsicherheiten der Parameter bei einem nichtlinearen Fit können abgeschätzt werden, indem man die nichtlineare Funktion entwickelt und dann behandelt wie eine lineare Funktion.



# Kapitel 7

## Machine Learning

### 7.1 Grundzüge von Machine Learning

Das Ziel von Machine Learning (ML) ist die Reproduktion von biologischem Lernverhalten in Computern. Ein Beispiel aus der Objekt-Klassifizierung: Menschen können schnell eine Lampe von einem Buch unterscheiden, obwohl es sehr viele verschiedene Lampen und Bücher gibt. Es ist allerdings schwierig, eine formelle Definition von *Lampe* zu geben und somit Algorithmen zu entwerfen. Wir Menschen treffen solche Entscheidungen nicht mithilfe von solchen Definitionen, sondern wir benutzen unsere vorherigen Erfahrungen. Eine Lampe ist eine Lampe, weil es unsere vorherigen Erfahrungen mit Lampen widerspiegelt.

Da Computer keine Erfahrungen sammeln wie Menschen, brauchen wir eine Methode, um Computern etwas beizubringen. Wir müssen folgendes für den Computer bereitstellen:

1. **Datensatz:** z.B. Bilder von Lampen
2. **Modell:** in der Biologie ist dies das Gehirn
3. **Verlustfunktion:** ein Mass dafür wie falsch die Aussagen sind.

Wenn wir diese haben, können wir den Computer trainieren und dann validieren.

#### 7.1.1 Datensatz

Ein Datensatz ist eine Tabelle mit beliebig vielen Merkmalen und einem Klassifizierungslabel (*class label*). Die Merkmale können Antworten zu ja-nein-Fragen oder aber auch Zahlen sein. Die Merkmale für eine Lampe können zum Beispiel die folgenden sein:

- Ist es hell?
- Kann man es aus- und einschalten?
- Verwendet es Elektrizität?

Das class label ist dann: Ist es eine Lampe? Wir werden ein Modell so trainieren, dass es aus den Merkmale das class label vorhersagen kann.

#### 7.1.2 Modell

Ein Modell ist eine Menge von möglichen Algorithmen, die die Merkmale als Argument nehmen und ein class label als Antwort zurückgeben. Wir beschriften diese Menge oft mit  $\mathcal{H}$ .  $\mathcal{H}$  wird auch als Hypothesenraum bezeichnet. Beispiele sind Entscheidungsbäume oder Neuronale Netze bestimmter Struktur.

### 7.1.3 Verlustfunktion

Eine Verlustfunktion  $L(D, f)$  ist eine Funktion, die einen Datensatz  $D$  und eine  $f \in \mathcal{H}$  als Argument nimmt und die Fehlerrate der  $f$  in  $D$  quantifiziert. Für Entscheidungsbäume wäre der Anteil von falschen Klassifizierungen eine mögliche Verlustfunktion. Es gibt keine allgemeine Verlustfunktion, sondern man muss eine für das Modell  $\mathcal{H}$  auswählen.

### 7.1.4 Training

Das Training minimiert die Verlustfunktion anhand von Trainingsdaten. Wie man dies spezifisch schafft, ist abhängig von dem Modell und der Verlustfunktion. Es ist oft in Python Packages für verschiedene Modelle implementiert.

### 7.1.5 Evaluation des Modells

Um ein Modell zu evaluieren, verwenden wir einen Testdatensatz  $D'$ . Dieser darf kein Teil des Trainingsdatensatzes sein und darf nicht für das Training verwendet worden sein. Auf dem Testdatensatz  $D'$  überprüfen wir, wie gut das Modell auf Daten funktioniert, die es noch nie gesehen hat.

### 7.1.6 Validierung

Oft verwendet man noch einen Dritten Datensatz, den Validierungsdatensatz  $D''$  im Training. Der Trainingsdatensatz  $D$  wird nochmals in zwei Datensätze aufgeteilt: Den eigentlichen Trainingsdatensatz und den Validierungsdatensatz. Beim Trainieren des Modells müssen wir oft Parameter wählen, die das Modell beeinflussen. Diese Parameter werden Hyperparameter genannt. Da wir oft nicht wissen, wie sich die Hyperparameter auf das Modell auswirken, müssen wir verschiedene Hyperparameter ausprobieren. Um den Einfluss der Hyperparameter zu testen, verwenden wir den Validierungsdatensatz  $D''$ . Beachten Sie, dass wir auch für das Ausprobieren der Hyperparameter den Testdatensatz  $D'$  nicht verwenden dürfen. Ansonsten könnte es passieren, dass wir ein Modell trainieren, das auf dem Testdatensatz gut funktioniert, aber auf neuen Daten schlecht abschneidet.

## 7.2 Ausgewählte Modelle

Im folgenden werden wir uns einige Machine learning Modelle anschauen. Das Ziel dieses Abschnitts ist, Ihnen einerseits einige grundlegende Ideen des maschinellen Lernens zu vermitteln, aber Ihnen andererseits auch die Verknüpfung zu den mathematischen Methoden aufzuzeigen, die Sie in der Vorlesung kennengelernt haben.

### 7.2.1 Lineare Regression als Machine Learning Modell

Wir beginnen mit einem der einfachsten Modelle, der linearen Regression, welche Sie ja bereits in in Kapitel ?? kennengelernt haben. Lineare regression ist eine klassische Methode aus der Statistik. In der Statistik liegt der Fokus auf dem Verstehen, wie eine oder mehrere Variablen eine von diesen Variablen abhängige Grösse erklären. Im maschinellen lernen hingegen sind wir besonders an den Vorhersagen interessiert, die wir durch ein auf die Daten angepasstes Modell erhalten können. Darüberhinaus eignen sich lineare Regressionsmodelle, und die darauf aufbauenden linearen Basisfunktionsmodelle, ausgezeichnet, die Grundkonzepte des maschinellen Lernens zu illustrieren.

Gegeben ist ein Datensatz aus Eingabe/Ausgabepaaren

$$D = \{(x_1, y_1), \dots, (x_n, y_n)\} \subset \mathbb{R} \times \mathbb{R}$$

und ein Modell der Form:

$$f(x, a, b) = ax + b.$$

Der Hypothesenraum ist also der Raum der linearen Funktionen

$$\mathcal{H} = \{ax + b \mid a, b \in \mathbb{R}\}.$$

Der Fehler, den wir durch eine Vorhersage  $\hat{y} = f(x, a, b)$  machen, können wir durch das Quadrat des Unterschieds zwischen der Vorhersage und dem tatsächlichen Wert  $y$  quantifizieren. Die Verlustfunktion wird entsprechend wie folgt definiert:

$$L(y, \hat{y}) = L(y, f(x, a, b)) = (f(x, a, b) - y)^2.$$

Um die optimalen Parameter zu finden, minimieren wir die Verlustfunktion über alle Datenpaare  $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$ :

$$\min_{a, b} \sum_{i=1}^n L(y_i, f(x_i, a, b)).$$

Da die Verlustfunktion eine quadratische Funktion und das Modell linear in den Parametern  $a$  und  $b$  ist, können wir die optimalen Parameter durch das Lösen der Normalgleichung (Gleichung ??) bestimmen.

### 7.2.2 Lineare Basisfunktionsmodelle

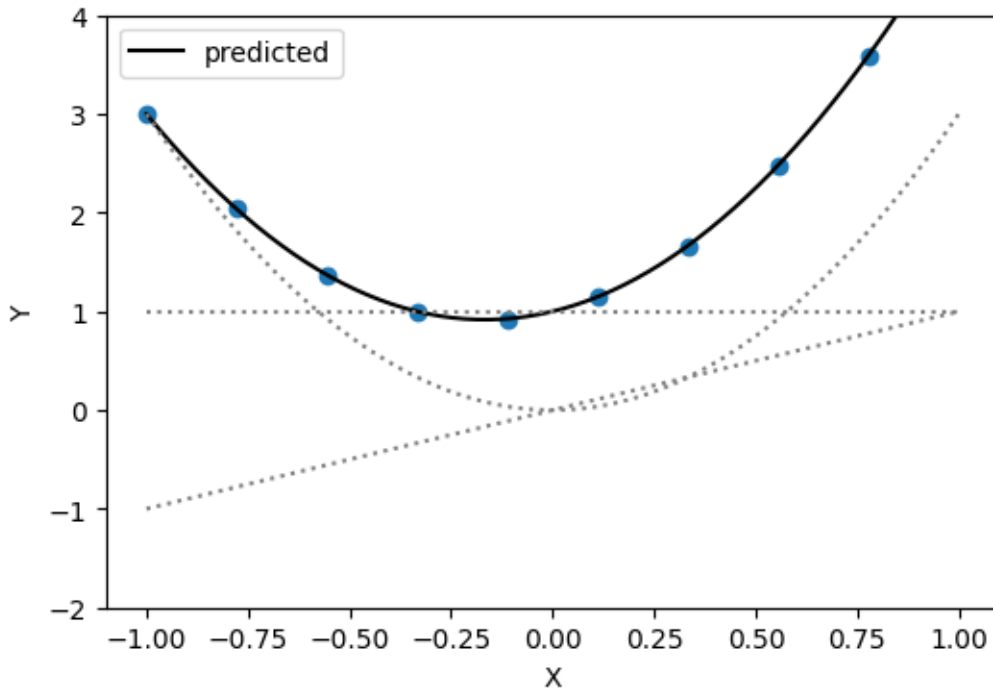
Der Zusammenhang zwischen den Merkmalen und der Vorhersage ist oft nicht linear. Ein einfacher, jedoch sehr flexibler Ansatz, um diese einfache linearen Regressionsmodelle zu erweitern, ist die Verwendung von Basisfunktionen. Auch dieses Modell kennen Sie bereits aus Kapitel ??, in welchem Sie als Basisfunktionen Polynome  $m$ -ter Ordnung verwendet haben. Neben Polynomen werden im Maschinellen lernen jedoch noch viele weitere Funktionen als Basisfunktionen verwendet. Ein lineares Basisfunktionsmodell hat die Form:

$$f(x, \mathbf{w}) = w_0 + \sum_{i=1}^m w_i \phi_i(x),$$

wobei  $\phi_i(x)$ ,  $i = 1, \dots, m$  die Basisfunktionen sind und der Vektor  $\mathbf{w} = (w_1, \dots, w_m)$  die dazugehörigen Parameter zusammenfasst. Da das Modell noch immer linear in den Modellparametern ist, lassen sich die Parameter wieder durch das Lösen der Normalgleichungen bestimmen. Jedoch führt die Transformation mit den Basisfunktionen dazu, dass die durch das Modell  $f(x, \mathbf{w})$  gemachten Vorhersagen nicht mehr nur lineare Kombinationen der Merkmale sind.

Eine Interpretation dieser Modelle ist, dass wir eine Eingabe  $x$  zuerst durch das Anwenden der Basisfunktionen  $\phi_1(x), \dots, \phi_m(x)$  transformieren, und dann in diesem  $m$ -Dimensionalen Raum eine lineare Regression durchführen. Die Basisfunktionen sollen also die Daten so transformieren, dass nach der Transformation ein einfacher linearer Zusammenhang zwischen den Merkmalen  $x_i$  und den Vorhersagen  $y_i$  besteht. Dies wird in Abbildung ?? illustriert. Die Daten wurden dabei anhand der Funktion  $f(x) = 3x^2 + s + 1$  generiert. Als Basisfunktionen werden  $\phi_0(x) = x^0$ ,  $\phi_1(x) = x$ ,  $\phi_2(x) = x^2$  verwendet. Die Vorhersage entspricht dann genau der gewichteten Summe dieser drei Basisfunktionen mit den Gewichten  $w_0 = 1$ ,  $w_1 = 1$ ,  $w_2 = 3$ .

Abbildung ?? illustriert eine etwas komplexere Situation. Die Daten (blaue Punkte) wurden anhand einer Sinusfunktion generiert, welche in Rot eingezeichnet ist, und dann mit einem Rauschen versehen. Für die Vorhersage wurden Polynomiale Basisfunktionen bis Grad 8 verwendet. Die Vorhersagen stimmen gut mit der in rot eingezeichnete wahren Funktion überein. Einen noch präziseren Fit an die Daten erhalten wir, wenn wir die Anzahl der Basisfunktionen erhöhen, also zum Beispiel Polynome bis Grad 30 als Basisfunktionen nehmen. Das Resultat ist in Abbildung



**Abbildung 7.1:** Lineares Basisfunktionenmodell: Die Daten (blaue Punkte) wurden anhand der Funktion  $f(x) = 3x^2 + s + 1$  generiert. Die grauen Linien zeigen die (gewichteten) Basisfunktionen  $\phi_0(x) = x^0$ ,  $\phi_1(x) = x$ ,  $\phi_2(x) = x^2$ . Die Vorhersage (schwarze Linie) entspricht genau der Summe dieser Basisfunktionen.

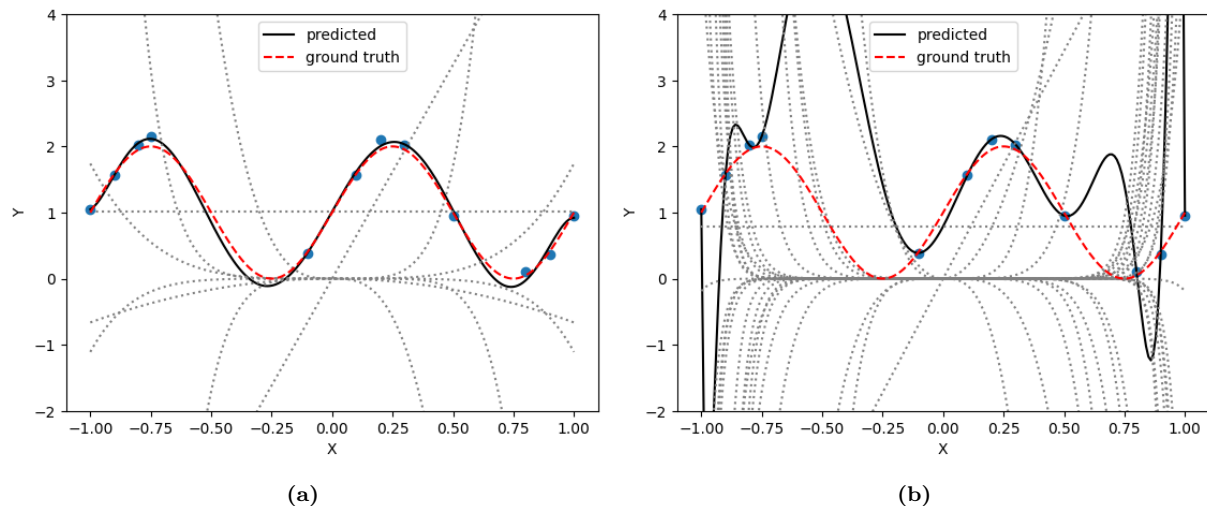
?? gezeigt. Die Daten werden nun perfekt gefittet, aber die Lösung entspricht nicht dem, was wir erwarten würden.

Dieses Beispiel zeigt mehrere grosse Probleme von polynomialen Basisfunktionsmodellen auf: Die Basisfunktionen sind global definiert, was bedeutet, dass sie über den gesamten Definitionsbereich definiert sind. Dies führt dazu, dass die Basisfunktionen in Bereichen, in denen keine Daten vorhanden sind, zu starken Ausschlägen führen können. Zudem haben die Basisfunktionen keinerlei Bezug zu den Daten. Ein weiteres Problem ist, dass die Anzahl der Basisfunktionen manuell festgelegt werden muss und nicht von den Daten bestimmt wird. Dies führt dazu, dass das Modell entweder zu einfach oder zu komplex sein kann.

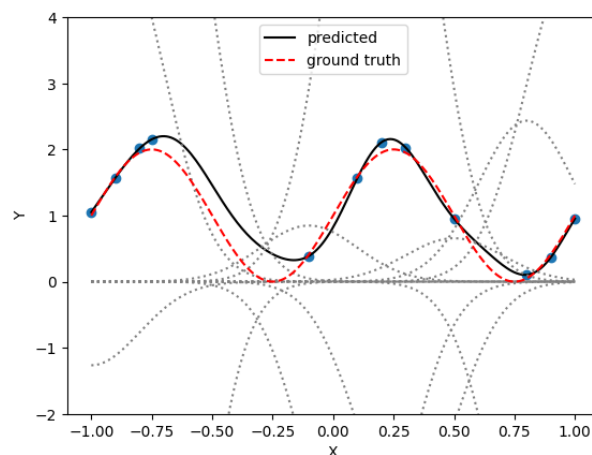
Wir können einen Teil dieser Probleme lösen, indem wir die Basisfunktionen lokal definieren und deren Position automatisch aus den Daten bestimmen. Eine verbreitete, lokale Basisfunktion ist die *Radiale Basisfunktion* (RBF), die wie folgt definiert ist:

$$\phi_i(x) = \exp\left(-\frac{1}{2} \frac{(x - \mu_i)^2}{s^2}\right).$$

Die Funktion entspricht von der Form her der unnormalisierten Dichtefunktion der Normalverteilung. Der Parameter  $\mu_i$  bestimmt die Position der Basisfunktion und  $s$  definiert deren Breite. Beachten Sie, dass der Wert von  $\phi_i$  gegen 0 geht, wenn die Distanz von Punkt  $x$  zu  $\mu_i$  gross wird. Wenn wir nun  $\mu_i$  so wählen, dass es die Position eines Datenpunktes ist, dann wird die Basisfunktion nur in der Nähe von diesem Datenpunkt gross sein und hat keinen Einfluss weit weg von den Datenpunkten. Das Modell besagt also, dass die Vorhersage eines neuen Punktes hauptsächlich lokal, von den observierten Werten der Punkte in der Nähe bestimmt wird. Wie gross der Einfluss ist, wird durch den Parameter  $s$  bestimmt. Da die Anzahl Basisfunktionen durch die Anzahl Datenpunkte bestimmt ist, wird auch die Komplexität des Modells automatisch höher, wenn mehr Daten zum Lernen zur Verfügung stehen. Abbildung ?? zeigt ein Beispiel für ein lineares Basisfunktionsmodell mit RBFs. Wir sehen, dass die Vorhersage deutlich besser ist, als für Polynome hohen Grads.



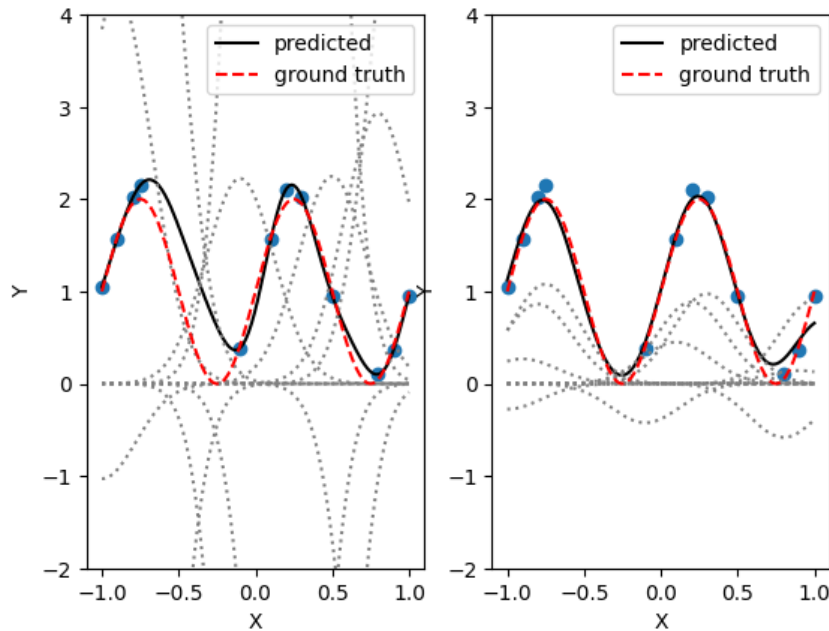
**Abbildung 7.2:** Zwei Beispiele für lineare Basisfunktionsmodelle mit Polynomialbasisfunktionen. ?? zeigt ein Modell mit Polynomen bis Grad 8 und ?? ein Modell mit Polynomen bis Grad 30. Die blauen Punkte zeigen die Daten, die rote Linie die wahre Funktion und die Schwarze Linie die Vorhersage des Modells.



**Abbildung 7.3:** Ein lineares Basisfunktionsmodell mit Radialbasisfunktionen. Die grauen, gestrichelten Linien zeigen die Basisfunktionen. Die rote Linie zeigt die wahre Funktion, mit der die Daten generiert wurden. Die schwarze Linie zeigt die Vorhersage des Modells.

### 7.2.3 Einschub: Regularisierung und Bayesianische Regression

Mit genügend Basisfunktionen können wir grundsätzlich alle Eingabedaten perfekt durch unser Modell interpolieren. Unser Ziel ist aber natürlich nicht, die Eingabedaten perfekt wiederzugeben, sondern das zugrundeliegende Muster zu finden, so dass wir gute Vorhersagen machen können. Wie in Abbildung ?? gezeigt können unterschiedliche Modelle die Daten ganz unterschiedlich erklären. Intuitiv bevorzugen wir die Erklärung links, da diese *einfacher* ist. Das Modell wurde so beschränkt, dass die Basis nur Polynome bis Grad 8 zulässt, während rechts Polynome bis Grad 30 verwendet wurden. Wir könnten versuchen, die Anzahl Basisfunktionen immer so gering wie möglich zu halten, und somit einfache Lösungen zu forcieren. Dies hätte aber den Nachteil, dass wir das Modell eventuell zu stark einschränken, und dem Modell nicht die Möglichkeit geben, komplexere Muster in den Daten zu erklären. Eine oft zu bevorzugende Alternative ist deshalb, das Modell so zu *Regularisieren*, dass auch mit vielen Basisfunktionen einfache Lösungen bevorzugt werden. Eine mögliche Umsetzung dieser Idee ist als *Ridge regression* bekannt. Hier wird ein Trade-off zwischen einer guten Anpassung an die Daten, gemessen von der Verlustfunktion, und einer einfachen Lösung, gemessen durch die Größe der Parameterwerte  $w$ , gesucht. Es wird also



**Abbildung 7.4:** Zwei RBF-Modelle mit unterschiedlichen Regularisierungsparametern. Links wurde der Regularisierungsparameter sehr klein gewählt. Entsprechend sind die Basisfunktionen sehr gross und das Modell ist sehr komplex. Rechts wurde der Regularisierungsparameter grösser gewählt. Die Basisfunktionen sind kleiner und das Modell ist einfacher.

folgendes Minimierungsproblem gelöst:

$$\min_{\mathbf{w}} \sum_{i=1}^n L(y_i, f(x_i, \mathbf{w})) + \lambda \|\mathbf{w}\|^2,$$

wobei  $\lambda$  ein zu wählender Parameter ist. Die Intuition hinter dieser Art von Regularisierung ist, dass bei kleineren Parameterwerte die Funktionswerte weniger schnell ändern können, und die Lösung somit *einfacher* ist. Abbildung ?? zeigt ein Beispiel für zwei RBF-Modelle mit unterschiedlichen Regularisierungsparametern.

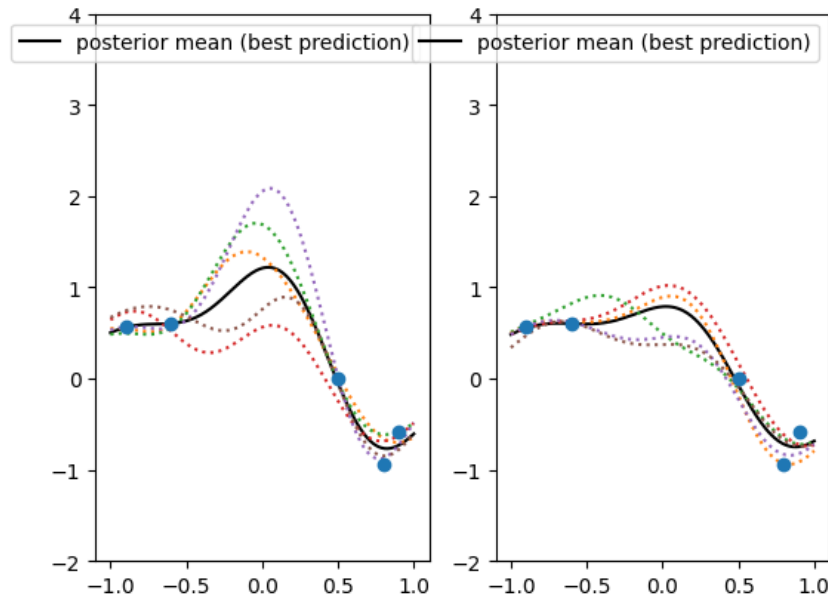
Eine ähnliche, aber noch mächtigere Möglichkeit, Eigenschaften von Lösungen zu definieren, bietet die *Bayesianische Lineare Regression*. Hier nehmen wir an, dass die Parameter  $w$  einer Normalverteilung  $w_i \sim N(0, \alpha)$  folgen. Auch dies hat den Effekt, dass kleinere Parameterwerte bevorzugt werden. Im Gegensatz zu Ridge Regression erlaubt diese Formulierung eine komplett probabilistische Behandlung des Problems, und wir erhalten nicht nur den besten Parameterwert, sondern die Lösung ist eine Wahrscheinlichkeitsverteilung  $p(\mathbf{w}|D)$  über gute Lösungen. Abbildung ?? zeigt ein Beispiel für eine Bayesianische lineare Regression.

Sowohl Ridge regression als auch Bayesianische lineare Regression haben einfache analytische Lösungen, die ähnlich der Normalgleichungen durch Linearen Gleichungssysteme gegeben sind. Die Details und ausführliche Diskussion dieser Methoden gehen aber über das Ziel dieser Vorlesung hinaus.

#### 7.2.4 Logistische Regression

Bisher haben wir uns nur mit Regressionsmodellen beschäftigt. Bei Regressionsmodellen sind die Vorhersagen reelle Zahlen. Eine wichtige Klasse von Problemen im Maschinellen Lernen sind aber Klassifikationsprobleme. Hier sind die Vorhersagen binäre Werte, also entweder 0 oder 1. Ein Beispiel ist die Vorhersage, ob ein Bild eine Katze oder einen Hund zeigt. In diesem Fall könnten wir Katzen mit dem Wert 0 und Hunde mit dem Wert 1 kodieren.





**Abbildung 7.5:** Ein Beispiel für eine Bayesianische Lineare Regression. Die schwarze Linie zeigt die beste Vorhersage des Modells. Die gestrichelten farbigen Linien zeigen zufällige Stichproben der Posteriorverteilung  $p(\mathbf{w}|D)$ . Beim Modell links wurde die Varianz für den Prior auf die Gewichte gross gewählt. Deshalb sind auch die möglichen Vorhersagen sehr unterschiedlich. Beim Modell rechts wurde die Varianz klein gewählt, und die möglichen Vorhersagen sind Entsprechend sehr viel enger verteilt.

Ein einfaches Modell, um binäre Vorhersagen zu machen, ist die *logistische Regression*. Im einfachsten Fall haben ist unsere Eingabe  $x$  ein Skalarwert,  $x \in \mathbb{R}$  und die Vorhersage  $y$  nimmt die Werte 0 oder 1 an. Logistische Regression versucht genau wie die lineare Regression, die Daten durch eine parametrische, lineare Funktion der Form  $ax + b$  zu erklären. Diese Funktion wird dann aber nicht direkt als Vorhersage verwendet. Stattdessen wird sie als Eingabe für die Logistische Funktion verwendet, die die Werte auf den Bereich  $[0, 1]$  abbildet. Die logistische Funktion ist wie folgt definiert:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Logistische Regression ist also ein Modell der Form:

$$f(x, a, b) = \sigma(ax + b).$$

Abbildung ?? illustriert die Situation. Die Datenpunkte nehmen  $y$ -Werte an, die entweder 0 oder 1 sind. Gelernt werden die Parameter einer linearen Funktion  $ax + b$ , die in grün eingezeichnet ist. Diese bestimmt die Werte der logistischen Funktion  $\sigma(x)$ , die in blau eingezeichnet ist und nach dem Trainieren des Modells gut an die Daten angepasst ist. Die Vorhersagen des Modells können als Wahrscheinlichkeit interpretiert werden, dass der Punkt  $x$  zur Klasse 1 gehört:

$$P(y = 1|X = x) = f(x, a, b).$$

Um die Klasse vorherzusagen, wird ein Schwellenwert  $t$  (z.B.  $t = 0.5$ ) gewählt. Wenn die Vorhersage grösser als  $t$  ist, wird die Klasse 1 vorhergesagt, andernfalls die Klasse 0.

**Loss Funktionen für die Klassifizierung** Wie oben besprochen benötigen wir für das Anpassen eines Modells an die Daten eine Verlustfunktion, welche die Differenz zwischen der Vorhersage des Modells  $\hat{y} = f(x, a, b)$  und dem tatsächlichen Wert  $y$  misst. Für die Regression haben wir die quadratische Verlustfunktion  $L(\hat{y}, y) = (f(x, a, b) - y)^2$  kennengelernt. Für die Logistische Regression, bei denen das Modell nur Vorhersagen muss, ob eine Eingabe  $x$  zu einer

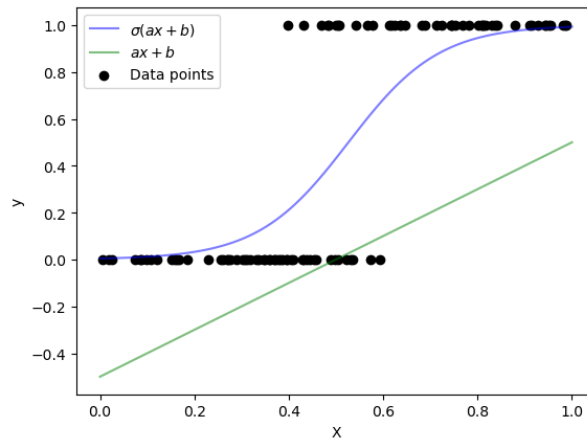


Abbildung 7.6: Logistische Regression in einem 1D Raum. .

bestimmten Klasse gehört, ist die Größe des Abstands meist nicht von Interesse. Stattdessen wollen wir nur messen, ob wir die dem Label  $y$  entsprechende Klasse gewählt haben.

Wenn  $y$  nur die Werte 0 oder 1 annehmen kann, verwenden wir dafür oft die *Cross Entropy Loss Funktion*. Die Cross Entropy Loss Funktion ist dann definiert als:

$$L(y, \hat{y}) = L(y, \sigma(f(x, a, b))) = -y \log(\sigma(f(x, a, b))) - (1 - y) \log(1 - \sigma(f(x, a, b)))$$

Dabei ist  $\sigma(x) = \frac{1}{1+\exp(-x)}$  die Sigmoidfunktion, die den Wert von  $f(x, a, b)$  auf das Intervall  $[0, 1]$  abbildet. Beachten Sie, dass der Verlust nur dann genau 0 wird wenn das Label mit der Vorhersage übereinstimmt. Ansonsten wird der Verlust positiv.

### Höher-dimensionale Eingaben und Basisfunktionen für die logistische Regression

Selbstverständlich können nicht nur 1D-Daten mittels logistischer Regression klassifiziert werden. Wie bei der linearen Regression können auch hier die Eingaben  $x$  mehrdimensional sein. In diesem Fall ist die Vorhersage des Modells

$$f(x, \mathbf{w}) = \sigma(w_0 + \sum_{i=1}^m w_i x_i),$$

wobei  $x_i$  die  $i$ -te Dimension der Eingabe  $x$  ist und  $w_i$  der entsprechende Parameter.

Während in dieser einfachen Form der logistischen Regression die Vorhersage immer noch linear in den Eingaben ist, können wir auch hier Basisfunktionen verwenden, um nicht-lineare Zusammenhänge zu modellieren.

Analog zum Fall der Regression, hat unser Modell hat dann die Form:

$$f(x, \mathbf{w}) = \sigma(w_0 + \sum_{i=1}^m w_i \phi_i(x)).$$

wobei  $\phi_i(x)$  die Basisfunktionen sind.

#### 7.2.5 Neuronale Netze

Wir haben gesehen, wie mir mit Basisfunktionsmodellen flexibel Modelle erstellen und damit auch nicht-lineare Zusammenhänge abbilden können. Durch die Wahl der Basisfunktionen sowie der Regularisierung können wir die Komplexität der Modelle steuern. Dank der linearen Struktur des Modells verfügen wir zudem eine einfache Möglichkeit, die optimalen Gewichte zu bestimmen.

Leider sind solche linearen Modelle für viele Anwendungen nicht flexibel genug. Wenn die Basisfunktionen nicht gut an die Charakteristiken der Daten angepasst sind, dann wird die Anzahl der Basisfunktionen, die benötigt werden um komplexe Zusammenhänge zu modellieren, oft zu gross. Dies ist insbesondere dann der Fall, wenn die Daten hochdimensional sind.

Eine Möglichkeit, dieses Problem zu umgehen, ist die Verwendung von neuronalen Netzen. Die Idee dahinter ist einfach: Statt im Voraus definierte Basisfunktionen zu verwenden, werden die Basisfunktionen selbst wieder parameterisiert, und die Parameter aus den Daten gelernt. Die Form der Basisfunktionen, die in diesem Kontext Neuronen, genannt werden, ist dabei wieder eine gewichtete Linearkombination aus den Eingaben. Wir definieren ein *Neuron*  $z(\mathbf{x}, \mathbf{w})$  mit Eingaben  $\mathbf{x} = (x_1, \dots, x_m)$  und Parameter  $\mathbf{w} = (w_0, \dots, w_m)$  wie folgt:

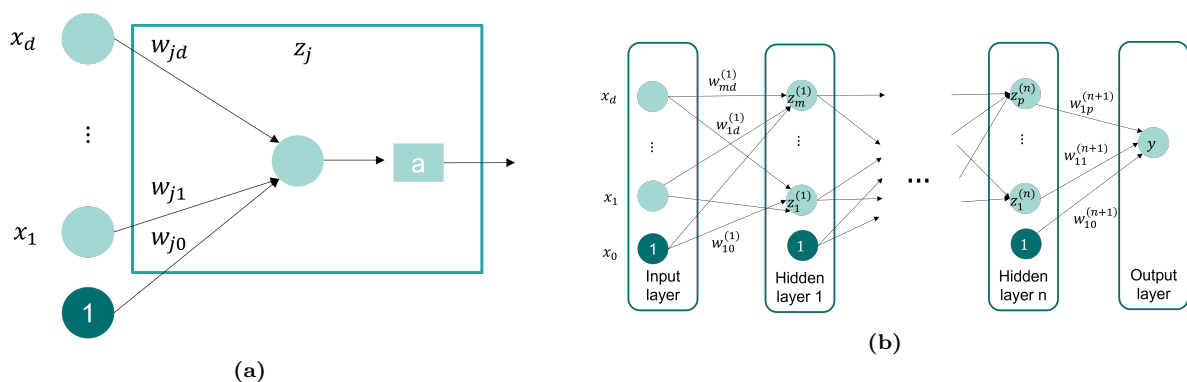
$$z(\mathbf{x}, \mathbf{w}) = a\left(\sum_{i=1}^m w_i x_i + w_0\right).$$

Damit auch nichtlineare Zusammenhänge repräsentiert werden können, wird auf das Resultat noch eine Aktivierungsfunktion  $a$  angewendet. Einige Beispiele von Aktivierungsfunktionen sind in Tabelle ?? aufgeführt.

|          |                                  |
|----------|----------------------------------|
| RELU     | $a(x) = \max(0, x)$              |
| $\sigma$ | $a(x) = \frac{1}{1+e^{-x}}$      |
| TANH     | $a(x) = \frac{1}{1+e^{-2x}} - 1$ |

**Tabelle 7.2.1:** Häufig verwendete Aktivierungsfunktionen.

Abbildung ?? illustriert den Aufbau eines Neurons grafisch. Wegen der nicht-linearen Aktivierungsfunktion können die optimalen Parameter eines Neurons nicht mehr analytisch über die Normalgleichung bestimmt werden. Stattdessen müssen wir die Gewichte des Modells iterativ über ein Verfahren wie das Gradientenabstiegsverfahren bestimmen. Dafür haben wir jetzt flexible Blöcke, mit denen wir komplexere Lernverhalten modellieren können. Wir können diese Neuronen nämlich nun hierarchisch anordnen, um ein *Multi-layer Neural Network* zu erhalten. Multi-layer Neural networks bestehen aus mehreren Schichten von Neuronen, wobei die Neuronen in jeder Schicht mit den Neuronen in der Nachfolgeschicht verbunden sind. Ein solches Netzwerk ist in Abbildung ?? dargestellt.



**Abbildung 7.7:** ??) Schematische Darstellung eines Neurons. ??) Ein Neuronales Netz mit  $n$  hidden Layer und einer einzigen Ausgabe  $y$ .

In unserer Interpretation von Neuronalen Netzen als Basisfunktionsmodelle, entsprechen die Neuronen in jeder Schicht den Basisfunktionen, mit denen die Eingaben für die Neuronen in der nächsten Schicht transformiert werden. Die Eingaben, die die letzte Schicht erreichen sollten nach erfolgreichem Lernen so transformiert sein, dass eine einfache Linearkombination der Eingaben das in den Daten erhaltene Muster erklären kann.

## 7.3 Training von Machine Learning Modellen

Mit linearen Basisfunktionen und neuronalen Netzen haben wir nun zwei Modelle kennengelernt, die uns erlauben, komplexe Zusammenhänge in den Daten zu modellieren. Wir haben aber auch schon gesehen, dass komplexe Modelle dazu neigen, die Daten zu *überfitten*. Das bedeutet, dass das Modell die Daten nicht nur gut erklärt, sondern auch das Rauschen in den Daten “lernt”. Das kann die Nützlichkeit des Modells für Voraussagen stark einschränken.

In der Praxis müssen wir sicherstellen, dass unser Modell nicht nur die Trainingsdaten gut erklärt, sondern auch auf neuen Daten gut funktioniert. Um dies zu erreichen, verwenden wir oft eine *Trainings-Validierungs-Test* Aufteilung. Die Trainingsdaten werden verwendet, um das Modell zu trainieren. Die Validierungsdaten werden verwendet, um die Hyperparameter des Modells zu optimieren. Die Testdaten werden verwendet, um die Performance des Modells zu evaluieren.

### 7.3.1 Hyperparameteroptimierung

Die Wahl der Hyperparameter ist ein wichtiger Schritt im Machine Learning. Hyperparameter sind Parameter, die nicht durch das Modell gelernt werden, sondern vor dem Training festgelegt werden müssen. Beispiele für Hyperparameter sind die Anzahl der Basisfunktionen in einem Basisfunktionsmodell, die Breite einer Radialen Basisfunktion oder die Anzahl der Schichten und Neuronen in einem neuronalen Netz.

Die Wahl der Hyperparameter ist oft nicht trivial und kann einen grossen Einfluss auf die Performance des Modells haben. Eine Möglichkeit, die Hyperparameter zu optimieren, ist die *Grid Search*. Dabei wird ein Gitter von Hyperparametern definiert, und für jede Kombination von Hyperparametern wird das Modell trainiert und auf den Validierungsdaten evaluiert. Die Hyperparameter, die die beste Performance auf den Validierungsdaten liefern, werden dann für das finale Modell verwendet.

### 7.3.2 Crossvalidation

Eine weitere Möglichkeit, die Performance des Modells zu evaluieren, ist die *Crossvalidation*. Dabei wird der Datensatz in  $k$  Teile aufgeteilt. Das Modell wird dann  $k$  mal trainiert und evaluiert, wobei jedes Mal ein anderer Teil des Datensatzes als Validierungsdaten verwendet wird. Die Performance des Modells wird dann als Durchschnitt der Performances über alle  $k$  Durchläufe berechnet.

## 7.4 Deep Learning

Die zurzeit unbestritten populärste Form von Machine Learning ist *Deep Learning*. Deep Learning ist eine spezielle Form von Machine Learning, die auf neuronalen Netzen basiert. Der Begriff deep learning kommt daher, dass die neuronalen Netze oft sehr tief sind, d.h. viele Schichten haben. Es gibt natürlich zahlreiche Möglichkeiten, ein neuronales Netz zu konstruieren. Je nach Anwendung kommen verschiedene Architekturen zum Einsatz. In diesem Abschnitt werden wir einige der wichtigsten Architekturen vorstellen.

### 7.4.1 Convolutional Neural Networks

Das Hauptziel von CNNs ist Bilderkennung. Wir möchten immer noch Objekte voneinander unterscheiden, aber statt skalarer Merkmale möchten wir jetzt Bilder betrachten. Als erste Idee könnten wir jeden Pixel als Eingang zu einem neuronalen Netz nehmen und ein riesiges Netzwerk verwenden, um die Klassifikation durchzuführen. Aber dafür bräuchten wir zu viele Neuronen. Stattdessen verwenden wir *convolution*<sup>1</sup>, um die Bilder zu vereinfachen. Für Faltung betrachten

---

<sup>1</sup>Convolution ist hier nicht im Sinne von *Complex* (convoluted), sondern im Sinne von *Faltung* zu verstehen.

wir Bilder als Matrizen. Sei  $E \in \mathbb{R}^{n \times n}$  die zu faltende Matrix und  $F \in 0,1^{2 \times 2}$  die Faltungsmatrix. Der Ausgang ist  $A \in \mathbb{R}^{n-1 \times n-1}$ .<sup>2</sup>

$$A_{i,j} = E_{i,j}F_{1,1} + E_{i+1,j}F_{2,1} + E_{i,j+1}F_{1,2} + E_{i+1,j+1}F_{2,2} \quad (7.4.1)$$

Man stellt dies am besten grafisch dar: Man legt die Faltungsmatrix auf jeden 2x2 Block in  $E$ , multipliziert die Einträge, die aufeinander liegen, summiert die Ergebnisse, und notiert die Summe auf einer neuen Matrix für jeden 2x2 Block in  $E$ .

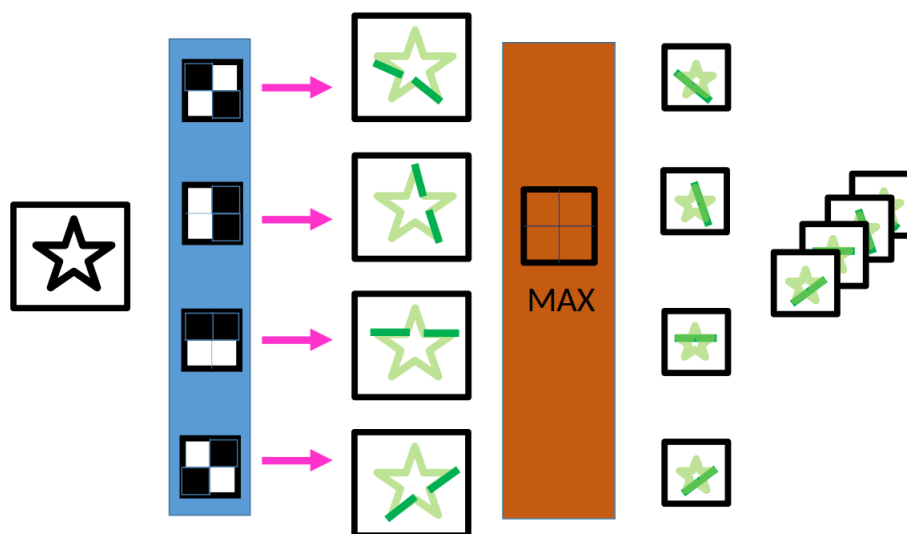
Man kann noch einen Filter (Aktivierungsfunktion) auf das Ergebnisse anwenden. Zum Beispiel:

$$A_{i,j} = \text{RELU}(E_{i,j}F_{1,1} + E_{i+1,j}F_{2,1} + E_{i,j+1}F_{1,2} + E_{i+1,j+1}F_{2,2}) \quad (7.4.2)$$

Eine weitere Methode, die für CNN verwendet wird, ist *Pooling*. Es wird ähnlich wie Faltung angewendet, aber es keine lineare Funktion, sondern *max* oder *min*:

$$A_{i,j} = \max\{E_{i,j}; E_{i+1,j}; E_{i,j+1}; E_{i+1,j+1}\} \quad (7.4.3)$$

Eine geschickte Wahl von Faltungsschichten führt nicht nur zu einer Reduktion der Bilddimension, sondern auch zu Hervorhebung von Merkmalen (Kanten, Ecken etc.) des gesuchten Objekts (siehe Abb. ??). Man kann die "gefalteten" Bilder dann mithilfe eines neuronalen Netzes klassifizieren.



**Abbildung 7.8:** Hier werden die Kanten von einem Stern mit einem CNN hervorgehoben. Die nachfolgenden neuronalen Netze können dann aus den Kanten entscheiden, ob es einen Stern ist.

## 7.4.2 Andere Architekturen

### 7.4.2.1 Autoencoder

Ein Autoencoder besteht aus einem Encoder und einem Decoder. Der Encoder überführt die Eingangsinformation (Bild oder Data) in einen mehrdimensionalen Raum (auch Code Space genannt). Der Decoder ist die umgekehrte Operation. Die Idee ist, dass ähnliche Eingänge näher zu einander im Code Space abgebildet werden. Dann können wir mithilfe von Clustering die Daten klassifizieren. Ein Beispiel ist Schrifterkennung. Bilder von Schriftzeichen werden mithilfe

<sup>2</sup>Falls  $n$  kein ganzzahliges Mehrfaches von der Größe von  $F$  ist, dann vernachlässigt man genügend der letzten Zeilen und Spalten von  $E$ .

von einem Encoder auf dem Code Space abgebildet. Dann können wir aus der Position im Code Space erkennen, welches Schriftzeichen es war.

Der Encoder und Decoder sind neuronale Netze, die auch trainiert werden müssen. Die Layer sind Sanduhr artig, dies führt dazu, dass nur die wichtigste Information im Code Space abgebildet wird. Was diese Information (bzw. was die Achsen von Code Space) bedeuten ist a priori unbekannt. Die Verlustfunktion für Training ist:

$$L(D, Enc, Dec) = \sum_{x \in D} ||x - Dec(Enc(x))||^2 \quad (7.4.4)$$

### 7.4.3 Generative Adversarial Networks (GAN)

GANs sind dafür geeignet, um Bilder zu generieren oder zu verändern. Ein GAN hat einen Generator, der Bilder generieren kann und einen Diskriminator, der Bilder beurteilen kann. Der Diskriminator versucht die Bilder vom Generator von echten Bildern unterscheiden zu können. Der Generator versucht Bilder, die vom Diskriminator als "echt" gesehen werden, zu generieren.

## 7.5 Kodierung kategorischer Daten

Wie wir gesehen haben, arbeiten alle bisherige Modelle auf numerischen Daten (entweder 0,1 oder  $\mathbb{R}$ ), aber im reellen Leben sind Daten oft nicht numerisch, sondern kategorisch <sup>3</sup>. Deshalb müssen wir uns bewusst eine Codierung auswählen. In dem folgenden Paragrafen betrachten wir verschiedene Methoden anhand des Titanic-Dataset (siehe Abbildung ??).

### 7.5.1 Ordinal Encoding

Dies ist die einfachste Methode. Mögliche Kategorien jedes Merkmals werden mit arbiträren ganzen Zahlen bezeichnet. Zum Beispiel im Titanic-Dataset können wir für das Merkmal *class* Crew mit 1, First mit 2 und Second mit 3 bezeichnen. Obwohl wir die Zahlen willkürlich gewählt haben, haben wir plötzlich Crew näher mit First als mit Second platziert. Das kann in der Auswertung nachteilig sein.

### 7.5.2 Mean Encoding

Hier bezeichnen wir jede Kategorie mit einem Mean, der mit Hilfe von dem *class label* generiert wird. In unserem Fall können wir dieses Mean als Überlebenschance nehmen. Für *Age* Merkmal bezeichnen wir *Kind* mit 1, denn jedes Kind hat überlebt, und *Adult* mit 0.4, denn 40 % der Erwachsene haben überlebt. Nachteil ist, dass wir plötzlich Information aus dem *class label* in die Merkmale übertragen haben. Dies kann zu Overfitting führen.

### 7.5.3 One-Hot Encoding

Für diese Methode machen wir aus einem Merkmal mit  $n$  Kategorien  $n$  Merkmale, d.h. z. B. für das Merkmal *class*: Es gibt jetzt drei Merkmale mit Werten 0,1. Diese sind "Ist Crew?", "Ist First?" und "Ist Second?". Dadurch haben wir keinen der Nachteile der vorherigen Methoden, aber unser Datensatz ist *viel*<sup>4</sup> größer geworden.

<sup>3</sup>z.B Gender wird oft mit männlich, weiblich oder divers bezeichnet. Diese lassen sich aber nicht natürlicherweise mit Zahlen ausdrücken.

<sup>4</sup>Der Titanic-Datensatz hat nach One-Hot Encoding ca. 2 mal mehr Merkmale.

| Class  | Sex | Age   | Survived? |
|--------|-----|-------|-----------|
| Crew   | F   | Adult | Y         |
| Crew   | F   | Adult | Y         |
| First  | M   | Adult | N         |
| First  | M   | Child | Y         |
| Second | F   | Adult | N         |
| Second | M   | Child | Y         |
| Second | M   | Adult | N         |

Abbildung 7.9: Auszug des Titanic Datasets.

## 7.6 Clustering & K-Means-Methode

Wir möchten unsere Daten mit skalaren Merkmalen (z.B. Helligkeit und Größe für Sterne) gemäß ihrer Entfernung zueinander in Clusters (Bündel) sammeln. Die k-Means-Methode dient dazu, die geeigneten Bündel zu finden. Unser Modell ist

$$\mathcal{H} = \left\{ (\theta_1, \dots, \theta_k, c) \mid \theta_1, \dots, \theta_k \in \mathbb{R}^d \wedge c : \mathbb{R}^d \rightarrow \{1, \dots, k\} \right\} \quad (7.6.1)$$

wobei wir  $k$  Bündel mit Mittelpunkt (auch Zentroid benannt)  $\theta$  und die Zuweisungsfunktion  $c$  betrachten. Die Funktion  $c$  gibt für jeden Punkt in  $\mathbb{R}^d$  an, zu welchem Bündel dieser Punkt gehört. Zu bemerken ist, dass dies kein neuronales Netz ist. Die Verlustfunktion ist so gedacht, dass der Abstand der Punkten zu ihrem zugewiesenen Cluster-Mittelpunkt bestraft wird. Wir formalisieren dies wie folgt:

$D \subseteq \mathbb{R}^d$  sind die gegebenen Punkte,  $\{\theta_1, \dots, \theta_k\} \subseteq \mathbb{R}^d$  die Cluster-Mittelpunkte,  $c$  die Zuweisungsfunktionen. Dann ist die Verlustfunktion:

$$\mathcal{L}(D, \theta_1, \dots, \theta_k, c) = \sum_{x \in D} \|x - \theta_{c(x)}\|^2 \quad (7.6.2)$$

Bemerkung:  $\theta_{c(x)}$  ist der  $x$  zugewiesene Mittelpunkt.

Für das Training verwenden den folgenden Algorithmus:

- **Gegeben:**  $D \in \mathbb{R}^d$
- **Initialisierung:** wähle  $\theta_1, \dots, \theta_k$  zufällig.
- **Zuweisung aktualisieren:** wir setzen  $c(x) = \operatorname{argmin}_{j \leq k} \|x - \theta_j\|$ . Dies bedeutet, dass jedem Punkt der nächstgelegene Cluster-Mittelpunkt zugewiesen wird.
- **Cluster-Mittelpunkte aktualisieren:** Sei  $S_j = \{x \in D \mid c(x) = j\}$  die Menge der Punkte in Cluster  $j$ .  $\forall j \leq k$  setze  $\theta_j = \frac{1}{|S_j|} \sum_{x \in S_j} x$ . Somit haben wir alle Cluster-Mittelpunkte auf den wirklichen Mittelpunkt des Clusters gesetzt.

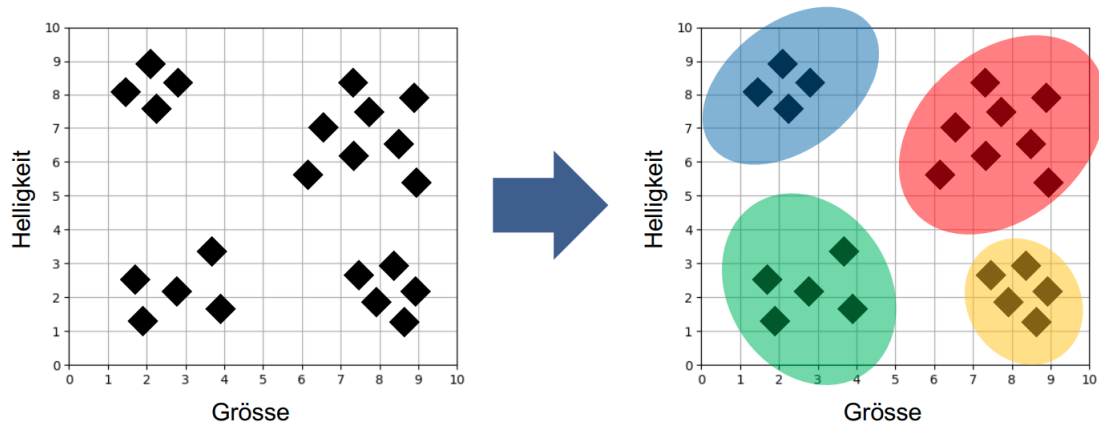


Abbildung 7.10: Graphische Vorstellung des Clustering.

- Führe die Aktualisierung-Schritte immer wieder durch, bis die Clusters sich nicht mehr ändern.

### 7.6.1 Overfitting

Wenn wir  $k$  zu gross wählen (z.B  $k \geq |D|$ ), dann ist die beste Zuordnung immer einen Punkt pro Cluster. Wir können dies nicht mit Cross-Validation (siehe ??) bemerken, denn  $\mathcal{L}$  strebt gegen 0 für  $k$  gegen  $|D|$ . Wenn wir mit Cross-Validation Overfitting verhindern wollen, dann müssen wir die Verlustfunktion so verändern, sodass sie auch große  $k$  bestraft. Wir verwenden mit  $\lambda > 0$ :

$$L = \mathcal{L} + \lambda e^k \quad (7.6.3)$$

Jetzt können wir Cross-Validation verwenden, um die optimale Anzahl Clusters zu finden.

## 7.7 Principal Component Analysis (PCA)

Als Letztes betrachten wir die PCA, was dafür gedacht ist, die Dimension (Anzahl Merkmale) ohne große Verluste von Information zu reduzieren. Dies ist nützlich, weil so unsere anderen Modelle schneller und korrekter funktionieren, wobei besonders wichtig ist, dass sie korrekter werden. Mit einer hohen Anzahl von Merkmalen wird der Suchraum  $\mathbb{H}$  größer und es kann sein, dass wir nicht die perfekte Lösung finden.

Formell ist PCA eine Methode unseren Datensatz  $\{x_1, \dots, x_n\} \subseteq \mathbb{R}^{D^5}$  auf einem reduzierten Datensatz  $\{t_1, \dots, t_n\} \subseteq \mathbb{R}^d$  abzubilden. Aus Lineare Algebra Perspektive ist die PCA eine Abbildung auf einen Unterraum. Wir möchten allerdings so viel Information wie möglich behalten. Die Abbildung wird  $\forall i \leq n$  folgenderweise definiert:

$$t_i = (x_i \cdot w_1, \dots, x_i \cdot w_d)^T \quad (7.7.1)$$

wobei  $\{w_1, \dots, w_d\} \subseteq \mathbb{R}^D$  die Gewichtsvektoren sind.

**Fall 1:  $d=1$**

In diesem Fall suchen wir nur die  $w_1$ , sodass  $t_i = (x_i \cdot w_1) \in \mathbb{R}$  die Streuung  $\sum t_i^2$  maximiert. Sei  $\mathbf{X}$  die  $n \times D$  Matrix, die in jeder Zeile ein Element aus dem Datensatz hat.

<sup>5</sup>Die  $x_i$ 's sind die Zeilen des Datensatzes und  $D$  ist die Anzahl von Merkmalen.



$$X = \begin{pmatrix} - & x_1 & - \\ - & x_2 & - \\ & \cdot & \\ & \cdot & \\ - & x_n & - \end{pmatrix} \quad (7.7.2)$$

Dann können wir  $w_1$  mit Hilfe folgender Gleichung finden:

$$w_1 = \operatorname{argmax}_{||w||=1} \left\{ \sum_i^n (x_i \cdot w_i)^2 \right\} = \operatorname{argmax}_{||w||=1} \{ ||\mathbf{X}w||^2 \} \quad (7.7.3)$$

Die Ausdruck  $||\mathbf{X}w||^2$  wird maximal, wenn  $w$  der Eigenvektor  $u_1^*$  zum größten Eigenwert  $\lambda_1^*$  ist. Dies kann mit Methoden aus der linearen Algebra finden.

**Fall 2:**  $d \geq 1$

In diesem Fall suchen wir  $d$  Gewichtsvektoren.  $w_1$  ist wieder wie in Fall 1 zu finden, für die  $w_2$  subtrahieren wir die Projektion von  $x$  auf den Unterraum  $\operatorname{Spann}[w_1]$  von  $x$  für jedes  $x \in X$  also:

$$X_1 = \{x - \operatorname{proj}_{u_1^*} x | x \in X\} = \{x - (x \cdot u_1^*)u_1^* | x \in X\} \quad (7.7.4)$$

Dann können wir noch einmal die Methode von Fall 1 (aber jetzt mit  $X_1$  als Datensatz) anwenden, um  $w_2$  zu bestimmen. Folglich können wir mit mehrmaligem Anwenden ( $w_1, \dots, w_d$ ) bestimmen. Formell geschrieben:

$$\mathbf{X}_k = \mathbf{X} - \sum_{s=1}^{k-1} \mathbf{X}w_s w_s^T \quad (7.7.5)$$

$$w_k = \operatorname{argmax}_{||w||=1} \{ ||\mathbf{X}_k w||^2 \} \quad (7.7.6)$$

Die Gleichung ?? ist äquivalent zu mehrmaligen Anwenden von Gleichung ??.



# Kapitel A

## Anhang

### SI-Präfixe

| Präfix | Symbol | Potenzdarstellung | Dezimaldarstellung                |
|--------|--------|-------------------|-----------------------------------|
| Yotta  | Y      | $10^{24}$         | 1'000'000'000'000'000'000'000'000 |
| Zetta  | Z      | $10^{21}$         | 1'000'000'000'000'000'000'000'000 |
| Exa    | E      | $10^{18}$         | 1'000'000'000'000'000'000'000'000 |
| Peta   | P      | $10^{15}$         | 1'000'000'000'000'000'000'000'000 |
| Tera   | T      | $10^{12}$         | 1'000'000'000'000'000'000'000'000 |
| Giga   | G      | $10^9$            | 1'000'000'000'000'000'000'000'000 |
| Mega   | M      | $10^6$            | 1'000'000'000'000'000'000'000'000 |
| Kilo   | k      | $10^3$            | 1'000'000'000'000'000'000'000'000 |
| Hekto  | h      | $10^2$            | 100                               |
| Dezi   | d      | $10^{-1}$         | 0.1                               |
| Centi  | c      | $10^{-2}$         | 0.01                              |
| Milli  | m      | $10^{-3}$         | 0.001                             |
| Micro  | $\mu$  | $10^{-6}$         | 0.000001                          |
| Nano   | n      | $10^{-9}$         | 0.000000001                       |
| Pico   | p      | $10^{-12}$        | 0.000000000001                    |
| Femto  | f      | $10^{-15}$        | 0.000000000000001                 |
| Atto   | a      | $10^{-18}$        | 0.000000000000000001              |
| Zepto  | z      | $10^{-21}$        | 0.000000000000000000001           |
| Yocto  | y      | $10^{-24}$        | 0.000000000000000000000001        |